claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a371aeb51e76016e0.jsonl`
- SHA-256: `bf947a38764eee367832bd0712398a5cc1d878c2f4498b0f0f0af7993e8bf94e`
- Source modified: `2026-07-02T02:55:42+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T02:47:11.978Z)

CONTEXT (read carefully):
- You are one auditor in a multi-agent tech-debt/critical-fix audit of the khelsutra-guru multi-repo workspace at C:/Users/avidu/Projects/khelsutra-guru. Today is 2026-07-02. All git repos were just pulled to current origin master/main.
- Repos: badminton-highlight-indexer (the ENGINE, Python, ~1500 tests, org Khelsutra), khelsutra (SPA web/ + marketing site/, TS/React, org avidullu), sports-data-collector, rally-annotator, sports-obsreport (small Python repos). Plus two NON-git dirs: "Annotation Setup", khelsutra-screencast.
- HARD RULES: READ-ONLY. Do NOT modify/create/delete any file, do NOT create branches/worktrees, do NOT pip install, do NOT run the full test suite (CI is trusted on master). You MAY run read-only git and gh commands (gh is authenticated), ruff/mypy IF already installed, and fast read-only scripts.
- EVERY claim must cite file:line (or a gh URL). A claim without file:line evidence is a hypothesis - mark it as such or drop it. The codebase drifts between sessions: re-verify anything you believe from docs against actual code.
- OWNER-GATED areas (flag as owner_gated=true, still report): alpha/serving go-live + Cloudflare dashboard work, Studio P10/P11 (corpus promotion + train/reeval), product/strategy/licensing decisions, real GPU/cloud spend, counsel ToS/DPA, repo rename off "badminton".
- Known-open items from project memory (VERIFY current state, don't trust blindly): engine PR #390 (D1 split main.py, open since 06-26) · audit doc docs/CODE_CLEANUP_AUDIT_2026-06-24.md remaining items D1, D13 (ByteTrack->trackers), Phase-3 (B3/B4/B6/B7/B8/D5/D6/D8-D10/O3/O4) · docs/DECISION_SNAPSHOT_REGRESSION.md next=P1 · docs/GOLDEN_REGRESSION_FIXTURES.md backlog M4/P0-rename/O1/O2 · engine issues #340 (Gemini re-bill), #332 (NSFW preflight), #349 (CPU-bound decode), #384 (CI single-source) · khelsutra issues #97-#114 UX/alpha backlog · DO droplet RETIRED 2026-06-25 (docs may still reference it; api.khelsutra.guru CNAME/CF-Access cleanup was pending).
- Categorize each finding: critical-fix (latent bug/correctness/data-loss/cost-leak), security, tech-debt (structure/duplication/dead code), test-gap, ci-infra, doc-rot, hygiene.
- Severity: P0 = latent bug or cost/data/security risk reachable in normal use; P1 = high-value debt blocking velocity or a stale-open thread; P2 = worthwhile cleanup; P3 = nice-to-have (report max 3 of these).
- Be selective: max ~12 findings, each genuinely worth acting on. Note tracked_in when an issue/PR/doc already tracks it (tracked is NOT done - still report). Your final message is data for an orchestrator, not prose for a human.

YOUR DIMENSION: latent correctness bugs in the ENGINE's hot paths. Repo: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer.
Hunt real, reachable bugs (not style) in: backend/main.py (the god module - API + job worker + orchestration), backend/worker/, backend/compute/, backend/storage/ (incl. gcs.py, gdrive_api.py), backend/api/, backend/flywheel.py, backend/config/ (loader/startup/presets), backend/utils/workspace.py. Look for: race conditions in the async job plumbing, error paths that swallow/mislabel failures, resource leaks (temp dirs, ffmpeg processes, GCS handles), cost-leak paths (Gemini re-billing, e.g. issue #340 - verify at file:line whether /api/process still

re-runs the paid segmenter for an already-processed video), schema-migration hazards (PRAGMA user_version ladder), Windows-vs-POSIX path bugs. Each finding needs a concrete failure scenario.

2. user (2026-07-02T02:47:20.321Z)

```
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
__pycache__
annotations
api
assets
audio
billing.py
compute
config
detectors
eval
features
flywheel.py
infrastructure
main.py
pipeline
providers
reporting
results.py
sports
stitcher
storage
tools
ui
utils
worker
```

3. user (2026-07-02T02:47:21.153Z)

```
backend\eval\__init__.py
backend\utils\__init__.py
backend\tools\__init__.py
backend\eval\audio\__init__.py
backend\pipeline\audio\__init__.py
backend\features\__init__.py
backend\utils\hashing.py
backend\pipeline\personalization\__init__.py
backend\api\__init__.py
backend\config\__init__.py
backend\config\presets.py
backend\pipeline\segmenters\__init__.py
backend\pipeline\validators\__init__.py
backend\results.py
backend\sports\base.py
backend\utils\redact.py
backend\worker\__init__.py
backend\eval\gemini_refine.py
backend\annotations\automated_provider.py
backend\annotations\base.py
backend\annotations\file_provider.py
backend\annotations\__init__.py
backend\api\auth.py
backend\api\job_worker.py
backend\billing.py
backend\compute\base.py
backend\compute\cloud_run.py
backend\compute\__init__.py
```

backend\config\loader.py
backend\config\startup.py
backend\eval\ablation.py
backend\eval\ablation_pdf.py
backend\eval\annotations.py
backend\eval\audio\e1_probe.py
backend\eval\batch.py
backend\eval\calibrate_local.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\classifier.py
backend\eval\distill_local.py
backend\eval\eval_partial_labels.py
backend\eval\eval_stats.py
backend\eval\experiment.py
backend\eval\export_wasb_segments.py
backend\eval\fusion_audio.py
backend\eval\fusion_compare.py
backend\eval\fusion_golden.py
backend\eval\golden_fixtures.py
backend\eval\golden_real_fixtures.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\eval\metrics.py
backend\eval\per_user_eval.py
backend\eval\per_user_gate.py
backend\eval\per_user_regression.py
backend\eval\promotion.py
backend\eval\rally_seq_proto.py
backend\eval\regression.py
backend\eval\score_calibration.py
backend\eval\segmentation_metrics.py
backend\eval\serve_contrast.py
backend\eval\splits.py
backend\eval\tolerance_metrics.py
backend\eval\training.py
backend\features\rally_seq.py
backend\flywheel.py
backend\infrastructure\database.py
backend\infrastructure\execution_policy.py
backend\pipeline\audio\hit_detector.py
backend\pipeline\detectors\audio_features.py
backend\pipeline\detectors__init__.py
backend\pipeline\detectors\court_gate.py
backend\pipeline\detectors\device.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\native_wasb_runner.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\detectors\stub_runner.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\personalization\serving.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters_keyhint.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\stitcher\compiler.py
backend\pipeline\stitcher\watermark_i18n.py
backend\pipeline\validators\base.py
backend\pipeline\validators\blur.py
backend\pipeline\validators\format.py
backend\pipeline\validators\pts_continuity.py

```
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\scene_cut.py
backend\providers\base.py
backend\providers\gemini.py
backend\providers\__init__.py
backend\reporting\harvest.py
(Showing 100 of 138 matching files; 38 more are not listed. Narrow the pattern or path to see
the rest.)
```

4. user (2026-07-02T02:47:32.380Z)

```
  2198 backend/main.py
   263 backend/api/job_worker.py
   177 backend/flywheel.py
   121 backend/utils/workspace.py
  1259 backend/infrastructure/database.py
   228 backend/compute/cloud_run.py
    57 backend/compute/base.py
  4303 total
backend/api:
__init__.py
__pycache__
auth.py
job_worker.py
static
uploads.py

backend/config:
__init__.py
__pycache__
loader.py
models.py
presets.py
startup.py

backend/storage:
__init__.py
__pycache__
base.py
capabilities.py
gcs.py
gdrive.py
gdrive_api.py
local.py
ref.py
s3.py

backend/worker:
__init__.py
__main__.py
__pycache__
```

5. user (2026-07-02T02:47:38.370Z)

```
<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1224 of 2199 total (38163 tokens,
cap 25000). Call Read with offset=1225 limit=1224 for the next page, or Grep to find a specific
section. Do NOT answer from this page alone if the answer may be further in the file.]</system-
reminder>

1       import argparse
2       import csv
3       import json
4       import logging
5       import math
```

```
6       import os
7       import shutil
8       import socket
9       import sys
10      import tempfile
11      import threading
12      import uuid
13      import webbrowser
14      from importlib.metadata import PackageNotFoundError
15      from importlib.metadata import version as _pkg_version
16      from urllib.parse import quote
17
18      import uvicorn
19      from dotenv import load_dotenv
20
21      # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
22      # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
23      # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
24      # stdlib-only import ? keeps this pre-pydantic line light.
25      from backend.utils.app_home import env_file_to_load, resolve_output_dir
26
27      load_dotenv(env_file_to_load())
28
29      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
30      logging.basicConfig(
31          level=logging.INFO,
32          format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
33          datefmt="%H:%M:%S",
34      )
35      logger = logging.getLogger(__name__)
36      from typing import Any, Dict, List, Optional, Tuple
37
38      from fastapi import FastAPI, HTTPException, Request
39      from fastapi.middleware.cors import CORSMiddleware
40      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
41      from fastapi.staticfiles import StaticFiles
42      from pydantic import BaseModel
43      from starlette.middleware.trustedhost import TrustedHostMiddleware
44
45      from backend import (
46          billing,  # M8: per-job cost ledger (USD internal / INR display)
47          storage,  # M2: URI-aware input acquisition (local = identity passthrough)
48      )
49      from backend.api import (
50          auth as edge_auth,  # M9: Cloudflare Access edge-auth (default-OFF)
51      )
52      from backend.config import (
53          AppConfig,
54          StartupCheckError,
55          StitchingConfig,
56          enforce_startup_checks,
57          write_light_default_config,  # P2b: batteries-included first-run config seeding
58      )
59      from backend.config import (  # new typed config
60          load_config as load_app_config,
61      )
62      from backend.infrastructure.database import Database
63      from backend.pipeline.segmenters.base import segmenter_registry
64      from backend.pipeline.stitcher.compiler import VideoCompiler
65      from backend.pipeline.stitcher.watermark_i18n import localize_watermark
66      from backend.pipeline.validators.base import registry
```

```
67      from backend.providers import provider_registry
68      from backend.reporting import emit_indexer_report
69      from backend.results import Failure  # standardized error/result contract
70      from backend.storage.capabilities import (
71          storage_capabilities,  # P3: secret-free "which media can this box use" report
72      )
73      from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
74          ffmpeg_bin,
75          ffprobe_bin,
76      )
77      from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
78      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
79      from backend.utils.paths import (
80          output_base,
81          resolve_under_root,
82          safe_output_filename,
83          validate_input_path,
84      )
85      from backend.utils.redact import (
86          redact_secrets,  # P0b: mask secret-shaped fields in /api/config
87      )
88      from backend.utils.single_instance import acquire_lock  # P0e: one instance per database
89
90      app = FastAPI(title="Sports Highlight Indexer API")
91
92
93      def create_app(config: AppConfig, db: Database) -> FastAPI:
94          """Factory: inject shared state via ``app.state`` so route handlers access it
through
95          ``request.app.state`` ? eliminating the Optional module-level globals that forced
96          ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.
97
98          Also sets the module-level ``global_config`` / ``db_instance`` for transitional
99          compatibility (helpers and tests still reference them). Route handlers use
100         ``request.app.state`` exclusively."""
101         global global_config, db_instance
102         global_config = config
103         db_instance = db
104         app.state.config = config
105         app.state.db = db
106         return app
107
108
109     def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
110         """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
111         default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
112         does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
113         src = os.environ if env is None else env
114         raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
115         origins = [o.strip() for o in raw.split(",") if o.strip()]
116         return origins or ["*"]
117
118
119     # CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by
browsers per the
120     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
121     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
122     # lock it to its origin; the middleware is fixed at startup.
123     app.add_middleware(
124         CORSMiddleware,
```

```
        allow_origins=_cors_origins_from_env(),
        allow_credentials=False,
        allow_methods=["*"],
        allow_headers=["*"],
    )


    def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
        """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A malicious
        web page can point a domain it controls at 127.0.0.1 and script requests to the local server;
        the browser sends ``Host: attacker.com``, which this rejects (the server only answers loopback
        Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the default keeps
        Starlette's TestClient host ``testserver`` so the in-process suite isn't broken. Read from the
        ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a routable/edge
        deploy sets its public host (or ``*`` when fronted by an auth gate that validates the host)."""
        src = os.environ if env is None else env
        raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
        hosts = [h.strip() for h in raw.split(",") if h.strip()]
        # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's TrustedHostMiddleware
        # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[::1]:port"), so IPv6
        # loopback is NOT supported by this guard ? access the appliance via 127.0.0.1/localhost.
        return hosts or ["localhost", "127.0.0.1", "testserver"]


    # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are honoured
    # (a remote page can't read responses cross-origin AND its Host header is rejected here). www_redirect
    # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
    app.add_middleware(
        TrustedHostMiddleware,
        allowed_hosts=_allowed_hosts_from_env(),
        www_redirect=False,
    )


    def _resolve_bind_host(
        config: Optional[AppConfig] = None, cli_host: Optional[str] = None
    ) -> str:
        """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env > ``0.0.0.0``
        when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS an auth gate
        in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a routable
        interface with no auth gate is unsafe, hence the localhost default; widen it consciously."""
        if cli_host and cli_host.strip():
            return cli_host.strip()
        env = os.environ.get("RALLY_BIND_HOST")
        if env and env.strip():
            return env.strip()
        sec = getattr(config, "security", None)
        if getattr(sec, "edge_auth_enabled", False):
            return "0.0.0.0"  # behind a Cloudflare-Access-style edge gate (M9) ? bind all
```

```
interfaces
174         return "127.0.0.1"
175
176
177     # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
O1) ---
178
179
180     def _port_is_free(host: str, port: int) -> bool:
181         """True if (host, port) can be bound right now (best-effort; a TOCTOU race is
acceptable ?
182         uvicorn surfaces a real bind failure loudly)."""
183         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
184             try:
185                 s.bind((host, port))
186                 return True
187             except OSError:
188                 return False
189
190
191     def _find_free_port(host: str) -> int:
192         """An OS-assigned free port on host."""
193         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
194             s.bind((host, 0))
195             return int(s.getsockname()[1])
196
197
198     def _resolve_app_port(host: str, port: int) -> int:
199         """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
200         return port if _port_is_free(host, port) else _find_free_port(host)
201
202
203     def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
204         """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
205         Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
206         (tests / headless) or the server never becomes healthy (don't open a dead page).
Never raises."""
207         if os.environ.get("RALLY_NO_BROWSER"):
208             return False
209         import time as _time
210         import urllib.request
211
212         deadline = _time.monotonic() + timeout
213         while _time.monotonic() < deadline:
214             try:
215                 with urllib.request.urlopen(health_url, timeout=2) as r:
216                     if getattr(r, "status", 200) == 200:
217                         break
218             except Exception:
219                 _time.sleep(0.3)
220         else:
221             return False
222         try:
223             webbrowser.open(url)
224         except Exception:
225             pass
226         return True
227
228
229     # Serves static frontend files directly (now under api/ per approved structure).
230     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
```

```
    # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
    # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
    _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
    try:
        os.makedirs(_STATIC_DIR, exist_ok=True)
    except OSError:
        # The dir ships with the package (a no-op create); guard against a read-only install
        # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
PermissionError.
        pass
    app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")

    # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
snapshot
    # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
if it's
    # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
no SPA change
    # is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
is enough
    # (no client-side path routing ? no history-fallback catch-all, which would also be a
route-order hazard).
    _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
    _UI_DIR = (
        _BUNDLED_UI_DIR
        if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
        else _STATIC_DIR
    )
    _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
    if os.path.isdir(_UI_ASSETS_DIR):
        app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")


    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        index_path = os.path.join(_UI_DIR, "index.html")
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
        return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"


    def _bundled_ui_version() -> Optional[Dict[str, Any]]:
        """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
        Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
        path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
        try:
            with open(path, "r", encoding="utf-8") as f:
                return json.load(f)
        except (OSError, ValueError):
            return None


    # Global variables initialized at startup
    db_instance: Optional[Database] = None
    global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
    # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
    # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
    _instance_lock: Optional[Any] = None

```

```
285     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
286     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
287     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
288     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
289     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
290     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
291     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
292     from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
293         _MAX_DRAIN_WAKE_SEC,
294         JobWaiting,
295         _arm_wake_timer,
296         _drain_due_jobs,
297         _get_executor,
298         _job_to_request,
299         _maybe_record_job_cost,
300         _run_claimed_job,
301         _schedule_next_drain_if_waiting,
302     )
303
304
305     # Legacy thin wrapper for any remaining direct calls during transition.
306     # New code should import load_app_config / AppConfig from backend.config directly.
307     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
308         cfg = load_app_config(config_path)
309         return cfg.model_dump(mode="json")
310
311
312     # --- API Models ---
313     class ProcessRequest(BaseModel):
314         video_path: str
315         player_pool: Optional[List[str]] = None
316         max_frames: Optional[int] = None
317         segmenter: Optional[str] = None
318         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
319         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
320         locale: Optional[str] = None
321         # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
322         # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
323         # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
324         compute: Optional[str] = None
325
326
327     class AnnotationsRequest(BaseModel):
328         # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio
329         # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
330         video_id: str
331         csv: str
332
333
334     class AssetRequest(BaseModel):
335         # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:
336         #   uploaded_path ? a local file from the chunked upload (the durable copy's source;
kept as the
337         #                   local working copy for the run), OR
```

```
        #   source_uri    ? a video ALREADY at rest in a medium (a pasted
s3:///gcs:///gdrive:// URI) ?
        #                   registered as-is, no copy (D14b).
        # medium_uri is the destination store (a bucket/Drive/local folder URI). The asset
is keyed by
        # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content
hash.
        uploaded_path: Optional[str] = None
        source_uri: Optional[str] = None
        medium_uri: str
        sport: Optional[str] = None


    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None


    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str
        # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
        # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
        locale: Optional[str] = None


    def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config(request: Request):
        # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
        # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
        # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
        # by tests/test_redact.py + the /api/config redaction test).
        cfg = request.app.state.config
        if cfg is None:
            return {}
        return redact_secrets(cfg.model_dump(mode="json"))


    @app.get("/api/health")
    def health(request: Request):
```

```
        """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
        (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
        confirm the engine is reachable and which detector is wired. Never raises."""
        cfg = request.app.state.config
        sport = getattr(cfg, "sport", None)
        indexing = getattr(cfg, "indexing", None)
        return {
            "status": "ok",
            "sport": sport,
            "default_segmenter": getattr(indexing, "default_segmenter", None),
        }


    # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
    # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
    API_VERSION = 1


    def _engine_version() -> str:
        """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
        try:
            return _pkg_version("badminton-highlight-indexer")
        except PackageNotFoundError:
            return "unknown"


    @app.get("/api/version")
    def api_version():
        """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
        return {
            "engine_version": _engine_version(),
            "api_version": API_VERSION,
            "ui": _bundled_ui_version(),
        }


    @app.get("/api/capabilities")
    def capabilities(request: Request):
        """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
        it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
        value. Best-effort + never raises (the wizard degrades gracefully)."""
        cfg = request.app.state.config
        indexing = getattr(cfg, "indexing", None)
        deployment = getattr(cfg, "deployment", None)
        ffmpeg = shutil.which(ffmpeg_bin())  # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
        ffprobe = shutil.which(ffprobe_bin())
        # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
        wasb = getattr(indexing, "wasb", None)
        weights_path = (
            os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
        )
        # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
        # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
        # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
        # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
```

```
        nvidia_smi = shutil.which("nvidia-smi")
        return {
            "api_version": API_VERSION,
            "engine_version": _engine_version(),
            "segmenters": sorted(segmenter_registry.list_available().keys()),
            "default_segmenter": getattr(indexing, "default_segmenter", None),
            "ffmpeg": {
                "ffmpeg": bool(ffmpeg),
                "ffprobe": bool(ffprobe),
                "ffmpeg_path": ffmpeg,
                "ffprobe_path": ffprobe,
            },
            "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
            "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
            "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
            "deployment": {
                "profile": getattr(deployment, "profile", "") or "",
                "compute_target": getattr(deployment, "compute_target", "") or "",
                # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
                # the "run in the cloud" toggle only when true (else it's local-only).
                "cloud_available": _cloud_available(cfg),
            },
            # P3: which storage media THIS box can store INTO (local always; s3/gcs if
SDK+config;
            # gdrive if a mount resolves). The P5 medium picker renders only usable
backends. No secrets.
            "storage": storage_capabilities(
                getattr(cfg, "storage", None), local_free_path=output_base(cfg)
            ),
        }


    def _friendly_video_name(filepath: str) -> str:
        """A human name for a video from its stored path/URI ? the filename stem
(``?/match_2026.mp4`` ?
        ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
(rsplit on ``/``).
        Falls back to the whole ``filepath`` if there's no basename."""
        base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
        stem = os.path.splitext(base)[0]
        return stem or base or str(filepath or "")


    def _video_dims(validation_results: Any) -> "tuple[Optional[float], Optional[str]]":
        """Pull ``(fps, resolution)`` from the persisted validation results, or ``(None,
None)``. The
        ``resolution_fps_check`` validator already probed ffprobe and stored
``details={width,height,fps}``
        on its result (every branch) ? so the videos row carries it with no schema change /
re-probe."""
        for r in validation_results or []:
            if isinstance(r, dict) and r.get("validator_name") == "resolution_fps_check":
                d = r.get("details") or {}
                w, h, fps = d.get("width"), d.get("height"), d.get("fps")
                resolution = f"{w}x{h}" if w and h else None
                return (fps if isinstance(fps, (int, float)) else None), resolution
        return None, None


    def _video_summary(row: Dict[str, Any]) -> Dict[str, Any]:
        """Map a DB video row to the API/SPA contract. ADDITIVE ? keeps the raw
``id``/``filepath`` (an
        existing test + any old client rely on them) and adds the fields the SPA library +
the persistent
```

```
503         "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
friendly
504         ``match_name`` (so an uploaded video shows its filename, not a generic fallback),
``local_path``,
505         ``created_at`` (from ``processed_at``), and ``fps``/``resolution`` (from the stored
ffprobe
506         validation details) so the library cards can show them. Mirrors the mock engine's
shape so mock and
507         real agree."""
508         fp = row.get("filepath") or ""
509         fps, resolution = _video_dims(row.get("validation_results"))
510         return {
511             **row,
512             "video_id": row.get("id"),
513             "match_name": _friendly_video_name(fp),
514             "local_path": fp,
515             "created_at": row.get("processed_at") or row.get("created_at"),
516             "fps": fps,
517             "resolution": resolution,
518         }
519
520
521     @app.get("/api/videos")
522     def list_videos(request: Request, limit: Optional[int] = None):
523         """List indexed videos (most recent first) so the console survives a reload ?
previously only a
524         single-row get existed (PACKAGING_PLAN.md P1). Rows are enriched to the SPA contract
525         (``video_id`` + a friendly ``match_name`` derived from the filename) so the library
shows real
526         names and the persistent "current video" label survives an upload ? see
``_video_summary``."""
527         return {"videos": [_video_summary(v) for v in
request.app.state.db.list_videos(limit=limit)]}
528
529
530     def _reels_dir(cfg) -> str:
531         return getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
532
533
534     _REEL_EXTS = (".mp4", ".mov")
535
536
537     @app.get("/api/reels")
538     def list_reels(request: Request):
539         """List compiled highlight reels present in the output dir, newest first, with a
download URL.
540
541         NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
542         does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
543         this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
544         later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
545         base = _reels_dir(request.app.state.config)
546         reels = []
547         try:
548             entries = os.scandir(base)
549         except OSError:
550             return {"reels": []}
551         with entries:
552             for e in entries:
553                 if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
554                     try:
```

```
                        st = e.stat()
                    except OSError:
                        continue
                    reels.append(
                        {
                            "filename": e.name,
                            "size_bytes": st.st_size,
                            "modified_at": st.st_mtime,
                            "download_url": f"/api/reels/{quote(e.name)}",
                        }
                    )
        reels.sort(key=lambda r: r["modified_at"], reverse=True)
        return {"reels": reels}


    @app.get("/api/reels/{filename}")
    def download_reel(filename: str, request: Request):
        """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
        Mirrors /api/highlights' safety (bare-name + containment checks)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        base = _reels_dir(request.app.state.config)
        path = os.path.join(base, name)
        try:
            path = resolve_under_root(path, base)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
            raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    # --- Annotate-in-UI (P8): serve a video's source + persist finished rally labels
-----------------


    # In-flight annotation-proxy generations ? dedup so a browser's many Range requests to
/api/source
    # don't each kick off a transcode.
    _proxy_inflight: "set[str]" = set()
    _proxy_lock = threading.Lock()


    def _proxies_dir(cfg) -> str:
        return os.path.join(output_base(cfg), "proxies")


    def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
        """Background, best-effort: generate the annotation proxy ONCE so subsequent
/api/source loads
        serve the lighter, seekable copy. Deduped; never blocks the request; the source
keeps serving if
        it fails."""
        with _proxy_lock:
            if video_id in _proxy_inflight:
                return
            _proxy_inflight.add(video_id)
        try:
            from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
            if not needs_proxy(src_local):
                return  # already light enough ? no proxy worth making
```

```
615             make_annotation_proxy(src_local, os.path.join(_proxies_dir(cfg),
f"{video_id}.mp4"))
616         except Exception as e:  # noqa: BLE001 - warming is best-effort
617             logger.warning("proxy: warm failed for %s: %s", video_id, e)
618         finally:
619             with _proxy_lock:
620                 _proxy_inflight.discard(video_id)
621
622
623     @app.get("/api/source/{video_id}")
624     def get_source(video_id: str, request: Request):
625         """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
Serves a
626         frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
still keyed
627         to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
in the
628         background for next time ? non-blocking, so the first load is never delayed. Proxy
warming is
629         LOCAL-source only (remote-source proxying is a follow-up)."""
630         rec = request.app.state.db.get_video(video_id)
631         if not rec or not rec.get("filepath"):
632             raise HTTPException(status_code=404, detail="Video not found.")
633         cfg = request.app.state.config
634
635         # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
636         try:
637             proxy_name = safe_output_filename(f"{video_id}.mp4")
638             proxy_path = resolve_under_root(os.path.join(_proxies_dir(cfg), proxy_name),
_proxies_dir(cfg))
639         except ValueError:
640             proxy_path = ""
641         if proxy_path and os.path.isfile(proxy_path):
642             return FileResponse(proxy_path, media_type="video/mp4")
643
644         # 2) Otherwise resolve + serve the source.
645         try:
646             local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage",
None))
647             is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage",
None))
648         except ValueError as e:  # unsupported scheme etc. ? a clean client error, never a
500
649             raise HTTPException(status_code=400, detail=str(e)) from e
650         except Exception as e:  # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
(not swallowed)
651             raise HTTPException(status_code=502, detail=f"Could not resolve the source
video: {e}") from e
652         if not os.path.isfile(local):
653             raise HTTPException(status_code=404, detail="Source video file not found.")
654
655         # 3) Warm a proxy in the background for next time (local source only).
656         if is_local:
657             threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()
658
659         media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
660         return FileResponse(local, media_type=media)
661
662
663     @app.post("/api/annotations")
664     def post_annotations(req: AnnotationsRequest, request: Request):
665         """Persist finished rally labels as the annotator CSV (P8), written verbatim to
666         ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5
``video_id`` ? so it joins
```

```
667         the video by content hash exactly as the training pipeline expects. Byte-compatible
with the VLC
668         plugin + browser extension. A later step (owner-gated) promotes labels into the
durable corpus.
669
670         Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is
validated as a bare
671         filename so an untrusted value can't escape the labels dir (arbitrary write)."""
672         try:
673             edge_auth.resolve_tenant(request.headers, request.app.state.config)
674         except edge_auth.EdgeAuthError as e:
675             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
676
677         cfg = request.app.state.config
678         labels_dir = os.path.join(output_base(cfg), "labels")
679         try:
680             name = safe_output_filename(f"{req.video_id}.rallies.csv")
681         except ValueError as e:
682             raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
683         os.makedirs(labels_dir, exist_ok=True)
684         path = os.path.join(labels_dir, name)
685         try:
686             path = resolve_under_root(path, labels_dir)
687         except ValueError as e:
688             raise HTTPException(status_code=400, detail=str(e)) from e
689         with open(path, "w", encoding="utf-8", newline="") as f:
690             f.write(req.csv or "")
691         # Count CSV data rows (a line starting with an integer rally_number) for an honest
ack.
692         num_rows = sum(1 for ln in (req.csv or "").splitlines() if ln[:1].isdigit() and ","
in ln)
693         logger.info("annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows,
path)
694         return {"status": "ok", "num_rows": num_rows, "csv_uri": f"labels/{name}"}
695
696
697     def _storage_settings(cfg) -> Dict[str, Any]:
698         """The storage config as a plain settings dict (NO secrets ? see StorageConfig)."""
699         sc = getattr(cfg, "storage", None)
700         if sc is None:
701             return {}
702         return sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
703
704
705     def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
706         """The durable, MD5-keyed destination inside the chosen medium:
``<medium>/<video_id><ext>``."""
707         return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
708
709
710     def _medium_local_dir(medium_ref, storage_settings: Dict[str, Any]) -> Optional[str]:
711         """The LOCAL directory a ``put`` into this medium would write into (so a free-space
BLOCK can be
712         enforced, D11), or ``None`` for a cloud bucket (which is cost/quota-warned, never
blocked).
713         ``local`` ? the folder; ``gdrive`` ? the resolved path under the Drive mount."""
714         if medium_ref.backend == "local":
715             return medium_ref.key or "."
716         if medium_ref.backend == "gdrive":
717             from backend.storage.capabilities import _gdrive_mount_root
718
719             root = _gdrive_mount_root(storage_settings)
720             if not root:
721                 return None
722             rel = medium_ref.key.lstrip("/")
```

```
            return os.path.join(root, *rel.split("/")) if rel else root
        return None  # s3 / gcs ? a bucket has no readable local capacity


    @app.post("/api/assets", status_code=201)
    def create_asset(req: AssetRequest, request: Request):
        """Store a video into the user's chosen medium and register it as a library asset
(P4, D14).

        SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root``
is set ? the
        hosted alpha jails inputs and has no per-user medium credentials, so it stays
upload-to-local.
        Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a
local file from
        the chunked upload) OR a ``source_uri`` (a video already at rest in a medium). An
uploaded file +
        a REMOTE medium ? a durable, MD5-keyed copy is ``put`` into the medium while the
local upload
        stays as the working copy; a ``local`` medium keeps the upload in place (it's
already on this box);
        a pasted at-rest URI is registered as-is (no copy, D14b). The asset is keyed by the
whole-file
        MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears
in
        ``GET /api/videos``."""
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

        cfg = request.app.state.config
        sec = getattr(cfg, "security", None)
        if getattr(sec, "allowed_video_root", None):
            raise HTTPException(
                status_code=403,
                detail=(
                    "storing into a custom medium is disabled on this hosted deployment "
                    "(security.allowed_video_root is set); upload to local instead."
                ),
            )
        if bool(req.uploaded_path) == bool(req.source_uri):
            raise HTTPException(
                status_code=400,
                detail="provide exactly one of 'uploaded_path' or 'source_uri'.",
            )
        if req.uploaded_path and not req.medium_uri:
            raise HTTPException(
                status_code=400, detail="'medium_uri' is required when uploading a file."
            )

        storage_settings = _storage_settings(cfg)

        # 1) Resolve a LOCAL working copy of the source + its whole-file MD5 (the join key,
D3).
        if req.uploaded_path:
            try:
                validate_input_path(req.uploaded_path, allowed_root=None)
                src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            if src_ref.backend != "local" or not os.path.isfile(src_ref.key):
                raise HTTPException(
                    status_code=404,
                    detail=f"uploaded file not found at '{req.uploaded_path}'",
```

```
                )
            local_working = src_ref.key
        else:
            try:
                validate_input_path(req.source_uri, allowed_root=None)
                src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            # A remote at-rest source is fetched to scratch to hash it (D3) ? reserve that
scratch (P2).
            if src_ref.backend != "local":
                need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
                require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
            try:
                local_working = storage.acquire_local_input(
                    req.source_uri, getattr(cfg, "storage", None)
                )
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            except Exception as e:  # noqa: BLE001 ? a fetch miss is a 502, surfaced not
swallowed
                raise HTTPException(
                    status_code=502, detail=f"could not resolve source_uri: {e}"
                ) from e
            if not os.path.isfile(local_working):
                raise HTTPException(
                    status_code=404, detail="source_uri did not resolve to a readable file."
                )

        video_id = compute_video_id(local_working)
        ext = os.path.splitext(local_working)[1] or ".mp4"

        # 2) Decide the durable location + copy only when needed.
        copied = False
        if req.source_uri:
            stored_uri = req.source_uri  # already at rest in its medium ? no copy (D14b)
            medium_backend = src_ref.backend
        else:
            try:
                medium_ref = storage.parse_uri(req.medium_uri)
            except ValueError as e:
                raise HTTPException(
                    status_code=400, detail=f"invalid medium_uri: {e}"
                ) from e
            medium_backend = medium_ref.backend
            if medium_ref.backend == "local":
                # "This computer" ? the upload already lives here; it IS the durable copy
(no duplicate).
                stored_uri = os.path.abspath(local_working)
            else:
                dest = _asset_dest_uri(req.medium_uri, video_id, ext)
                # Free-space: BLOCK a local/Drive-mount destination (D11); a cloud bucket is
warn-only.
                local_dir = _medium_local_dir(medium_ref, storage_settings)
                if local_dir is not None:
                    require_free_bytes(
                        local_dir, os.path.getsize(local_working), stage="store"
                    )
                else:
                    logger.info(
                        "[ASSETS] cloud medium %s ? capacity not blocked (cost/quota warn
only).",
                        medium_ref.backend,
                    )
                try:
```

```
                    stored_uri = storage.put(local_working, dest, config=storage_settings)
                    copied = True
                except ValueError as e:
                    raise HTTPException(status_code=400, detail=str(e)) from e
                except Exception as e:  # noqa: BLE001 ? a put failure is surfaced, never
silently dropped
                    logger.error(
                        "[ASSETS] put failed video_id=%s dest=%s: %s",
                        video_id,
                        dest,
                        e,
                        exc_info=True,
                    )
                    raise HTTPException(
                        status_code=502, detail=f"could not store into the medium: {e}"
                    ) from e

        # 3) Register the asset (keyed by MD5) so it appears in GET /api/videos ("my
videos").
        if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
            raise HTTPException(status_code=500, detail="failed to register the asset.")
        logger.info(
            "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s owner=%s",
            video_id,
            medium_backend,
            stored_uri,
            copied,
            owner_id,
        )
        return JSONResponse(
            status_code=201,
            content={
                "video_id": video_id,
                "medium": medium_backend,
                "stored_uri": stored_uri,
                "copied": copied,
                "sport": req.sport,
                "status": "stored",
            },
        )


    def _ingest_remote_segments(
        db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
    ) -> int:
        """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
        persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
        ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
        Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
        backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
        skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
        cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
        and prunes any partial rows (destroy-AFTER-confirm)."""
        uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
        ref = storage.parse_uri(uri)
        n = 0
        with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
            local = storage.fetch(
```

```
                uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
            )
            with open(local, newline="", encoding="utf-8") as f:
                for row in csv.DictReader(f):
                    if n >= max_rows:
                        logger.warning(
                            "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
                            max_rows,
                            uri,
                        )
                        break
                    try:
                        start, end = float(row["start"]), float(row["end"])
                    except (KeyError, TypeError, ValueError):
                        continue
                    if not (math.isfinite(start) and math.isfinite(end)):
                        continue  # reject inf/nan times rather than persist a junk rally
                    meta: Dict[str, Any] = {
                        "source": "cloud_run",
                        "ending_reason": (row.get("ending_reason") or "").strip(),
                    }
                    sc = (row.get("shot_count") or "").strip()
                    if sc:
                        try:
                            meta["shot_count"] = int(
                                float(sc)
                            )  # OverflowError on 'inf'/'1e400'
                        except (ValueError, OverflowError):
                            meta["shot_count"] = sc
                    db.add_play_segment(video_id, start, end, metadata=meta)
                    n += 1
        return n


    def _cloud_available(cfg) -> bool:
        """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).

        True when the control plane is wired for Cloud Run: either the configured default
target is
        already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
        (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
        are present so a forced per-request override can actually dispatch + collect a
result.

        Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
        deployment = getattr(cfg, "deployment", None)
        if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
            return True
        has_job = bool(
            os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
        )
        out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
        return bool(has_job and out_root)


    def _maybe_dispatch_remote(
        cfg,
        db,
        req: ProcessRequest,
        video_id: str,
```

```
        seg_name: str,
        val_results,
        play_wm: int,
        cand_wm: int,
        notify,
    ) -> Optional[Tuple[Dict[str, Any], bool]]:
        """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
        == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
        THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.

        Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
        actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
        doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
        (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
        the in-process path stays byte-identical)."""
        from backend.compute import ComputeJobSpec, get_compute_backend

        def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
            # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
            # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
            # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
            # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
            # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
            # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
            return (
                {
                    "video_id": video_id,
                    "status": "failed",
                    "message": msg,
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                    "compute": {
                        "path": "cloud_run" if ran else "not_run",
                        "requested": req.compute,
                        "target": "cloud_run",
                        "dispatched": ran,
                    },
                },
                ran,
            )

        # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
        # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
        # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
        choice = (req.compute or "").strip().lower()
        if choice and choice not in ("local", "cloud"):
            logger.warning(
                "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
                req.compute,
```

```
            )
            choice = ""
        if choice == "local":
            return None  # explicit "run on my machine" ? in-process regardless of the
server's target
        if choice == "cloud":
            if not _cloud_available(cfg):
                return _failed(
                    "cloud-serving was requested but this server isn't configured for it "
                    "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
                    False,
                )
            compute = get_compute_backend(cfg, target_override="cloud_run")
        else:
            compute = get_compute_backend(
                cfg
            )  # server default (default-OFF unless cloud_run)
        if compute is None:
            return None  # default-OFF ? run the in-process segmenter below (unchanged)

        storage_cfg = getattr(cfg, "storage", None)

        # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
        try:
            input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
        except ValueError as e:
            return _failed(
                f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
            )
        if input_ref.backend == "local":
            return _failed(
                "cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
                "can't be read by the Cloud Run job. Put the video in your bucket first.",
                False,
            )
        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
        if not out_root:
            return _failed(
                "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
                "? that's where the Cloud Run job writes the result.",
                False,
            )

        notify("dispatching (cloud_run)")
        logger.info(
            "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
            req.video_path,
            out_root,
        )
        # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
        # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
        # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
        # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
        overrides = []
        sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
        if sk is not None:
            overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
```

```
        spec = ComputeJobSpec(
            video_uri=req.video_path,
            out_uri=out_root,
            segmenter=seg_name,
            config_overrides=tuple(overrides),
        )
        try:
            result = compute.run_infer(
                spec
            )  # dispatch the Cloud Run Job + bucket-poll to completion
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
            return _failed(f"cloud dispatch error: {e}", True)
        if result.returncode != 0:
            return _failed(f"cloud job failed (returncode {result.returncode}).", True)
        if result.video_id and result.video_id != video_id:
            # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
            # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
            logger.warning(
                "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
                "and the worker hash; ingesting under the local id.",
                result.video_id,
                video_id,
            )

        notify("ingesting (cloud_run)")
        storage_dict = (
            storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
        )
        try:
            _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
        except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
            logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
            return _failed(
                f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
                True,
            )

        # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
        segments = [
            s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
        ]
        _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
        # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
        # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
        provenance = {
            "path": "cloud_run",
            "requested": req.compute,
            "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
            "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
            "execution": result.execution,
            "outputs_uri": result.outputs_uri,
        }
        logger.info(
            "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
            result.execution,
```

```
            provenance["job"],
            provenance["region"],
            result.outputs_uri,
            video_id,
        )
        return (
            {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
                "compute": provenance,
            },
            True,
        )


    def _execute_processing(
        config=None, db=None, req=None, progress=None
    ) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
        module-level ``global_config``/``db_instance`` (backward-compatible with existing
        tests that pass only ``req``). Route handlers always pass explicit params; older
        test code that calls ``_execute_processing(req)`` still works.

        ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        """
        # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
        # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
        if config is not None and not isinstance(config, AppConfig):
            # Old signature: _execute_processing(req, progress=None)
            if req is None and db is not None and isinstance(db, ProcessRequest):
                req = db
                db = None
            elif req is None and isinstance(config, ProcessRequest):
                req = config
                config = None
        # Resolve config/db from module globals when not passed explicitly
        if config is None:
            config = global_config
        if db is None:
            db = db_instance

        notify = progress or (lambda stage: None)

        notify("hashing")
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
        # consumers (hash / validators / segmenter) use the resolved local path.
        local_video = storage.acquire_local_input(
            req.video_path, getattr(config, "storage", None)
        )
        video_id = compute_video_id(local_video)
        was_reingest = db.get_video(video_id) is not None
```

```
1173
1174        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
1175        notify("validating")
1176        logger.info(f"Running validations for {req.video_path}...")
1177        val_results, val_context = registry.run_all_with_context(local_video, config)
1178        failures = [r for r in val_results if not r.passed]
1179
1180        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
1181        default_seg = getattr(
1182            getattr(config, "indexing", None), "default_segmenter", "motion"
1183        )
1184        seg_name = req.segmenter or default_seg
1185        segmenter_actual = None
1186        segmentation_ran = False
1187        # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
1188        # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
1189        # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
1190        compute_actual: Optional[str] = None
1191        response: Optional[Dict[str, Any]] = None
1192        try:
1193            if failures:
1194                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
1195                # status; it no longer destroys the video's prior good segments.
1196                db.add_video(
1197                    video_id,
1198                    req.video_path,
1199                    "failed",
1200                    [r.model_dump() for r in val_results],
1201                )
1202                response = {
1203                    "video_id": video_id,
1204                    "status": "failed",
1205                    "message": "Video failed validation checks.",
1206                    "results": [r.model_dump() for r in val_results],
1207                }
1208                return response
1209
1210            # 2. Run segmenter & indexer
1211            logger.info("[API] Video passed checks. Segmenting and indexing...")
1212            db.add_video(
1213                video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
1214            )
1215
1216            segmenter_cls = segmenter_registry.get(seg_name)
1217            if not segmenter_cls:
1218                # Submit-time validation already 400s unknown names; this is the defensive
1219                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
1220                available = list(segmenter_registry.list_available().keys())
1221                response = {
1222                    "video_id": video_id,
1223                    "status": "failed",
1224                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
```

6. user (2026-07-02T02:47:44.440Z)

```
1225                    "results": [r.model_dump() for r in val_results],
1226                    "play_segments": [],
1227                }
1228                return response
```

```
1229
1230            logger.info(f"[API] Using segmenter: {seg_name}")
1231            notify(f"segmenting ({seg_name})")
1232            segmenter_actual = seg_name
1233            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
1234            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
1235            play_wm, cand_wm = db.segment_watermarks(video_id)
1236            # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
1237            # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
1238            # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
1239            # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
1240            # they're dropped on the cloud path in this slice ? a documented follow-up.)
1241            remote = _maybe_dispatch_remote(
1242                config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
1243            )
1244            if remote is not None:
1245                response, segmentation_ran = (
1246                    remote  # `ran` = did the cloud job actually execute (report)
1247                )
1248                return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
1249            # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
1250            compute_actual = "in_process"
1251            segmenter = segmenter_cls(db, config)
1252            try:
1253                result = segmenter.process_video(
1254                    video_id, local_video, req.player_pool, req.max_frames
1255                )
1256            except Exception:
1257                # A raising segmenter must not leave half-written rows: restore the pre-run
1258                # state (same destroy-after-confirm contract as the Failure branch), then
let
1259                # the job worker record the failure.
1260                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
1261                raise
1262            segmentation_ran = True
1263
1264            if isinstance(result, Failure):
1265                logger.error(f"[API] Segmenter failed: {result.message}")
1266                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
1267                response = {
1268                    "video_id": video_id,
1269                    "status": "failed",
1270                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
1271                    "results": [r.model_dump() for r in val_results],
1272                    "play_segments": [],
1273                }
1274                return response
1275
1276            segments = result.value if hasattr(result, "value") else result
1277            notify("finalizing")
1278            _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
1279
1280            response = {
1281                "video_id": video_id,
1282                "status": "success",
1283                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
```

```
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
            # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
            # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
            # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
            if isinstance(response, dict):
                response.setdefault(
                    "compute",
                    {
                        "path": compute_actual or "not_run",
                        "requested": getattr(req, "compute", None),
                    },
                )
            prov = response.get("compute", {}) if isinstance(response, dict) else {}
            logger.info(
                "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
                video_id,
                getattr(req, "compute", None),
                prov.get("path"),
                prov.get("dispatched"),
                (response or {}).get("status") if isinstance(response, dict) else None,
            )
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
            # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
            # doesn't surface arbitrary run signals.)
            paths = emit_indexer_report(
                db=db,
                video_id=video_id,
                # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
                # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
                # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
                # still preserved in DB provenance via db.add_video.
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=req.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=was_reingest,
                segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths


    def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
        """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.

        Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
        missing cloud SDK, the network, or a credential returns ``None`` so the free-space
```

```
guard degrades
        to a no-op rather than blocking a job it cannot measure."""
        try:
            sc = (
                storage_config.model_dump()
                if hasattr(storage_config, "model_dump")
                else (storage_config or {})
            )
            backend = storage.get_storage(input_ref.backend, config=sc)
            return int(backend.stat(input_ref).size)
        except Exception as e:  # noqa: BLE001 - measuring is best-effort; inability must
not block
            logger.debug(
                "free-space: could not stat remote source %s: %s", input_ref.uri, e
            )
            return None


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest, request: Request):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
        # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
        # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

        allowed_root = getattr(
            getattr(request.app.state.config, "security", None), "allowed_video_root", None
        )
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(
                req.video_path, getattr(request.app.state.config, "storage", None)
            )
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
        # rejects a non-local URI as out-of-root.
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            raise HTTPException(
                status_code=404, detail=f"Video file not found at '{req.video_path}'"
```

```
            )
        # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
        # if this box can't hold that copy. Best-effort ? an unstattable remote
(network/SDK) or an
        # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
        # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
        if input_ref.backend != "local":
            need = _remote_input_size_bytes(
                input_ref, getattr(request.app.state.config, "storage", None)
            )
            require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
            )

        job_id = uuid.uuid4().hex
        if not request.app.state.db.create_job(
            job_id,
            req.video_path,
            req.segmenter,
            req.player_pool,
            req.max_frames,
            owner_id=owner_id,
            locale=req.locale,
            compute=req.compute,
        ):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(
            _drain_due_jobs, request.app.state.config, request.app.state.db
        )
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}/cost")
    def get_job_cost(job_id: str, request: Request):
        """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
        GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
        null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
        cost = request.app.state.db.get_job_cost(job_id)
        if cost is None:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        fx = request.app.state.config.billing.fx_inr_per_usd
        usd = cost.get("cost_usd")
        cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
        cost["fx_inr_per_usd"] = fx
        return cost


    @app.get("/api/videos/{video_id}/cost")
    def get_video_cost(video_id: str, request: Request):
        """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
        per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
```

```
several."""
1450        agg = request.app.state.db.get_video_cost(video_id)
1451        fx = request.app.state.config.billing.fx_inr_per_usd
1452        agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
1453        agg["fx_inr_per_usd"] = fx
1454        return agg
1455
1456
1457    @app.get("/api/jobs/{job_id}")
1458    def get_job(job_id: str, request: Request):
1459        """Job state: {status, progress, result, reports}. `status` = execution state
1460        (queued|running|done|failed); the pipeline verdict is result["status"]."""
1461        job = request.app.state.db.get_job(job_id)
1462        if not job:
1463            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
1464        result = job.get("result")
1465        return {
1466            "job_id": job["id"],
1467            "status": job["status"],
1468            "progress": job.get("progress"),
1469            "video_id": job.get("video_id"),
1470            "error": job.get("error"),
1471            "result": result,
1472            "reports": (result or {}).get("reports"),
1473            "created_at": job.get("created_at"),
1474            "started_at": job.get("started_at"),
1475            "finished_at": job.get("finished_at"),
1476        }
1477
1478
1479    # --- Chunked upload (B2, PR-2) ----------------------------------------------------
1480    # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
1481    # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
1482    # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
1483    # sequential-part logic, and abort cleanup ? is unchanged.
1484    from backend.api import uploads as uploads_api
1485
1486    app.include_router(uploads_api.router)
1487
1488
1489    # --- Highlight download (B3, PR-2) ------------------------------------------------
1490
1491
1492    @app.get("/api/highlights/{video_id}/{filename}")
1493    def download_highlight(video_id: str, filename: str, request: Request):
1494        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
1495        (which only writes into output_dir; the browser previously got back an unreachable
1496        server-local path)."""
1497        try:
1498            name = safe_output_filename(filename)
1499        except ValueError as e:
1500            raise HTTPException(status_code=400, detail=str(e)) from e
1501        if not request.app.state.db.get_video(video_id):
1502            raise HTTPException(status_code=404, detail="Indexed video record not found.")
1503        output_dir = (
1504            getattr(
1505                getattr(request.app.state.config, "output", None), "output_dir", "output"
1506            )
1507            or "output"
1508        )
1509        path = os.path.join(output_dir, name)
1510        try:
```

```
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(
                status_code=404,
                detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
            )
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest, request: Request):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type,
        }
        segments = request.app.state.db.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}


    class _CompileError(Exception):
        """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
        maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""

        def __init__(self, status_code: int, detail: str):
            super().__init__(detail)
            self.status_code = status_code
            self.detail = detail


    def _resolve_compile(
        db, cfg, video_id: str, segment_ids: List[int], output_filename: str
    ):
        """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
        row could change between submit and run). Verifies the video exists, the requested
segments match
        and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
        so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
        ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
        video = db.get_video(video_id)
        if not video:
            raise _CompileError(404, "Indexed video record not found.")
        # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
        try:
            out_name = safe_output_filename(output_filename)
        except ValueError as e:
            raise _CompileError(400, str(e)) from e

        all_segments = db.query_play_segments(video_id, {})
        matched = [s for s in all_segments if s["id"] in segment_ids]
        if not matched:
            raise _CompileError(400, "No matching play segments found to stitch.")

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
```

```
        kept = []
        for s in matched:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched):
            logger.info(
                "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
                stitching.min_shot_count,
                len(matched) - len(kept),
                len(matched),
            )
        if not kept:
            raise _CompileError(
                400,
                f"All {len(matched)} matching segments fall below "
                f"stitching.min_shot_count={stitching.min_shot_count}.",
            )
        return video, kept, out_name


    @app.post("/api/compile", status_code=202)
    def compile_highlights(req: CompileRequest, request: Request):
        """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
reel (#329).

        Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
concat + watermark)
        can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
synchronously returned a
        Cloudflare 524 while the engine kept going (the reel built, but the client saw
"failed" and never
        got the URL). The cheap checks (video exists, segments match + survive the floor,
safe filename)
        fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's
`result` carries
        {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
        # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
before queuing.
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
        try:
            video, _kept, out_name = _resolve_compile(
                request.app.state.db,
                request.app.state.config,
                req.video_id,
                req.segment_ids,
                req.output_filename,
            )
        except _CompileError as e:
            raise HTTPException(status_code=e.status_code, detail=e.detail) from e

        job_id = uuid.uuid4().hex
        # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches
the row by id so
        # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
```

```
localized reel.
        if not request.app.state.db.create_compile_job(
            job_id,
            req.video_id,
            video["filepath"],
            req.segment_ids,
            out_name,
            locale=req.locale,
            owner_id=owner_id,
        ):
            raise HTTPException(
                status_code=500, detail="Failed to persist compile job record."
            )
        _get_executor().submit(
            _drain_due_jobs, request.app.state.config, request.app.state.db
        )
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
        """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
        and return the {status, filepath, download_url} result the SPA polls for. Returns a
        ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
        so the SPA always gets a clean verdict instead of a worker-crash error."""
        notify = progress or (lambda stage: None)
        params = json.loads(job.get("params") or "{}")
        video_id = job.get("video_id") or ""
        segment_ids = params.get("segment_ids") or []
        locale = params.get("locale")

        notify("resolving")
        try:
            video, kept, out_name = _resolve_compile(
                db, cfg, video_id, segment_ids, params.get("output_filename", "")
            )
        except _CompileError as e:
            return {"status": "failed", "message": e.detail, "video_id": video_id}

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        output_dir = (
            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
        )
        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
        # unknown/absent locale keeps the configured default unchanged.
        localized_watermark = localize_watermark(stitching.watermark, locale)
        compiler = VideoCompiler(
            output_dir,
            begin_padding=stitching.begin_padding,
            end_padding=stitching.end_padding,
            watermark=localized_watermark,
        )
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        notify("stitching")
        try:
            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
```

```
            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
            local_src = storage.acquire_local_input(
                video["filepath"], getattr(cfg, "storage", None)
            )
            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
            logger.error(
                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
            )
            return {
                "status": "failed",
                "message": f"Highlight stitching failed: {e}",
                "video_id": video_id,
            }

        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
        notify("finished")
        return {
            "status": "success",
            "filepath": out_path,
            "download_url": download_url,
            "video_id": video_id,
        }


    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
        for r in results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"  {status_symbol} {r.validator_name}: {r.message}")
            if r.details:
                print(f"         Details: {r.details}")

        # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
        print("-" * 60)
        print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
        # Lazy import: cv2 is NOT installed in the LIGHT tier (torch-free default)
        # and this --debug-clip path is CLI-only, not on the served API path.
        import cv2

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            print("[FAIL] OpenCV could not open video.")
            return

        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)

        start_frame = max(0, int((timestamp - 3.0) * fps))
        end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
```

```
        # Measure frame-by-frame changes in sample region
        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
        prev_gray = None
        motions = []

        for _ in range(start_frame, end_frame):
            ret, frame = cap.read()
            if not ret:
                break
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            if prev_gray is not None:
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
                motion_ratio = cv2.countNonZero(thresh) / (
                    thresh.shape[0] * thresh.shape[1]
                )
                motions.append(motion_ratio)
            prev_gray = gray

        cap.release()

        if motions:
            avg_motion = sum(motions) / len(motions)
            # Support both legacy dict (from wrapper) and future direct model
            if hasattr(cfg, "indexing"):
                threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
            else:
                threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
            print(f"  Sample Frame Count: {len(motions)}")
            print(
                f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})"
            )
            if avg_motion > threshold:
                print(
                    "  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play."
                )
            else:
                print(
                    "  Result: [DEAD TIME] Idle movement. Segment classified as dead time."
                )
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)


    def _enforce_startup_or_exit(config) -> None:
        """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
        any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
        the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
        auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
        ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
        the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
```

```
1800        fail-fast wiring + the default-OFF no-op are unit-testable."""
1801        try:
1802            enforce_startup_checks(config, logger=logger, include_exposure=True)
1803        except StartupCheckError as e:
1804            logger.error("[STARTUP] refusing to start: %s", e)
1805            sys.exit(1)
1806
1807
1808    def _maybe_sweep_gemini_orphans(config) -> None:
1809        """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous
crashed run.
1810        A no-op unless the configured provider is Gemini AND its API key is present. Never
raises ?
1811        a failed sweep is logged and swallowed. The Gemini client is built with an HTTP
timeout so
1812        a hung connection can't stall startup indefinitely. Extracted for unit tests."""
1813        try:
1814            provider_name = getattr(config, "ai_provider", "")
1815            if provider_name != "gemini":
1816                return
1817            api_key = os.environ.get("GEMINI_API_KEY")
1818            if not api_key:
1819                return
1820            provider = provider_registry.get(
1821                "gemini", api_key=api_key, model_name=config.ai_model
1822            )
1823            deleted = provider.cleanup_orphaned_files()
1824            if deleted:
```

7. user (2026-07-02T02:47:50.218Z)

```
1825                logger.info(
1826                    "[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted
1827                )
1828        except Exception as e:
1829            logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
1830
1831
1832    def main():
1833        parser = argparse.ArgumentParser(
1834            description="Sports Highlight Indexer Orchestrator"
1835        )
1836        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1837        parser.add_argument(
1838            "--output-dir",
1839            default="output",
1840            help="Directory where processed clips/highlights are saved",
1841        )
1842        parser.add_argument(
1843            "--setup",
1844            action="store_true",
1845            help="Launch dependency checker and setup utility",
1846        )
1847        parser.add_argument(
1848            "--debug-clip",
1849            type=float,
1850            help="Timestamp (in seconds) of a video clip to run detailed diagnostics on",
1851        )
1852        parser.add_argument(
1853            "--server", action="store_true", help="Run the FastAPI local API server"
1854        )
1855        parser.add_argument(
1856            "--app",
1857            action="store_true",
1858            help="Launch the app: start the local server (localhost-only) and open the "
```

```
            "bundled UI in your browser. Falls back to a free port if --port is busy.",
        )
        parser.add_argument(
            "--port", type=int, default=8000, help="Port to run the FastAPI server on"
        )
        parser.add_argument(
            "--host",
            default=None,
            help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0 when "
            "security.edge_auth_enabled. Expose on a network ONLY behind an auth gate.",
        )
        parser.add_argument(
            "--max-frames",
            type=int,
            help="Limit processing to the first N sub-sampled frames for debugging",
        )

        available_segmenters = list(segmenter_registry.list_available().keys())
        parser.add_argument(
            "--segmenter",
            choices=available_segmenters,
            help=f"Choose rally detection segmenter. Available: {available_segmenters}",
        )
        parser.add_argument(
            "--trim-start",
            type=float,
            help="Start time in seconds for the debug trim segment feature",
        )
        parser.add_argument(
            "--trim-end",
            type=float,
            help="End time in seconds for the debug trim segment feature",
        )
        parser.add_argument(
            "--trim-output",
            help="Output filename for the debug trimmed segment (saves in output-dir unless
absolute path)",
        )

        args = parser.parse_args()

        if args.setup:
            # Invoke the diagnostics/bootstrap script (renamed from the old root
            # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
            # build system). Resolve its path from this file so `indexer --setup`
            # works regardless of the caller's cwd.
            import subprocess

            doctor_path = os.path.join(
                os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
                "scripts",
                "doctor.py",
            )
            print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
            subprocess.run([sys.executable, doctor_path])
            return

        if args.trim_start is not None and args.trim_end is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to extract a segment.")
                return

            # Determine output path
            out_filename = args.trim_output or "trimmed_segment.mp4"
            if os.path.isabs(out_filename):
```

```
                out_path = out_filename
                out_dir = os.path.dirname(out_path)
                out_name = os.path.basename(out_path)
            else:
                out_dir = resolve_output_dir(
                    args.output_dir
                )  # P0a: app-home aware (CWD-identical when unset)
                out_path = os.path.join(out_dir, out_filename)
                out_name = out_filename

            os.makedirs(out_dir, exist_ok=True)
            print(
                f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}..."
            )

            # global_config isn't initialized this early; load the typed config so the trim
            # clip carries the same configured stitching paddings + watermark as a real
            # highlight (watermark default-on).
            stitching = load_app_config().stitching
            compiler = VideoCompiler(
                out_dir,
                begin_padding=stitching.begin_padding,
                end_padding=stitching.end_padding,
                watermark=stitching.watermark,
            )
            try:
                res_path = compiler.compile_highlights(
                    args.video_path, [(args.trim_start, args.trim_end)], out_name
                )
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # P2b (PACKAGING_PLAN.md gate O8): batteries-included first run for the friendly
`--app`
        # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
        # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
        # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
        # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER
overwrites an
        # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
        # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
        if args.app:
            seeded_path, seeded = write_light_default_config()
            if seeded:
                print(
                    f"[APP] First run: seeded a truly-local default config at {seeded_path}
"
                    f"(detector 'motion' ? offline, no API key or GPU needed). "
                    f"Use the in-app setup to switch to Gemini."
                )
```

```
1979
1980        # Initialize Global Config and DB
1981        global global_config, db_instance
1982        global_config = load_app_config()  # returns typed AppConfig
1983        _enforce_startup_or_exit(
1984            global_config
1985        )  # M5: fail fast on an incoherent ACTIVE profile
1986
1987        # The CLI --output-dir overrides the configured output directory. It now lives
1988        # on the typed model (OutputConfig.output_dir), so every consumer ? including
1989        # /api/compile ? reads the same value instead of a write-only runtime hack.
1990        # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
1991        # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
1992        global_config.output.output_dir = resolve_output_dir(args.output_dir)
1993        os.makedirs(global_config.output.output_dir, exist_ok=True)
1994
1995        db_path = os.path.join(
1996            global_config.output.output_dir, global_config.output.db_name
1997        )
1998
1999        # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
2000        # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
2001        # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
2002        # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
2003        global _instance_lock
2004        try:
2005            _instance_lock = acquire_lock(db_path + ".lock")
2006        except OSError as exc:
2007            # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
2008            # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
2009            # legitimate single instance).
2010            logger.warning(
2011                "[JOBS] Could not create the single-instance lock at %s.lock (%s);
proceeding "
2012                "WITHOUT the second-instance guard.",
2013                db_path,
2014                exc,
2015            )
2016            _instance_lock = None
2017        else:
2018            if _instance_lock is None:
2019                print(
2020                    f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
2021                    f"start ? a second instance would corrupt in-flight job state. Stop the
other one, or "
2022                    f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance."
2023                )
2024                sys.exit(1)
2025
2026        db_instance = Database(db_path)
2027
2028        # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
2029        # the executor is in-process. Mark them failed so pollers see a terminal state.
2030        swept = db_instance.fail_stale_jobs()
2031        if swept:
2032            logger.warning(
```

```
2033                f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed."
2034            )
2035
2036        # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
2037        # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
2038        # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
2039        _maybe_sweep_gemini_orphans(global_config)
2040
2041        # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
2042        if args.video_path and not args.server and not args.app:
2043            print(f"[CLI] Processing video file: {args.video_path}")
2044            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
2045            # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
2046            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
2047            storage_cfg = getattr(global_config, "storage", None)
2048            try:
2049                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
2050            except ValueError as e:
2051                print(f"[FAIL] {e}")
2052                return
2053            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
2054                print(f"[FAIL] Video file not found: {args.video_path}")
2055                return
2056            local_video = storage.acquire_local_input(args.video_path, storage_cfg)
2057
2058            video_id = compute_video_id(local_video)
2059            was_reingest = db_instance.get_video(video_id) is not None
2060
2061            # Validate (with-context variant surfaces ffprobe_meta for the report)
2062            print("[CLI] Validating...")
2063            val_results, val_context = registry.run_all_with_context(
2064                local_video, global_config
2065            )
2066            failures = [r for r in val_results if not r.passed]
2067
2068            # Save video status
2069            status = "failed" if failures else "processed"
2070            db_instance.add_video(
2071                video_id, args.video_path, status, [r.model_dump() for r in val_results]
2072            )
2073
2074            seg_name = args.segmenter or getattr(
2075                getattr(global_config, "indexing", None), "default_segmenter", "motion"
2076            )
2077            segmenter_actual = None
2078            segmentation_ran = False
2079            try:
2080                print("\n" + "=" * 40)
2081                print("VALIDATION SUMMARY")
2082                print("=" * 40)
2083                for r in val_results:
2084                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
2085                    print(f"{status_symbol} {r.validator_name}: {r.message}")
2086                print("=" * 40 + "\n")
2087
2088                if failures:
2089                    print("[FAIL] Video failed checks. Indexing halted.")
2090                    return
```

```
2091
2092                # Index
2093                segmenter_cls = segmenter_registry.get(seg_name)
2094                if not segmenter_cls:
2095                    print(f"[FAIL] Segmenter '{seg_name}' not found.")
2096                    return
2097
2098                print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
2099                segmenter_actual = seg_name
2100                play_wm, cand_wm = db_instance.segment_watermarks(video_id)
2101                segmenter = segmenter_cls(db_instance, global_config)
2102                result = segmenter.process_video(
2103                    video_id, local_video, max_frames=args.max_frames
2104                )
2105                segmentation_ran = True
2106
2107                if isinstance(result, Failure):
2108                    print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
2109                    print("[FAIL] Indexing halted for video.")
2110                    segments = []
2111                else:
2112                    segments = result.value if hasattr(result, "value") else result
2113                    print(
2114                        f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}"
2115                    )
2116                # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
2117                _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
2118            finally:
2119                # Always emit the per-video observability report. Never raises.
2120                paths = emit_indexer_report(
2121                    db=db_instance,
2122                    video_id=video_id,
2123                    # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI would
write a
2124                    # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source URI
is kept
2125                    # in DB provenance via db.add_video.
2126                    video_path=local_video,
2127                    config=global_config,
2128                    val_results=val_results,
2129                    val_context=val_context,
2130                    segmenter_requested=args.segmenter,
2131                    segmenter_actual=segmenter_actual,
2132                    was_reingest=was_reingest,
2133                    segmentation_ran=segmentation_ran,
2134                )
2135                if paths:
2136                    print(f"[REPORT] Technical report : {paths.get('technical_md')}")
2137                    print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
2138                    print(f"[REPORT] Machine JSON     : {paths.get('json')}")
2139            return
2140
2141        # Start Local Server Mode
2142        if args.app or args.server or not args.video_path:
2143            if args.app:
2144                # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
2145                # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
2146                bind_host = "127.0.0.1"
2147                port = _resolve_app_port(bind_host, args.port)
2148                if port != args.port:
2149                    print(f"[APP] Port {args.port} is busy ? using {port} instead.")
```

```
                url = f"http://{bind_host}:{port}/"
                # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
                # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
                # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
                print(
                    f"[APP] Starting KhelSutra ? opening {url} in your browser "
                    f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop."
                )
                threading.Thread(
                    target=_open_browser_when_ready,
                    args=(url, url + "api/health"),
                    daemon=True,
                ).start()
            else:
                bind_host = _resolve_bind_host(
                    global_config, args.host
                )  # P0c: config/profile-driven (default 127.0.0.1)
                port = args.port
                print(
                    f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}..."
                )
                if bind_host not in ("127.0.0.1", "localhost", "::1"):
                    print(
                        f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
                        f"gate (edge_auth_enabled / a reverse proxy) is in front, and set "
                        f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header guard
defaults to "
                        f"loopback-only and will 400 forwarded requests otherwise)."
                    )
            # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
            # in-process wake-timer for any parked 'waiting' job died with the old process,
so
            # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
            _get_executor().submit(_drain_due_jobs, global_config, db_instance)
            # Stamp shared state on the app so route handlers access it via
request.app.state
            # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).
            create_app(global_config, db_instance)
            uvicorn.run(app, host=bind_host, port=port)


    if __name__ == "__main__":
        # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
        # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
        # module object. If `main()` runs here it configures `__main__`'s globals, leaving
        # `backend.main.db_instance` = None; the async-job drain then crashes silently on
        # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
        # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
        # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
        import backend.main as _bm

        _bm.main()
```

8. user (2026-07-02T02:47:59.289Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11      `backend.main` module object at call-time (`import backend.main as _main`), never as a
12      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13      ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14      `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15      inside the functions also avoids the circular import at module load (main.py imports
this
16      module; this module must not import main at import time).
17      """
18
19      import logging
20      import threading
21      import traceback
22      from concurrent.futures import ThreadPoolExecutor
23      from typing import Any, Dict, Optional
24
25      logger = logging.getLogger(__name__)
26
27      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
28      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
29      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
30      # already parallelize their AI handoff internally via their own pools.
31      #
32      # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
33      # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
34      # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
35      #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
36      #   - detector timeout_sec kills hung WSL/GPU calls
37      #   - Cloud Run poll has a max_wait_sec bound
38      # A future slice should add a per-job wall-clock timeout (the executor itself
39      # cannot enforce timeouts on the Thread).
40      _job_executor: Optional[ThreadPoolExecutor] = None
41
42
43      def _get_executor() -> ThreadPoolExecutor:
44          """Lazy module-global executor; tests monkeypatch this for inline execution."""
45          global _job_executor
46          if _job_executor is None:
47              _job_executor = ThreadPoolExecutor(
48                  max_workers=1, thread_name_prefix="jobworker"
49              )
50          return _job_executor
51
52
53      class JobWaiting(Exception):
54          """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
55          WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
56          NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
57          releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
58          is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
```

```
contract.
59
60          The wiring is additive: the production segmenter path never raises this today (zero
61          behaviour change), so a job runs exactly as before. The hook is what a later slice
62          (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
63          """
64
65          def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
66              super().__init__(f"job parked for {delay_sec:.0f}s")
67              self.delay_sec = max(0.0, float(delay_sec))
68              self.progress = progress
69
70
71      def _job_to_request(job: Dict[str, Any]):
72          """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
73          re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
74          the original submit. The row stores exactly the ProcessRequest fields."""
75          import backend.main as _main
76
77          return _main.ProcessRequest(
78              video_path=job["video_path"],
79              player_pool=job.get("player_pool"),
80              max_frames=job.get("max_frames"),
81              segmenter=job.get("segmenter"),
82              # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
83              # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
84              # the job row for analytics). NULL ? server default = pre-picker behaviour.
85              compute=job.get("compute"),
86          )
87
88
89      def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
90          """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
91          terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
92
93          Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
94          the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
95          result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
96          job self-scheduled and will be re-claimed when `next_poll_at` is due.
97          """
98          import backend.main as _main
99
100         job_id = job["id"]
101         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
102         # original /api/process pipeline). Both record their terminal state the same way;
only process
103         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
104         is_compile = job.get("kind") == "compile"
105         try:
106             if is_compile:
107                 result = _main._execute_compile(
108                     config,
109                     db,
110                     job,
111                     progress=lambda stage: db.set_job_progress(job_id, stage),
112                 )
113             else:
114                 req = _main._job_to_request(job)
```

```
                result = _main._execute_processing(
                    config,
                    db,
                    req,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            if result.get("video_id"):
                db.set_job_video_id(job_id, result["video_id"])
            db.mark_job_done(job_id, result)
            if not is_compile:
                _maybe_record_job_cost(
                    job, result, config, db
                )  # M8 cost ledger (default-OFF; process jobs only)
                _maybe_run_flywheel(
                    job, result, config
                )  # M11 data-flywheel (process jobs only)
        except _main.JobWaiting as w:
            # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
            logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
            db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
        except Exception as e:
            logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
            db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    def _maybe_record_job_cost(
        job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
    ) -> None:
        """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
        ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
        (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
        persist it.

        ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
        ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
        FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
        + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
        With the placeholder pricebook the cost is 0 regardless.

        Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
        claimed row (carries ``owner_id``); ``result`` is the pipeline output."""

        billing_cfg = getattr(config, "billing", None)
        if not billing_cfg or not getattr(billing_cfg, "enabled", False):
            return
        try:
            usage = (result or {}).get("usage") or {}
            db.record_job_cost(
                job["id"],
                usage=usage,
                pricebook_version=billing_cfg.pricebook_version,
                gpu_type=usage.get("gpu_type"),
                compute_backend=(result or {}).get("compute_backend"),
                owner_id=job.get("owner_id"),
                margin=billing_cfg.margin,
            )
            logger.info(
                "Recorded cost for job %s (pricebook=%s).",
                job["id"],
```

```
                billing_cfg.pricebook_version,
            )
        except Exception as e:  # never wedge a completed job on bookkeeping
            logger.warning(
                "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
            )


    def _maybe_run_flywheel(
        job: Dict[str, Any], result: Dict[str, Any], config: Any
    ) -> None:
        """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
        ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
        (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
        land), so this captures **nothing** in production. When activated it captures the
consented job's
        artifacts into the per-video workspace (? later labeling ? the golden training set).

        Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
        row (carries ``owner_id``); ``result`` is the pipeline output."""

        flywheel_cfg = getattr(config, "flywheel", None)
        if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
            return
        try:
            from backend import flywheel

            flywheel.run_for_job(job, result, config)
        except Exception as e:  # never wedge a completed job on the flywheel
            logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)


    def _drain_due_jobs(config: Any, db: Any) -> int:
        """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
        `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
        up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
        a single-worker box keeps making progress with no central timer (the DB is the
clock).
        Returns the number of jobs that reached a terminal/parked state this tick.
        """
        import backend.main as _main

        ran = 0
        while True:
            job = db.claim_due_job()
            if job is None:
                break
            ran += 1
            _main._run_claimed_job(job, config, db)
        # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
        # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
        # on the next submit/drain. Best-effort ? never raises into the worker.
        _main._schedule_next_drain_if_waiting(config, db)
        return ran
```

```
228
229
230     def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
231         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
232         self-scheduled job resumes with no further client submit. The drain itself re-checks
233         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
234         import backend.main as _main
235
236         try:
237             delay = db.seconds_until_next_due()
238         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
239             logger.error(f"Could not compute next-due delay: {e}")
240             return
241         if delay is None:
242             return  # nothing waiting
243         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
244         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
245
246
247     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
248         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
249         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
250         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
251         import backend.main as _main
252
253         timer = threading.Timer(
254             delay_sec,
255             lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
256         )
257         timer.daemon = True
258         timer.start()
259
260
261     #: A parked job further out than this is re-checked by a capped wake rather than one
long
262     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
263     _MAX_DRAIN_WAKE_SEC = 60.0
264
```

9. user (2026-07-02T02:47:59.928Z)

```
1       """Per-video workspace path helper ? one folder per video, local?cloud identical.
2
3       See ``docs/PER_VIDEO_WORKSPACE.md``. The goal: every per-video artifact lives under
4       ONE folder keyed by the MD5 ``video_id``
(``backend/utils/hashing.py::compute_video_id``):
5
6           <root>/[<prefix>/]<video_id>/{source,detector,outputs,reels,logs,tmp}/...
7
8       The ``root`` is a **storage URI** (``local`` path / ``gcs://`` / ``s3://`` /
``gdrive://``), so a
9       LOCAL deployment and a CLOUD (bucket) deployment share the *same relative layout* ? only
the root
10      differs. This module is **pure** (string/URI joins, no I/O) so it is trivially testable
and works
11      for every backend; the storage layer (``backend/storage/``) does the actual read/write.
12
13      Two roots by design (D3): a **durable** workspace root (may be ``gcs://``) for artifacts
worth
14      keeping, and a **fast local compute-scratch** mirror (always local) for frame caches /
```

```
extracted
    chunks that must never be written to a cloud root. ``tmp/`` is the local-only bucket.

    This is Phase 0: the helper + config fields only ? NOT yet wired into write sites (that
converges
    in P2?P4, flag-gated by ``storage.single_folder_per_video``). No behaviour changes on
import.
    """

    from __future__ import annotations

    import os
    from dataclasses import dataclass

    # Per-video subfolders. ``tmp`` is LOCAL-ONLY (frame PNGs, handoff/yolo/gemini chunks,
compile
    # clips) ? huge + ephemeral, never pushed to a cloud workspace; cache-hygiene drops it.
    SUBDIRS = ("source", "detector", "outputs", "reels", "logs", "tmp")
    LOCAL_ONLY_SUBDIRS = ("tmp",)


    def join_uri(root: str, *parts: str) -> str:
        """Join path components onto a storage root with forward slashes.

        Matches how the codebase already composes storage URIs (e.g.
``f"{prefix}/weights/..."`` in
        ``backend/worker``). Works for ``gcs://bucket``, ``s3://bucket``, ``local:///abs``
and bare
        paths alike; empty/blank parts are skipped so ``join_uri(root, "", "x")`` ==
``join_uri(root,
        "x")``. Forward slashes are valid path separators in Python on Windows too.
        """
        base = root.rstrip("/")
        # Skip empty / whitespace-only parts; for real parts strip only slashes so spaces
inside a
        # name (e.g. a mounted "My Drive" path) are preserved.
        tail = "/".join(p.strip("/") for p in parts if p and p.strip())
        return f"{base}/{tail}" if tail else base


    def resolve_workspace_root(
        workspace_root: str, output_root: str, output_dir: str
    ) -> str:
        """Durable workspace root, by precedence: ``storage.workspace_root`` ?
        ``storage.output_root`` ? ``output.output_dir``. The first non-empty wins (non-
breaking)."""
        return (
            (workspace_root or "").strip()
            or (output_root or "").strip()
            or (output_dir or "output")
        )


    @dataclass(frozen=True)
    class VideoWorkspace:
        """The single folder for one video. Pure path math; build with :func:`for_video`.

        ``base`` = ``<root>/[<prefix>/]<video_id>``. Subfolder accessors return URIs/paths
under it.
        """

        root: str
        video_id: str
        prefix: str = "workspaces"

```

```
70          @property
71          def base(self) -> str:
72              if self.prefix:
73                  return join_uri(self.root, self.prefix, self.video_id)
74              return join_uri(self.root, self.video_id)
75
76          def path(self, sub: str = "", name: str = "") -> str:
77              """A path under this video's folder: ``base[/sub][/name]``."""
78              return join_uri(self.base, sub, name)
79
80          @property
81          def source(self) -> str:
82              return self.path("source")
83
84          @property
85          def detector(self) -> str:
86              return self.path("detector")
87
88          @property
89          def outputs(self) -> str:
90              return self.path("outputs")
91
92          @property
93          def reels(self) -> str:
94              return self.path("reels")
95
96          @property
97          def logs(self) -> str:
98              return self.path("logs")
99
100         @property
101         def tmp(self) -> str:
102             return self.path("tmp")
103
104
105     def for_video(root: str, video_id: str, prefix: str = "workspaces") -> VideoWorkspace:
106         """Build the :class:`VideoWorkspace` for ``video_id`` under ``root`` (a storage
URI)."""
107         if not video_id:
108             raise ValueError("video_id is required to build a per-video workspace")
109         return VideoWorkspace(root=root, video_id=video_id, prefix=prefix)
110
111
112     def local_scratch_base(scratch_root: str, video_id: str) -> str:
113         """The fast LOCAL compute-scratch mirror for a video (always native FS, never a
cloud root).
114
115         Used for frame caches / extracted chunks (the ``tmp/`` bucket) during compute,
before durable
116         artifacts are pushed to the workspace. Uses ``os.path.join`` (OS-native separators)
since this
117         is always a local filesystem path.
118         """
119         if not video_id:
120             raise ValueError("video_id is required for a local scratch path")
121         return os.path.join(scratch_root or os.getcwd(), video_id, "tmp")
122
```

10. user (2026-07-02T02:48:08.043Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
```

```
    ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and preserves
    today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally MOUNTED
    Google Drive (Google Drive for Desktop) as plain files.

    Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
    ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an identity
    passthrough. See ``docs/DECOUPLED_COMPUTE/``.
    """

    from __future__ import annotations

    import os
    import tempfile
    from typing import Any, Dict, Iterator, Optional

    from backend.storage.base import StorageBackend
    from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri

    __all__ = [
        "StorageBackend",
        "StorageRef",
        "StorageStat",
        "parse_uri",
        "resolve_input_uri",
        "get_storage",
        "fetch",
        "put",
        "list_uris",
        "acquire_local_input",
        "input_is_local",
        "resolve_input_ref",
    ]


    def _storage_dict(storage_config: Any) -> Dict[str, Any]:
        """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
        if storage_config is None:
            return {}
        if hasattr(storage_config, "model_dump"):
            return storage_config.model_dump()  # type: ignore[no-any-return]
        return dict(storage_config)


    def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
        """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under the input
        config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported URI scheme ?
        callers at the API/CLI boundary catch that and return a clean client error (never a 500).

        Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL path (e.g.
        ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI string."""
        sc = _storage_dict(storage_config)
        uri = resolve_input_uri(
            raw,
            input_backend=sc.get("input_backend", "local"),
            input_root=sc.get("input_root", ""),
        )
        return parse_uri(uri)
```

```
62
63
64      def input_is_local(raw: str, storage_config: Any = None) -> bool:
65          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
66          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
67          that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
68          ``resolve_input_ref``)."""
69          return resolve_input_ref(raw, storage_config).backend == "local"
70
71
72      def acquire_local_input(
73          raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
74      ) -> str:
75          """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
76
77          Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
78          byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
79          non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
80          path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
81          regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
82          worker's ``cmd_infer`` already ships."""
83          sc = _storage_dict(storage_config)
84          ref = resolve_input_ref(raw, sc)
85          if ref.backend == "local":
86              return (
87                  ref.key
88              )  # identity ? no temp, no copy, byte-identical to passing the path directly
89          scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
90          return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
91
92
93      def _backend_class(name: str):
94          """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
95          if name == "local":
96              from backend.storage.local import LocalStorage
97
98              return LocalStorage
99          if name == "s3":
100             from backend.storage.s3 import S3Storage
101
102             return S3Storage
103         if name == "gcs":
104             from backend.storage.gcs import GCSStorage
105
106             return GCSStorage
107         if name == "gdrive":
108             from backend.storage.gdrive import GDriveStorage
109
110             return GDriveStorage
111         if name == "gdrive-api":
112             from backend.storage.gdrive_api import GDriveApiStorage
113
114             return GDriveApiStorage
115         raise ValueError(
116             f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
117         )
118
```

```

    def get_storage(
        backend: Optional[str] = None,
        *,
        config: Optional[Dict[str, Any]] = None,
        **overrides,
    ) -> StorageBackend:
        """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
        the implementation; its per-backend settings come from ``config[backend]`` merged
with
        ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
        cfg = config or {}
        name = backend or cfg.get("backend", "local")
        # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
        # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
        # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
        settings: Dict[str, Any] = dict(
            cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
        )
        settings.update(overrides)
        return _backend_class(name)(**settings)


    def fetch(
        uri: str,
        dst: "str | os.PathLike[str] | None" = None,
        *,
        config: Optional[Dict[str, Any]] = None,
        overwrite: bool = False,
    ) -> str:
        """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
        (no copy); for s3/gcs it downloads to ``dst``."""
        ref = parse_uri(uri)
        return get_storage(ref.backend, config=config).fetch_to_local(
            ref, dst, overwrite=overwrite
        )


    def put(
        local_path: "str | os.PathLike[str]",
        uri: str,
        *,
        config: Optional[Dict[str, Any]] = None,
        content_type: Optional[str] = None,
    ) -> str:
        """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
        ref = parse_uri(uri)
        get_storage(ref.backend, config=config).put(
            local_path, ref, content_type=content_type
        )
        return uri


    def list_uris(
        prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
    ) -> Iterator[str]:
        """Yield the URI of every object under ``prefix_uri``."""
        ref = parse_uri(prefix_uri)
        for r in get_storage(ref.backend, config=config).list(ref):
            yield r.uri
```

178

11. user (2026-07-02T02:48:15.611Z)

```
1       import json
2       import logging
3       import os
4       import shutil
5       import sqlite3
6       from contextlib import contextmanager
7       from typing import Any, Dict, List, Optional, Tuple
8
9       from backend import billing  # pure cost math (no backend imports ? no cycle)
10
11      logger = logging.getLogger(__name__)
12
13      # The highest schema version this binary knows how to migrate TO (the top of the
migration ladder
14      # in `_init_db`). PACKAGING_PLAN.md P2d uses it for the upgrade-safety contract on a
downloadable,
15      # auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back the
DB up before
16      # an upgrade migration runs.
`tests/test_database.py::test_schema_version_matches_ladder` pins it to
17      # the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
18      SCHEMA_VERSION = 9
19
20
21      class SchemaTooNewError(RuntimeError):
22          """The on-disk DB was created by a newer app version than this binary understands
(P2d)."""
23
24
25      class Database:
26          def __init__(self, db_path: str):
27              self.db_path = db_path
28              self._init_db()
29
30          @contextmanager
31          def _connect(self):
32              """Transaction scope that also CLOSES the connection.
33
34              ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
35              never closes, so every call leaked its connection to GC (lingering file
36              handles on the WAL sidecars under Windows). Internal methods use this.
37              """
38              conn = self._get_connection()
39              try:
40                  with conn:
41                      yield conn
42              finally:
43                  conn.close()
44
45          def _get_connection(self):
46              # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
47              # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
48              conn = sqlite3.connect(self.db_path, timeout=30.0)
49              conn.row_factory = sqlite3.Row
50              # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
51              conn.execute("PRAGMA foreign_keys = ON")
52              # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
53              conn.execute("PRAGMA busy_timeout = 30000")
54              try:
55                  conn.execute("PRAGMA journal_mode = WAL")
```

```
            except sqlite3.OperationalError:
                pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
            return conn

        def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
            """Run ONE bool-returning single-statement write inside a connection scope.

            Captures the connect/commit/except-logger.error/return-False boilerplate shared
by
            the bool-returning single-statement writers. ``error_msg`` is the caller's exact
            per-method message (already formatted with its own ids) ? logged verbatim on a
            swallowed write fault so each writer keeps its specific log text. Returns True
on
            success, False (after logging ``error_msg``) on any exception."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(sql, params)
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"{error_msg}: {e}")
                return False

        @staticmethod
        def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
            """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.

            A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
            transaction, so an interrupted v3/v4 block can leave the column added while
            ``user_version`` stays un-bumped ? and the next open would re-run the same
            ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
            future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
            ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
            ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
            """
            try:
                cursor.execute(ddl)
            except sqlite3.OperationalError as exc:
                if "duplicate column name" not in str(exc).lower():
                    raise

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._connect() as conn:
                cursor = conn.cursor()

                self._create_base_tables(cursor)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                # P2d upgrade-safety (no-op for a fresh DB or one already at
SCHEMA_VERSION):
                # (1) DOWNGRADE GUARD ? a DB from a NEWER app (version > what we know) has
schema this
                #     binary can't reason about; opening it could corrupt data, so refuse
loudly.
                if version > SCHEMA_VERSION:
                    raise SchemaTooNewError(
                        f"This database (schema v{version}) was created by a newer version
```

```
of the app "
113                         f"than this one (knows up to v{SCHEMA_VERSION}). Upgrade the app, or
point it at "
114                         f"a different data directory (RALLY_APP_HOME). DB: {self.db_path}"
115                     )
116                 # (2) BACKUP-BEFORE-MIGRATE ? an EXISTING older DB is about to be upgraded
in place; a
117                 #     failed migration could corrupt it, so snapshot it first (best-effort:
WARN, never
118                 #     block the upgrade). version 0 = a brand-new DB ? nothing to back up.
119                 if 0 < version < SCHEMA_VERSION:
120                     self._backup_before_migrate(conn, version)
121
122                 if version < 1:
123                     self._migrate_to_v1(cursor)
124
125                 if version < 2:
126                     self._migrate_to_v2(cursor)
127
128                 if version < 3:
129                     self._migrate_to_v3(cursor)
130
131                 if version < 4:
132                     self._migrate_to_v4(cursor)
133
134                 if version < 5:
135                     self._migrate_to_v5(cursor)
136
137                 if version < 6:
138                     self._migrate_to_v6(cursor)
139
140                 if version < 7:
141                     self._migrate_to_v7(cursor)
142
143                 if version < 8:
144                     self._migrate_to_v8(cursor)
145
146                 if version < 9:
147                     self._migrate_to_v9(cursor)
148                 # future: if version < 10: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 10
149
150                 conn.commit()
151
152         def _backup_before_migrate(
153             self, conn: sqlite3.Connection, from_version: int
154         ) -> None:
155             """Snapshot the DB to ``<db>.bak-v<from_version>`` before an upgrade migration
(P2d).
156
157             Checkpoints the WAL into the main file first so the copy is complete. Best-
effort: a backup
158             failure WARNS but does NOT block the upgrade (the ladder is itself crash-safe;
refusing to
159             start because a backup couldn't be written would be worse than proceeding
without one)."""
160             backup = f"{self.db_path}.bak-v{from_version}"
161             try:
162                 conn.execute("PRAGMA wal_checkpoint(FULL)")
163                 shutil.copy2(self.db_path, backup)
164                 logger.info(
165                     "[DB] backed up v%s database before upgrading ? %s",
166                     from_version,
167                     backup,
168                 )
```

```
            except Exception as exc:  # noqa: BLE001 ? the backup is a best-effort safety net; its failure
                # must NEVER block the in-place upgrade (the migration ladder is itself crash-safe). Log loud.
                logger.warning(
                    "[DB] could not back up the database before migrating from v%s (%s); "
                    "proceeding with the in-place upgrade.",
                    from_version,
                    exc,
                )

        def backup_database(self, dest: str) -> str:
            """Write a consistent copy of the DB to ``dest`` (an 'export my data' affordance, P2d).
            Checkpoints the WAL first so ``dest`` is self-contained. Returns ``dest``."""
            parent = os.path.dirname(os.path.abspath(dest))
            if parent:
                os.makedirs(parent, exist_ok=True)
            with self._connect() as conn:
                conn.execute("PRAGMA wal_checkpoint(FULL)")
            shutil.copy2(self.db_path, dest)
            return dest

        def _create_base_tables(self, cursor):
            """Create the videos / play_segments / events base tables (idempotent)."""
            # Videos Table
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS videos (
                    id TEXT PRIMARY KEY,
                    filepath TEXT NOT NULL,
                    processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    status TEXT NOT NULL,
                    validation_results TEXT
                )
            """)

            # Play Segments Table (Extensible metadata JSON)
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS play_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    server TEXT,
                    receiver TEXT,
                    metadata TEXT,
                    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                )
            """)

            # Events Table
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS events (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    segment_id INTEGER NOT NULL,
                    timestamp REAL NOT NULL,
                    type TEXT NOT NULL,
                    player TEXT,
                    details TEXT,
                    FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
                )
            """)

        def _migrate_to_v1(self, cursor):
```

```
            # v1: raw candidate detections (pre-confirmation), detector-agnostic.
            # Common fields are columns; detector-specific metrics go in `stats` JSON,
            # so a new segmenter reuses this table by setting its own `detector`/`stats`.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS candidate_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    detector TEXT NOT NULL,
                    chunk_index INTEGER,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    confidence REAL,
                    stats TEXT,
                    detection_params TEXT,
                    confirmed_segment_id INTEGER,
                    human_label TEXT,
                    notes TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                    FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
                )
            """)
            cursor.execute("""
                CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                    ON candidate_segments(video_id, detector)
            """)
            cursor.execute("PRAGMA user_version = 1")

        def _migrate_to_v2(self, cursor):
            # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
            # /api/process request. `status` is the EXECUTION state of the job
            # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
            # that failed validation) lives inside `result` JSON as result["status"].
            # `result` is the full sync-era response dict (incl. report paths), so the
            # observability report is the job's artifact, per the plan.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS jobs (
                    id TEXT PRIMARY KEY,
                    video_path TEXT NOT NULL,
                    video_id TEXT,
                    segmenter TEXT,
                    player_pool TEXT,
                    max_frames INTEGER,
                    status TEXT NOT NULL DEFAULT 'queued',
                    progress TEXT,
                    error TEXT,
                    result TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    started_at TIMESTAMP,
                    finished_at TIMESTAMP
                )
            """)
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
            cursor.execute("PRAGMA user_version = 2")

        def _migrate_to_v3(self, cursor):
            # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
            # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
due
            # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
            # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
            # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
            # ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
```

```
them.
294             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
295             self._add_column_idempotent(
296                 cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP"
297             )
298             cursor.execute(
299                 "CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status, next_poll_at)"
300             )
301             cursor.execute("PRAGMA user_version = 3")
302
303         def _migrate_to_v4(self, cursor):
304             # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
305             # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
306             # path can scope rows to an owner. **NULL = local install = byte-identical to
307             # today** ? every existing row stays NULL and every existing query (which never
308             # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
309             # the ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
310             # Additive ONLY: no served-behavior change rides on this slice.
311             self._add_column_idempotent(
312                 cursor, "ALTER TABLE videos ADD COLUMN user_id TEXT"
313             )
314             self._add_column_idempotent(
315                 cursor, "ALTER TABLE candidate_segments ADD COLUMN user_id TEXT"
316             )
317             # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
318             # of index churn while making the eventual per-user lookups cheap.
319             cursor.execute(
320                 "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
321                 "WHERE user_id IS NOT NULL"
322             )
323             cursor.execute(
324                 "CREATE INDEX IF NOT EXISTS idx_candidates_user "
325                 "ON candidate_segments(user_id) WHERE user_id IS NOT NULL"
326             )
327             cursor.execute("PRAGMA user_version = 4")
328
329         def _migrate_to_v5(self, cursor):
330             # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
331             # version): the resolved per-user `fusion_config` overlay + an INLINE
332             # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
333             # and provenance that earned it. `is_active` pins the one row the serving bridge
334             # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
335             # only persists/loads it ? NOTHING reads it on the served path yet.
336             #
337             #   user_id              ? owner of the model (NULL allowed = the local install)
338             #   version              ? monotonic per-user model version
339             #   baseline_version     ? the shared baseline this was fit against (upgrade
switch)
340             #   feature_schema_hash  ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
341             #   fusion_config        ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
342             #   classifier_json      ? JSON LogisticRegression.to_dict() (None = config-only
model)
343             #   promotion_evidence   ? JSON per_user_gate PromotionDecision (why it was
promoted)
344             #   n_videos_at_fit      ? held-out-user corpus size when fit (min_n trust floor
input)
345             #   is_active            ? the one resolved row for this user (partial-unique
below)
346             cursor.execute("""
347                 CREATE TABLE IF NOT EXISTS user_models (
```

```
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    user_id TEXT,
                    version INTEGER NOT NULL DEFAULT 1,
                    baseline_version TEXT,
                    feature_schema_hash TEXT,
                    fusion_config TEXT,
                    classifier_json TEXT,
                    promotion_evidence TEXT,
                    n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
                    is_active INTEGER NOT NULL DEFAULT 0,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                )
            """)
            # One model version per user (NULLs compare distinct in SQLite, so the local
            # install can still carry multiple versioned rows; the active-row guard below
            # is what enforces a single served model).
            cursor.execute("""
                CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
                    ON user_models(user_id, version)
            """)
            # At most ONE active model per user: a partial-unique index over the active
rows.
            # (SQLite treats NULL user_id rows as distinct, so the local install's single
            # active row is still guarded ? there is one local user.)
            cursor.execute("""
                CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
                    ON user_models(user_id) WHERE is_active = 1
            """)
            cursor.execute("PRAGMA user_version = 5")

        def _migrate_to_v6(self, cursor):
            # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost
columns on
            # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records
until
            # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is
stored in
            # USD (matches GCP billing = one source of truth); INR is a display conversion
at a
            # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD
COLUMNs are
            # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no
served change.
            for col, typ in (
                ("owner_id", "TEXT"),  # who pays (the non-owner user / tenant)
                ("compute_backend", "TEXT"),  # local_gpu | cloud_run | ...
                ("gpu_type", "TEXT"),  # L4 | T4 | ... (selects the per-second GPU rate)
                ("gpu_active_seconds", "REAL"),
                ("wall_seconds", "REAL"),
                ("bytes_out", "INTEGER"),  # egress
                ("gemini_cost_usd", "REAL"),  # provider-reported, already USD
                ("pricebook_version", "TEXT"),
                ("cost_usd", "REAL"),  # computed total (USD = source of truth)
                (
                    "cost_breakdown_json",
                    "TEXT",
                ),  # {gpu_seconds: .., egress_gb: .., gemini_usd: ..}
            ):
                self._add_column_idempotent(
                    cursor, f"ALTER TABLE jobs ADD COLUMN {col} {typ}"
                )
            # Partial index keeps the NULL=no-cost fast path churn-free while making per-
owner billing cheap.
            cursor.execute(
                "CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
```

```
                "WHERE owner_id IS NOT NULL"
            )
            # Versioned pricebook: one row per (version, resource, gpu_type). Rates change +
differ by
            # gpu_type ? re-version (never mutate a shipped version, so a recorded cost
stays auditable).
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS pricebook (
                    version TEXT NOT NULL,
                    resource TEXT NOT NULL,
                    gpu_type TEXT NOT NULL DEFAULT '',
                    unit_price_usd REAL NOT NULL DEFAULT 0.0,
                    currency TEXT NOT NULL DEFAULT 'USD',
                    effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                )
            """)
            cursor.execute(
                "CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
                "ON pricebook(version, resource, gpu_type)"
            )
            # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0
(nothing
            # user-facing changes). The owner INSERTs a real, calibrated version (real GCP
$/GPU-s,
            # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this
seed.
            for resource, gpu_type in (
                ("gpu_seconds", "L4"),
                ("gpu_seconds", "T4"),
                ("wall_seconds", ""),
                ("egress_gb", ""),
            ):
                cursor.execute(
                    "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type,
unit_price_usd, "
                    "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')",
                    (resource, gpu_type),
                )
            cursor.execute("PRAGMA user_version = 6")

        def _migrate_to_v7(self, cursor):
            # v7: record the UI LOCALE a job was submitted in (the language the SPA was
rendered in,
            # e.g. "kn", "hi-IN"). NULLABLE ? NULL = unrecorded = byte-identical to today
(CLI / older
            # clients / pre-v7 rows). Enables PMF analytics ("which language markets use the
tool",
            # count_jobs_by_locale) and pairs with the localized reel watermark (the SPA
sends the same
            # locale to /api/compile). Additive-only ALTER; idempotent so an interrupted
upgrade re-runs.
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN locale TEXT")
            cursor.execute("PRAGMA user_version = 7")

        def _migrate_to_v8(self, cursor):
            # v8: record the per-request COMPUTE choice the SPA compute-picker sent ("local"
forces the
            # in-process segmenter; "cloud" forces a Cloud Run dispatch). It must survive
the async
            # submit?worker boundary ? the worker reconstructs the ProcessRequest from this
row, so a
            # NULL/unset value (CLI / older clients / pre-v8 rows) means "use the server
default", i.e.
            # byte-identical to today. Additive-only ALTER; idempotent so an interrupted
upgrade re-runs.
```

```
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN compute TEXT")
            cursor.execute("PRAGMA user_version = 8")

        def _migrate_to_v9(self, cursor):
            # v9: the jobs table now holds TWO kinds of work ? `kind`='process' (the
original, NULL ?
            # process for pre-v9 rows) and `kind`='compile' (the async reel stitch, #329;
/api/compile
            # is now 202+poll like /api/process so a slow ffmpeg stitch can't trip the edge
524). `params`
            # is the kind-specific JSON payload a compile job carries
(segment_ids/output_filename/locale)
            # since those don't map onto the process columns. Additive-only ALTERs;
idempotent.
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN kind TEXT")
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN params TEXT")
            cursor.execute("PRAGMA user_version = 9")

        def add_video(
            self,
            video_id: str,
            filepath: str,
            status: str,
            validation_results: List[Dict[str, Any]],
        ) -> bool:
            """Register or update a video row WITHOUT touching its child segments.

            Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
            with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
            That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
            before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
            row in place; clearing segments is now an explicit, deliberate operation
            (clear_video_data / prune_segments).
            """
            return self._execute_write(
                "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
                "ON CONFLICT(id) DO UPDATE SET "
                "filepath=excluded.filepath, status=excluded.status, "
                "validation_results=excluded.validation_results",
                (video_id, filepath, status, json.dumps(validation_results)),
                "Failed to add video",
            )

        def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
            """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).

            Captured before a (re)segmentation so the run's new rows (id > watermark) can be
            told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
            """
            with self._connect() as conn:
                cur = conn.cursor()
                p = cur.execute(
                    "SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE video_id = ?",
                    (video_id,),
                ).fetchone()[0]
                c = cur.execute(
                    "SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE video_id =
?",
                    (video_id,),
                ).fetchone()[0]
```

```
                return int(p), int(c)

        def prune_segments(
            self,
            video_id: str,
            *,
            play_upto: Optional[int] = None,
            play_after: Optional[int] = None,
            cand_upto: Optional[int] = None,
            cand_after: Optional[int] = None,
        ) -> None:
            """Delete play/candidate segments by id range (events cascade from
play_segments).

            Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
            successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
            and keep the OLD ones when the run failed/produced nothing (``*_after``).
            """
            with self._connect() as conn:
                cur = conn.cursor()
                if play_upto is not None:
                    cur.execute(
                        "DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
                        (video_id, play_upto),
                    )
                if play_after is not None:
                    cur.execute(
                        "DELETE FROM play_segments WHERE video_id = ? AND id > ?",
                        (video_id, play_after),
                    )
                if cand_upto is not None:
                    cur.execute(
                        "DELETE FROM candidate_segments WHERE video_id = ? AND id <= ?",
                        (video_id, cand_upto),
                    )
                if cand_after is not None:
                    cur.execute(
                        "DELETE FROM candidate_segments WHERE video_id = ? AND id > ?",
                        (video_id, cand_after),
                    )
                conn.commit()

        def add_play_segment(
            self,
            video_id: str,
            start_time: float,
            end_time: float,
            server: Optional[str] = None,
            receiver: Optional[str] = None,
            metadata: Optional[Dict[str, Any]] = None,
        ) -> Optional[int]:
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (
                            video_id,
                            start_time,
                            end_time,
                            end_time - start_time,
                            server,
                            receiver,
```

```
                            json.dumps(metadata or {}),
                        ),
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
                return None

        def add_event(
            self,
            segment_id: int,
            timestamp: float,
            event_type: str,
            player: Optional[str] = None,
            details: Optional[Dict[str, Any]] = None,
        ) -> bool:
            return self._execute_write(
                "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
                (segment_id, timestamp, event_type, player, json.dumps(details or {})),
                "Failed to add event",
            )

        def add_candidate_segment(
            self,
            video_id: str,
            detector: str,
            start_time: float,
            end_time: float,
            chunk_index: Optional[int] = None,
            confidence: Optional[float] = None,
            stats: Optional[Dict[str, Any]] = None,
            detection_params: Optional[Dict[str, Any]] = None,
        ) -> Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (
                            video_id,
                            detector,
                            chunk_index,
                            start_time,
                            end_time,
                            end_time - start_time,
                            confidence,
                            json.dumps(stats or {}),
                            json.dumps(detection_params or {}),
                        ),
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
```

```
            """Record that a candidate detection was confirmed as a given play_segment."""
            return self._execute_write(
                "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
                (segment_id, candidate_id),
                f"Failed to link candidate {candidate_id} to segment {segment_id}",
            )

        def query_candidate_segments(
            self,
            video_id: str,
            detector: Optional[str] = None,
            only_unlabeled: bool = False,
        ) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._connect() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._connect() as conn:
                row = (
                    conn.cursor()
                    .execute("SELECT * FROM videos WHERE id = ?", (video_id,))
                    .fetchone()
                )
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
            """All indexed videos, most-recently-added first. Orders by rowid (a monotonic
insertion
            counter, since id is a TEXT PK) rather than the 1-second-granularity
processed_at, which
            would tie within a batch. Mirrors get_video's row shape (validation_results
parsed). Powers
            GET /api/videos for the operator console (PACKAGING_PLAN.md P1)."""
            sql = "SELECT * FROM videos ORDER BY rowid DESC"
            params: tuple = ()
            if limit is not None:
                sql += " LIMIT ?"
                params = (int(limit),)
            with self._connect() as conn:
                rows = conn.cursor().execute(sql, params).fetchall()
            out: List[Dict[str, Any]] = []
```

```
            for row in rows:
                d = dict(row)
                d["validation_results"] = json.loads(d["validation_results"] or "[]")
                out.append(d)
            return out

        def query_play_segments(
            self, video_id: str, filters: Dict[str, Any]
        ) -> List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._connect() as conn:
                cursor = conn.cursor()
                rows = cursor.execute(query, params).fetchall()

                for row in rows:
                    r_dict = dict(row)
                    r_id = r_dict["id"]
                    r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                    # Fetch associated events
                    event_rows = cursor.execute(
                        "SELECT * FROM events WHERE segment_id = ?", (r_id,)
                    ).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._connect() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()
```

```
759
760         # --- Async job store (DEFERRED_HARDENING_PLAN #16) ------------------------------
761         # Writers follow the repo convention (swallow + logger.error, return bool); the
762         # worker logs loudly on a failed transition so a job can't silently wedge.
763
764         def create_job(
765             self,
766             job_id: str,
767             video_path: str,
768             segmenter: Optional[str] = None,
769             player_pool: Optional[List[str]] = None,
770             max_frames: Optional[int] = None,
771             policy: Optional[Dict[str, Any]] = None,
772             owner_id: Optional[str] = None,
773             locale: Optional[str] = None,
774             compute: Optional[str] = None,
775         ) -> bool:
776             """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
777             (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
778             (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
779             ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded.
780             ``compute`` (v8) is the SPA compute-picker choice ("local"/"cloud") the worker
reconstructs +
781             honours; NULL ? the server default (byte-identical to pre-picker behaviour)."""
782             return self._execute_write(
783                 "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
784                 "owner_id, locale, compute) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?)",
785                 (
786                     job_id,
787                     video_path,
788                     segmenter,
789                     json.dumps(player_pool) if player_pool is not None else None,
790                     max_frames,
791                     json.dumps(policy) if policy is not None else None,
792                     owner_id,
793                     locale,
794                     compute,
795                 ),
796                 f"Failed to create job {job_id}",
797             )
798
799         def create_compile_job(
800             self,
801             job_id: str,
802             video_id: str,
803             video_path: str,
804             segment_ids: List[int],
805             output_filename: str,
806             locale: Optional[str] = None,
807             owner_id: Optional[str] = None,
808         ) -> bool:
809             """Insert an async REEL-COMPILE job (#329) in state 'queued', `kind`='compile'.
The stitch
810             params (segment_ids / output_filename / locale) ride the JSON `params` column
since they don't
811             map onto the process columns; `video_path` is the target video's source ref (the
table's
812             NOT NULL column) and `video_id` is set up-front (so /api/jobs surfaces it
immediately). The
813             worker dispatches on `kind` and runs the ffmpeg stitch off the request
```

```
thread."""
            params = json.dumps(
                {
                    "segment_ids": list(segment_ids),
                    "output_filename": output_filename,
                    "locale": locale,
                }
            )
            return self._execute_write(
                "INSERT INTO jobs (id, video_path, video_id, status, owner_id, locale, kind,
params) "
                "VALUES (?, ?, ?, 'queued', ?, ?, 'compile', ?)",
                (job_id, video_path, video_id, owner_id, locale, params),
                f"Failed to create compile job {job_id}",
            )

        def count_jobs_by_locale(self) -> Dict[str, int]:
            """PMF analytics: how many jobs were submitted per UI locale. Unrecorded locales
(NULL ?
            CLI / pre-v7 / older clients) bucket under '(unset)'. Returns {locale: count},
busiest first."""
            with self._connect() as conn:
                rows = (
                    conn.cursor()
                    .execute(
                        "SELECT COALESCE(locale, '(unset)') AS loc, COUNT(*) AS n "
                        "FROM jobs GROUP BY loc ORDER BY n DESC, loc"
                    )
                    .fetchall()
                )
            return {str(r[0]): int(r[1]) for r in rows}

        def mark_job_running(self, job_id: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
                "progress='starting' WHERE id = ?",
                (job_id,),
                f"Failed to mark job {job_id} running",
            )

        def set_job_progress(self, job_id: str, progress: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET progress=? WHERE id = ?",
                (progress, job_id),
                f"Failed to set progress for job {job_id}",
            )

        def set_job_video_id(self, job_id: str, video_id: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET video_id=? WHERE id = ?",
                (video_id, job_id),
                f"Failed to set video_id for job {job_id}",
            )

        def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
            """Job EXECUTED to completion. The pipeline's own verdict (success/failed
            validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
            return self._execute_write(
                "UPDATE jobs SET status='done', progress='finished', result=?, "
                "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                (json.dumps(result), job_id),
                f"Failed to mark job {job_id} done",
            )

```

```
        def mark_job_failed(self, job_id: str, error: str) -> bool:
            """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
            return self._execute_write(
                "UPDATE jobs SET status='failed', error=?, "
                "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                (error, job_id),
                f"Failed to mark job {job_id} failed",
            )

        def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
            """Fetch one job; `result`/`player_pool` JSON deserialized."""
            with self._connect() as conn:
                row = (
                    conn.cursor()
                    .execute("SELECT * FROM jobs WHERE id = ?", (job_id,))
                    .fetchone()
                )
                if not row:
                    return None
                d = dict(row)
                d["result"] = json.loads(d["result"]) if d["result"] else None
                d["player_pool"] = (
                    json.loads(d["player_pool"]) if d["player_pool"] else None
                )
                d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
                return d

        # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)
----------------------------
        def get_pricebook_rates(
            self, version: str, gpu_type: Optional[str] = None
        ) -> Dict[str, float]:
            """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``.
The per-second
            GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
``gpu_type=''``.
            An unknown version ? ``{}`` (so every cost computes to 0.0)."""
            gt = gpu_type or ""
            rates: Dict[str, float] = {}
            with self._connect() as conn:
                rows = (
                    conn.cursor()
                    .execute(
                        "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE
version = ?",
                        (version,),
                    )
                    .fetchall()
                )
            for r in rows:
                res, row_gt, price = (
                    r["resource"],
                    (r["gpu_type"] or ""),
                    float(r["unit_price_usd"]),
                )
                if res == billing.GPU_SECONDS:
                    if row_gt == gt:  # the GPU rate for THIS gpu_type
                        rates[res] = price
                else:
                    rates[res] = price  # wall/egress: gpu_type-independent
            return rates

        def record_job_cost(
            self,
            job_id: str,
```

```
            *,
            usage: Dict[str, float],
            pricebook_version: str,
            gpu_type: Optional[str] = None,
            compute_backend: Optional[str] = None,
            owner_id: Optional[str] = None,
            margin: float = 0.0,
        ) -> Optional[Dict[str, Any]]:
            """Compute the job's cost from ``usage`` × the pricebook + persist every cost
column.
            Returns the stored cost dict, or ``None`` on write failure. The caller gates
this on
            ``billing.enabled`` (default-OFF) ? nothing records until billing is turned
on."""
            rates = self.get_pricebook_rates(pricebook_version, gpu_type)
            # SILENT-UNDER-BILL GUARD: a GPU-seconds AMOUNT with no RATE for this gpu_type
would bill at
            # $0 indistinguishably from a free run (a new SKU / typo / gpu_type=None).
Surface it loudly +
            # in the breakdown so the gap is auditable, not invisible. (A missing amount =
$0 IS safe.)
            gpu_amt = float(usage.get(billing.GPU_SECONDS, 0.0) or 0.0)
            unpriced_gpu = gpu_amt > 0 and billing.GPU_SECONDS not in rates
            if unpriced_gpu:
                logger.warning(
                    "[BILLING] no pricebook rate for gpu_type=%r in version=%r ? %.1f GPU-
seconds billed "
                    "at $0 (UNPRICED; recorded in cost_breakdown_json). Add a rate or fix
gpu_type.",
                    gpu_type,
                    pricebook_version,
                    gpu_amt,
                )
            cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
            if unpriced_gpu:
                breakdown["unpriced_gpu_seconds"] = gpu_amt
            if margin:
                marked = billing.apply_margin(cost_usd, margin)
                breakdown["margin"] = round(
                    marked - cost_usd, 6
                )  # keep ?(breakdown) == cost_usd
                cost_usd = marked
            bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
            ok = self._execute_write(
                "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
                "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?,
cost_usd=?, "
                "cost_breakdown_json=? WHERE id = ?",
                (
                    owner_id,
                    compute_backend,
                    gpu_type,
                    usage.get(billing.GPU_SECONDS),
                    usage.get(billing.WALL_SECONDS),
                    bytes_out,
                    usage.get(billing.GEMINI_USD),
                    pricebook_version,
                    cost_usd,
                    json.dumps(breakdown),
                    job_id,
                ),
                f"Failed to record cost for job {job_id}",
            )
            if not ok:
```

```
                return None
            return {
                "cost_usd": cost_usd,
                "cost_breakdown": breakdown,
                "pricebook_version": pricebook_version,
            }

        def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
            """The job's cost row (``None`` if the job doesn't exist; ``cost_usd`` is
``None`` until
            recorded). ``cost_breakdown`` is the deserialized JSON breakdown."""
            with self._connect() as conn:
                row = (
                    conn.cursor()
                    .execute(
                        "SELECT id, video_id, owner_id, compute_backend, gpu_type,
gpu_active_seconds, "
                        "wall_seconds, bytes_out, gemini_cost_usd, pricebook_version,
cost_usd, "
                        "cost_breakdown_json FROM jobs WHERE id = ?",
                        (job_id,),
                    )
                    .fetchone()
                )
            if not row:
                return None
            d = dict(row)
            raw = d.pop("cost_breakdown_json", None)
            d["cost_breakdown"] = json.loads(raw) if raw else {}
            return d

        def get_video_cost(self, video_id: str) -> Dict[str, Any]:
            """Aggregate cost across a video's jobs: total ``cost_usd`` + the per-job rows.
A video is
            1:1 with an infer job, but a re-ingest can produce several ? sum them."""
            with self._connect() as conn:
                rows = (
                    conn.cursor()
                    .execute(
                        "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM
jobs "
                        "WHERE video_id = ?",
                        (video_id,),
                    )
                    .fetchall()
                )
            jobs: List[Dict[str, Any]] = []
            total = 0.0
            for r in rows:
                cost = r["cost_usd"]
                if cost is not None:
                    total += float(cost)
                jobs.append(
                    {
                        "job_id": r["id"],
                        "cost_usd": cost,
                        "pricebook_version": r["pricebook_version"],
                        "cost_breakdown": json.loads(r["cost_breakdown_json"])
                        if r["cost_breakdown_json"]
                        else {},
                    }
                )
            return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}

        # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
```

```
------------------------
        def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
            """Attach/replace a job's JSON ExecutionPolicy."""
            return self._execute_write(
                "UPDATE jobs SET policy=? WHERE id = ?",
                (json.dumps(policy) if policy is not None else None, job_id),
                f"Failed to set policy for job {job_id}",
            )

        def set_job_waiting(
            self, job_id: str, delay_sec: float, progress: Optional[str] = None
        ) -> bool:
            """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
            worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
            computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
            return self._execute_write(
                "UPDATE jobs SET status='waiting', "
                "next_poll_at=datetime('now', ?), "
                "progress=COALESCE(?, progress) WHERE id = ?",
                (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
                f"Failed to set job {job_id} waiting",
            )

        def seconds_until_next_due(self) -> Optional[float]:
            """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
            schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
            already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
            try:
                with self._connect() as conn:
                    row = (
                        conn.cursor()
                        .execute(
                            "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
                            "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END)
AS secs "
                            "FROM jobs WHERE status='waiting'"
                        )
                        .fetchone()
                    )
                if row is None or row["secs"] is None:
                    return None
                return max(0.0, float(row["secs"]))
            except Exception as e:
                logger.error(f"Failed to compute next-due delay: {e}")
                return None

        def claim_due_job(self) -> Optional[Dict[str, Any]]:
            """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
            flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
            double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
            first. Returns the claimed job dict, or None if nothing is runnable."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    row = cur.execute(
                        "SELECT id, status FROM jobs WHERE status='queued' "
```

```
                        "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
                        "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1"
                    ).fetchone()
                    if not row:
                        return None
                    cur.execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
                        "WHERE id=? AND status=?",
                        (row["id"], row["status"]),
                    )
                    conn.commit()
                    if cur.rowcount != 1:
                        return None  # lost the race to a concurrent worker
                return self.get_job(row["id"])
            except Exception as e:
                logger.error(f"Failed to claim a due job: {e}")
                return None

        def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
            """Crash recovery: any job still queued/running from a previous process is dead
            (the executor is in-process). Mark failed so clients see a terminal state, not a
            hang. Called once at startup. Returns the number of jobs swept."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    cur.execute(
                        "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
                        "WHERE status IN ('queued', 'running')",
                        (reason,),
                    )
                    conn.commit()
                    return cur.rowcount
            except Exception as e:
                logger.error(f"Failed to sweep stale jobs: {e}")
                return 0

        # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) ------------------------
        # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
        # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
        # serving.py) resolves them, and wiring that bridge into the production decision
seam
        # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
        # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
        # JSON-serialised dicts; the rest are scalar columns.

        def upsert_user_model(
            self,
            user_id: Optional[str],
            *,
            version: int = 1,
            baseline_version: Optional[str] = None,
            feature_schema_hash: Optional[str] = None,
            fusion_config: Optional[Dict[str, Any]] = None,
            classifier_json: Optional[Dict[str, Any]] = None,
            promotion_evidence: Optional[Dict[str, Any]] = None,
            n_videos_at_fit: int = 0,
            is_active: bool = False,
        ) -> Optional[int]:
            """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.

            Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
            rowid). When ``is_active`` is True this row becomes the user's single active
```

```
model ?
            any previously-active row for the SAME user is first cleared, so the
            ``idx_user_models_one_active`` partial-unique invariant always holds (one active
model
            per user). Returns the row id, or None on failure.
            """
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    if is_active:
                        # Demote the current active row first (NULL user_id = local: IS NULL
match).
                        if user_id is None:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
                                (version,),
                            )
                        else:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id = ? AND is_active = 1 AND version != ?",
                                (user_id, version),
                            )
                    cur.execute(
                        """INSERT INTO user_models
                           (user_id, version, baseline_version, feature_schema_hash,
                            fusion_config, classifier_json, promotion_evidence,
                            n_videos_at_fit, is_active)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                           ON CONFLICT(user_id, version) DO UPDATE SET
                             baseline_version=excluded.baseline_version,
                             feature_schema_hash=excluded.feature_schema_hash,
                             fusion_config=excluded.fusion_config,
                             classifier_json=excluded.classifier_json,
                             promotion_evidence=excluded.promotion_evidence,
                             n_videos_at_fit=excluded.n_videos_at_fit,
                             is_active=excluded.is_active""",
                        (
                            user_id,
                            version,
                            baseline_version,
                            feature_schema_hash,
                            json.dumps(fusion_config)
                            if fusion_config is not None
                            else None,
                            json.dumps(classifier_json)
                            if classifier_json is not None
                            else None,
                            json.dumps(promotion_evidence)
                            if promotion_evidence is not None
                            else None,
                            n_videos_at_fit,
                            1 if is_active else 0,
                        ),
                    )
                    conn.commit()
                    rid = cur.execute(
                        "SELECT id FROM user_models WHERE "
                        + ("user_id IS NULL" if user_id is None else "user_id = ?")
                        + " AND version = ?",
                        ((version,) if user_id is None else (user_id, version)),
                    ).fetchone()
                    return int(rid[0]) if rid else None
            except Exception as e:
```

```
1227                logger.error(
1228                    f"Failed to upsert user_model (user={user_id}, v={version}): {e}"
1229                )
1230                return None
1231
1232        def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
1233            """Return the single active per-user model row for ``user_id`` (NULL = local
install),
1234            with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
1235            or None if the user has no active model. Does not touch the served pipeline."""
1236            with self._connect() as conn:
1237                if user_id is None:
1238                    row = (
1239                        conn.cursor()
1240                        .execute(
1241                            "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1"
1242                        )
1243                        .fetchone()
1244                    )
1245                else:
1246                    row = (
1247                        conn.cursor()
1248                        .execute(
1249                            "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
1250                            (user_id,),
1251                        )
1252                        .fetchone()
1253                    )
1254                if not row:
1255                    return None
1256                d = dict(row)
1257                for col in ("fusion_config", "classifier_json", "promotion_evidence"):
1258                    d[col] = json.loads(d[col]) if d[col] else None
1259                return d
1260
```

12. user (2026-07-02T02:49:02.312Z)

```
1       """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3       Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4       proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
5       under security.upload_dir, then submitted to /api/process by the returned path.
6       Upload state is in-process by design (this is the single-box alpha): a server restart
7       aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8       arrives with PR-3 identity scoping.
9
10      The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11      ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12      reads the live value the server set at startup (and so tests that monkeypatch
13      ``backend.main.global_config`` flow through unchanged).
14      """
15
16      import os
17      import threading
18      import uuid
19      from typing import Any, Dict, Optional
20
21      from fastapi import APIRouter, HTTPException, Request
22      from pydantic import BaseModel
23
24      from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
25      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
```

```
    from backend.utils.paths import safe_output_filename

    router = APIRouter()

    _uploads: Dict[str, Dict[str, Any]] = {}
    _uploads_lock = threading.Lock()


    class UploadInitRequest(BaseModel):
        filename: str
        total_size_bytes: Optional[int] = None


    class UploadCompleteRequest(BaseModel):
        total_parts: int


    def _upload_cfg():
        # Resolved lazily (import inside the function) to read the LIVE startup config that
        # main() sets on the module global, and to avoid a circular import at module load
        # (main.py imports this router; this module must not import main at import time).
        import backend.main as _main

        sec = getattr(_main.global_config, "security", None)
        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
        exts = [
            e.lower().lstrip(".")
            for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
        ]
        return upload_dir, max_bytes, part_bytes, exts


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if st:
            try:
                os.remove(st["tmp_path"])
            except OSError:
                pass


    @router.post("/api/upload/init")
    def upload_init(req: UploadInitRequest):
        """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
        upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
        try:
            name = safe_output_filename(req.filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        ext = os.path.splitext(name)[1].lower().lstrip(".")
        if ext not in exts:
            raise HTTPException(
                status_code=400,
                detail=f"Extension '.{ext}' not allowed. Allowed: {sorted(exts)}",
            )
        if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
            raise HTTPException(
                status_code=413,
                detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
            )
        os.makedirs(upload_dir, exist_ok=True)
        # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
```

```
effort ? an
91          # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
(never blocks a
92          # legit upload); the per-part total cap in upload_part is the hard backstop either
way.
93          require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
94          upload_id = uuid.uuid4().hex
95          tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
96          open(tmp_path, "wb").close()
97          with _uploads_lock:
98              _uploads[upload_id] = {
99                  "filename": name,
100                 "tmp_path": tmp_path,
101                 "parts": 0,
102                 "bytes": 0,
103             }
104         return {
105             "upload_id": upload_id,
106             "max_part_mb": part_bytes // (1024 * 1024),
107             "next_part": f"/api/upload/{upload_id}/part/0",
108         }
109
110
111     @router.put("/api/upload/{upload_id}/part/{index}")
112     async def upload_part(upload_id: str, index: int, request: Request):
113         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
114         _, max_bytes, part_bytes, _ = _upload_cfg()
115         with _uploads_lock:
116             st = _uploads.get(upload_id)
117         if st is None:
118             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
119         if index != st["parts"]:
120             raise HTTPException(
121                 status_code=409,
122                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
123             )
124
125         received = 0
126         overflow = None
127         with open(st["tmp_path"], "ab") as fh:
128             async for chunk in request.stream():
129                 received += len(chunk)
130                 if received > part_bytes:
131                     overflow = (
132                         f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
133                     )
134                     break
135                 if st["bytes"] + received > max_bytes:
136                     overflow = (
137                         f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
138                     )
139                     break
140                 fh.write(chunk)
141         if overflow:
142             _abort_upload(upload_id)
143             raise HTTPException(
144                 status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
145             )
146
147         with _uploads_lock:
148             st["parts"] += 1
149             st["bytes"] += received
150         return {
151             "upload_id": upload_id,
```

```
152             "received_parts": st["parts"],
153             "received_bytes": st["bytes"],
154         }
155
156
157     @router.post("/api/upload/{upload_id}/complete")
158     def upload_complete(upload_id: str, req: UploadCompleteRequest):
159         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
160         upload_dir, _, _, _ = _upload_cfg()
161         with _uploads_lock:
162             st = _uploads.get(upload_id)
163         if st is None:
164             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
165         if st["parts"] == 0 or st["parts"] != req.total_parts:
166             raise HTTPException(
167                 status_code=409,
168                 detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
169             )
170         final_path = os.path.join(upload_dir, st["filename"])
171         if os.path.exists(
172             final_path
173         ):  # never overwrite an earlier upload with the same name
174             stem, ext = os.path.splitext(st["filename"])
175             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
176         os.replace(st["tmp_path"], final_path)
177         with _uploads_lock:
178             _uploads.pop(upload_id, None)
179         video_id = compute_video_id(final_path)
180         return {
181             "path": os.path.abspath(final_path),
182             "video_id": video_id,
183             "size_bytes": os.path.getsize(final_path),
184             "next": "POST /api/process with this path",
185         }
186
187
188     @router.delete("/api/upload/{upload_id}")
189     def upload_abort(upload_id: str):
190         """Abort an in-flight upload and delete its partial file."""
191         with _uploads_lock:
192             known = upload_id in _uploads
193         if not known:
194             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
195         _abort_upload(upload_id)
196         return {"status": "aborted", "upload_id": upload_id}
197
```

13. user (2026-07-02T02:49:03.051Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
```

```
10           cold-start is absorbed by the async poll.
11        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14       The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15       so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17       The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18       with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19       owner-verified on the M7 real run.
20       """
21
22       from __future__ import annotations
23
24       import json
25       import logging
26       import os
27       import tempfile
28       import uuid
29       from abc import ABC, abstractmethod
30       from typing import Any, List, Optional
31
32       from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33       from backend.infrastructure.execution_policy import (
34           DEFAULT_POLICY,
35           ExecutionPolicy,
36           poll_until,
37       )
38       from backend.storage import fetch, get_storage, parse_uri
39
40       LOG = logging.getLogger("compute.cloud_run")
41
42       # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
43       # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
44       _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
45           "deployment.compute_target=local_gpu",
46           "indexing.detector_impl=native",
47       )
48
49
50       class CloudRunJobClient(ABC):
51           """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
52
53           Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
54           overrides the job's container ``args`` for this execution and starts it."""
55
56           @abstractmethod
57           def run_job(
58               self,
59               job_name: str,
60               argv: List[str],
61               *,
62               region: str,
63               project: Optional[str] = None,
64           ) -> str:
65               """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
```

```
            execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""


    class GoogleCloudRunJobClient(CloudRunJobClient):
        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
        a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""

        def run_job(
            self,
            job_name: str,
            argv: List[str],
            *,
            region: str,
            project: Optional[str] = None,
        ) -> str:
            # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
            # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
            # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
            if not project:
                raise RuntimeError(
                    "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
                    "to resolve the job path; set it on the control plane (see
get_compute_backend)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.JobsClient()
            )  # pragma: no cover - exercised only on the real M7 run
            name = client.job_path(project, region, job_name)
            overrides = run_v2.RunJobRequest.Overrides(
                container_overrides=[
                    run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
                ]
            )
            op = client.run_job(
                request=run_v2.RunJobRequest(name=name, overrides=overrides)
            )
            return (
                getattr(op, "metadata", None)
                and getattr(op.metadata, "name", "")
                or "execution"
            )


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(
            self,
            *,
            run_client: CloudRunJobClient,
```

```
            job_name: str,
            region: str,
            project: Optional[str] = None,
            storage_config: Optional[dict] = None,
            policy: ExecutionPolicy = DEFAULT_POLICY,
            token: Optional[str] = None,
            container_overrides: Optional[tuple[str, ...]] = None,
        ) -> None:
            self._client = run_client
            self._job_name = job_name
            self._region = region
            self._project = project
            self._storage_config = storage_config or {}
            self._policy = policy
            # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
            # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
            self._token = token
            self._container_overrides = (
                _DEFAULT_CONTAINER_OVERRIDES
                if container_overrides is None
                else tuple(container_overrides)
            )

        def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            marker = poll_until(
                lambda: self._read_marker(done_uri),
                self._policy,
                label="cloud-run-infer",
                logger=LOG,
            )
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
```

```
183             if not video_id:
184                 LOG.warning(
185                     "cloud-run: job=%s completion marker present but empty/unreadable (no "
186                     "video_id) ? treating as failure",
187                     self._job_name,
188                 )
189                 rc = rc or 1
190             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
191             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
192             LOG.info(
193                 "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
194             )
195             return ComputeResult(
196                 returncode=rc,
197                 outputs_uri=outputs_uri,
198                 video_id=video_id,
199                 report=report,
200                 execution=str(execution or ""),
201             )
202
203         # --- storage-backed completion signal (bucket-poll) ---
204         def _read_marker(self, done_uri: str) -> Optional[dict]:
205             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
206             ref = parse_uri(done_uri)
207             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
208                 return None
209             return (
210                 self._read_json(done_uri) or {}
211             )  # empty dict still terminates the poll (present == done)
212
213         def _read_json(self, uri: str) -> dict:
214             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
215             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
216             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
217             try:
218                 ref = parse_uri(uri)
219                 dst = os.path.join(
220                     tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json"
221                 )
222                 local = fetch(uri, dst, config=self._storage_config)
223                 with open(local, "r", encoding="utf-8") as f:
224                     data: Any = json.load(f)
225                 return data if isinstance(data, dict) else {}
226             except Exception:
227                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
228                 return {}
229
```

14. user (2026-07-02T02:49:21.777Z)

```
1       """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? capture ? shadow-eval ?
promote.
2
3       The consent?capture?eval?goldenize loop: on a **consented adult** job, durably
**capture** the source
4       + rally output into the per-video workspace ? **shadow-eval** the owned model vs Gemini
? **propose
5       promotion** of good ones into the golden TRAINING set, so the owned model climbs to the
D11 ship floor.
6
7       ? **DEFAULT-OFF + INERT ? two independent guards keep this from doing ANYTHING in
```

```
production today:**
        1. ``config.flywheel.enabled`` defaults ``False`` ? the job-completion hook is a
strict no-op.
        2. :func:`consent_attested` is a **FAKED hard-deny** (returns ``False`` for every job)
until the real
           consent schema lands (counsel-reviewed ToS/DPA ? consent columns ? wiring). So even
           ``enabled=True`` captures **nothing**. This is the privacy guardrail ? D10 + the
CLAUDE.md rule:
           adults-only, explicit opt-in, train-on-derived + auto-delete-raw; **minors
excluded**; per-user
           training is a *separate* opt-in; paid tier never trains.

      This module is the PLUMBING ? it REUSES the real ``workspace`` / ``eval.experiment`` /
      ``eval.promotion`` / ``storage`` code so the loop works the day consent + the owned
model + golden
      labels are wired. It ships **no live behavior change**. Pairs with M9-consent (the
consent columns).
      """

      from __future__ import annotations

      import json
      import logging
      import os
      import tempfile
      from typing import Any, Dict, Optional, Sequence

      from backend import storage as _storage
      from backend.utils import workspace as _workspace

      logger = logging.getLogger(__name__)


      def _sc(storage_config: Any) -> Optional[Dict[str, Any]]:
          """Normalize an ``AppConfig.storage`` (pydantic model / dict / None) to the dict
          ``backend.storage`` expects."""
          if storage_config is None:
              return None
          if hasattr(storage_config, "model_dump"):
              return storage_config.model_dump()
          return storage_config if isinstance(storage_config, dict) else None


      def consent_attested(job: Dict[str, Any], config: Any = None) -> bool:
          """The flywheel's consent gate ? currently a **FAKED hard-deny** (always ``False``).

          The flywheel may capture a user's footage into the training pool ONLY with attested,
**adults-only**
          consent (D10 / the CLAUDE.md guardrail: minors excluded; per-user training is a
separate explicit
          opt-in; paid tier never trains). The REAL gate ? reading counsel-defined consent
columns + an
          adults-only attestation + a retention class ? lands with **M9-consent**; until then
this denies
          every job so the harness captures nothing. **Do not relax without counsel sign-
off.**"""
          return False


      def capture_job(
          workspace: "_workspace.VideoWorkspace",
          *,
          rally_output: Dict[str, Any],
          source_path: Optional[str] = None,
          storage_config: Any = None,
```

```
    ) -> Dict[str, str]:
        """Durably persist a consented job's artifacts into its per-video ``workspace`` (the raw material a
        human later labels ? the golden training set). Always writes a small ``capture.json`` manifest
        (``video_id`` + rally output + provenance); when a local ``source_path`` is given, also stores the
        source video. Returns the written URIs. Reuses :func:`backend.storage.put` + the ``VideoWorkspace``
        layout, so it honours whatever ``storage.backend`` the deploy uses (local / gcs / s3)."""
        written: Dict[str, str] = {}
        cfg = _sc(storage_config)
        fd, manifest_local = tempfile.mkstemp(suffix=".json")
        try:
            with os.fdopen(fd, "w", encoding="utf-8") as fh:
                json.dump(
                    {"video_id": workspace.video_id, "rally_output": rally_output},
                    fh,
                    indent=2,
                )
            manifest_uri = _workspace.join_uri(workspace.outputs, "capture.json")
            written["manifest_uri"] = _storage.put(manifest_local, manifest_uri, config=cfg)
            if source_path and os.path.exists(source_path):
                source_uri = _workspace.join_uri(
                    workspace.source, os.path.basename(source_path)
                )
                written["source_uri"] = _storage.put(source_path, source_uri, config=cfg)
        finally:
            try:
                os.unlink(manifest_local)
            except OSError:
                pass
        return written


    def shadow_eval(
        candidates: Sequence[Any],
        owned_variant: Any,
        gemini_variant: Any,
        *,
        labeled: bool = False,
        iou: float = 0.5,
    ) -> Dict[str, Any]:
        """Owned-vs-Gemini shadow comparison (the D11 climb signal). Delegates to the experiment harness:
        a labeled golden corpus ? :func:`backend.eval.experiment.compare` (B?A deltas + honest CIs); an
        unlabeled single video ? :func:`backend.eval.experiment.compare_unlabeled` (agreement). The owned
        model + a golden corpus are owner/GPU-side, so this only *runs* once they exist; the wrapper makes
        the call site real today (and is exercised via tests with stub variants)."""
        from backend.eval import experiment

        if labeled:
            return experiment.compare(candidates, owned_variant, gemini_variant, iou=iou)
        if len(candidates) != 1:
            raise ValueError("unlabeled shadow_eval compares exactly one video at a time")
        return experiment.compare_unlabeled(
            candidates[0], owned_variant, gemini_variant, agree_iou=iou
        )


    def propose_promotion(
```

```
        served_owned: Sequence[float],
        served_gemini: Sequence[float],
        *,
        min_n: int = 5,
    ) -> Any:
        """Whether the owned model is **served-safe** to promote toward default ? wraps
        :func:`backend.eval.promotion.promote_if_served_safe` (the foundational *no-
regression-at-served-
        windowing* gate). ``served_*`` are per-video served F1s on the same corpus; Gemini
is the control
        (A), the owned model the proposal (B)."""
        from backend.eval.promotion import promote_if_served_safe

        return promote_if_served_safe(served_gemini, served_owned, min_n=min_n)


    def run_for_job(
        job: Dict[str, Any],
        result: Dict[str, Any],
        config: Any,
        *,
        source_path: Optional[str] = None,
    ) -> Optional[Dict[str, Any]]:
        """The gated orchestrator called at job completion. Returns ``None`` (a no-op)
unless **both**
        ``config.flywheel.enabled`` AND :func:`consent_attested` ? so today it never
captures (consent is
        faked-deny). On a consented job it captures the artifacts into the per-video
workspace; shadow-eval
        + promotion are wired but stay inert until the owned model + golden labels exist.
Best-effort: the
        caller wraps this so a bookkeeping hiccup never wedges a completed job."""
        fw = getattr(config, "flywheel", None)
        if not fw or not getattr(fw, "enabled", False):
            return None
        if not consent_attested(job, config):
            logger.info(
                "[FLYWHEEL] job %s not captured ? no attested consent (faked-deny).",
                job.get("id"),
            )
            return None

        video_id = (result or {}).get("video_id")
        if not video_id:
            logger.info(
                "[FLYWHEEL] job %s has no video_id ? nothing to capture.", job.get("id")
            )
            return None

        storage_cfg = getattr(config, "storage", None)
        output_dir = getattr(getattr(config, "output", None), "output_dir", "output")
        root = _workspace.resolve_workspace_root(
            getattr(storage_cfg, "workspace_root", "") or "",
            getattr(storage_cfg, "output_root", "") or "",
            output_dir,
        )
        ws = _workspace.for_video(root, video_id, prefix=fw.workspace_prefix)
        written = capture_job(
            ws,
            rally_output={"play_segments": (result or {}).get("play_segments", [])},
            source_path=source_path,
            storage_config=storage_cfg,
        )
        logger.info(
            "[FLYWHEEL] captured job %s (video %s) ? %s", job.get("id"), video_id, written
```

```
176         )
177         return {"captured": written, "video_id": video_id}
178
```

15. user (2026-07-02T02:49:22.203Z)

```
1       """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3       Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4       - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5         for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6         so it is abandoned, not joined ? it never blocks the process.
7       - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8         ``max_wait_sec`` deadline).
9
10      **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11      waiting (a fake clock advances instantly).
12
13      **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14      transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15      ``execution.policy`` logger to watch the poll trail.
16      """
17
18      from __future__ import annotations
19
20      import logging
21      import threading
22      import time as _time
23      from dataclasses import asdict, dataclass
24      from enum import Enum
25      from typing import Callable, Optional, TypeVar
26
27      T = TypeVar("T")
28
29      #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
30      LOG = logging.getLogger("execution.policy")
31
32
33      class WaitMode(str, Enum):
34          """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
35
36          WAIT_AND_RESUME = "wait_and_resume"  # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
37          ABORT = "abort"  # one attempt; on hang/fail, stop immediately
38          BLOCK = "block"  # bounded in-process blocking wait (no reschedule)
39
40
41      class PolicyTimeout(TimeoutError):
42          """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
43
44
45      @dataclass(frozen=True)
46      class ExecutionPolicy:
47          """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
```

```
48
49          mode: WaitMode = WaitMode.WAIT_AND_RESUME
50          poll_every_n_sec: float = 10.0
51          op_timeout_sec: float = (
52              120.0  # HARD per-attempt deadline ? the hang-killer that was missing
53          )
54          max_wait_sec: float = 1800.0  # total wall budget for a poll loop
55          max_retries: int = 5  # retries (=> max_retries+1 attempts) for failures/hangs
56          backoff_base_sec: float = 3.0  # exponential: backoff_base * 2**attempt
57          on_timeout: str = "reschedule"  # reschedule | partial | fail  (consumed by P2/P3)
58          resumable: bool = True
59
60          def to_dict(self) -> dict:
61              d = asdict(self)
62              d["mode"] = self.mode.value
63              return d
64
65          @classmethod
66          def from_dict(cls, d: dict) -> "ExecutionPolicy":
67              known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
68              kw = {k: v for k, v in d.items() if k in known}
69              if "mode" in kw and not isinstance(kw["mode"], WaitMode):
70                  kw["mode"] = WaitMode(kw["mode"])
71              return cls(**kw)
72
73
74      #: The sane default (also what a job gets if it declares no policy).
75      DEFAULT_POLICY = ExecutionPolicy()
76
77
78      def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
79          """Deterministic exponential backoff (no jitter ? reproducible tests)."""
80          return policy.backoff_base_sec * (2**attempt)
81
82
83      def run_bounded(
84          fn: Callable[[], T],
85          policy: ExecutionPolicy = DEFAULT_POLICY,
86          *,
87          label: str = "op",
88          logger: Optional[logging.Logger] = None,
89          sleep: Callable[[float], None] = _time.sleep,
90      ) -> T:
91          """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
92
93          Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
94          last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
95          """
96          log = logger or LOG
97          attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
98          last_exc: Optional[BaseException] = None
99          for attempt in range(attempts):
100             box: dict = {}
101             done = threading.Event()
102
103             # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
104             # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
105             def _runner(_box: dict = box, _done: threading.Event = done) -> None:
106                 try:
107                     _box["val"] = fn()
```

```
                except (
                    BaseException
                ) as e:  # relay ANY error (incl. timeouts) to the caller thread
                    _box["exc"] = e
                finally:
                    _done.set()

            threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
            if done.wait(timeout=policy.op_timeout_sec):
                if "exc" in box:
                    last_exc = box["exc"]
                    log.info(
                        "%s: attempt %d/%d failed: %s",
                        label,
                        attempt + 1,
                        attempts,
                        last_exc,
                    )
                else:
                    if attempt:
                        log.info(
                            "%s: succeeded on attempt %d/%d", label, attempt + 1, attempts
                        )
                    return box["val"]  # type: ignore[return-value]
            else:
                last_exc = PolicyTimeout(
                    f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s"
                )
                log.warning(
                    "%s: attempt %d/%d HUNG (>%.0fs) ? abandoning",
                    label,
                    attempt + 1,
                    attempts,
                    policy.op_timeout_sec,
                )
            if attempt < attempts - 1:
                wait = _backoff_sec(policy, attempt)
                log.debug("%s: backing off %.1fs before retry", label, wait)
                sleep(wait)
        assert last_exc is not None
        log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
        raise last_exc


    def poll_until(
        check: Callable[[], Optional[T]],
        policy: ExecutionPolicy = DEFAULT_POLICY,
        *,
        label: str = "op",
        logger: Optional[logging.Logger] = None,
        sleep: Callable[[float], None] = _time.sleep,
        monotonic: Callable[[], float] = _time.monotonic,
    ) -> T:
        """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
        ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.

        Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
        """
        log = logger or LOG
        start = monotonic()
        polls = 0
        log.info(
```

```
            "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
            label,
            policy.poll_every_n_sec,
            policy.max_wait_sec,
        )
        while True:
            result = check()
            if result is not None:
                log.info(
                    "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
                )
                return result
            elapsed = monotonic() - start
            if elapsed >= policy.max_wait_sec:
                log.warning(
                    "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
                )
                raise PolicyTimeout(
                    f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
                )
            polls += 1
            log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
            sleep(policy.poll_every_n_sec)

```

16. user (2026-07-02T02:49:33.335Z)

{"b":"**Severity: cost + correctness.** A page refresh or the SPA's \"Try again\" re-uploads the same file and POSTs `/api/process` again. Since `video_id` is the **MD5 of the file**, the re-submit carries the **same `video_id`** ? but the engine **re-runs the full (paid) segmenter anyway** instead of returning the existing result. On the Gemini path that's a repeated, real API charge (? $0.10 per 8-min match) for zero new information, plus another multi-minute compute run.\n\n### Confirmed current behavior (`backend/main.py`)\n`_execute_processing` computes `was_reingest = db_instance.get_video(video_id) is not None` ? i.e. it **detects** the video is already processed ? but uses that only for destroy-after-confirm safety (`_finalize_reingest` prunes/replaces). There is **no \"already processed ? reuse, skip the segmenter\" path**: it runs the segmenter and replaces the segments regardless. So a re-submit re-bills.\n\n### Live trigger (alpha run, 2026-06-22)\nAn 8.3-min upload finished with 27 rallies, but the SPA's 10-min poll ceiling timed out **~seconds before completion** and showed \"Couldn't process ? Try again.\" Clicking \"Try again\" would have re-uploaded the identical file and re-run Gemini end-to-end. (We recovered by loading `?video=\u003cid\u003e` instead.) The SPA-side resume is tracked in `avidullu/khelsutra#111`; **this issue is the engine-side guardrail** so we're protected even if a re-submit slips through (defense in depth).\n\n### Proposed fix\n- Make `/api/process` **idempotent by cont","n":340,"s":"OPEN","t":"[cost] /api/process re-runs the paid segmenter for an already-processed video ? refresh/\"Try again\" re-bills Gemini"}

{"m":null,"n":390,"s":"OPEN","t":"refactor(D1/P10): extract orchestration.py from main.py (god-module split, step 1)"}

17. user (2026-07-02T02:49:33.341Z)

```
1       """Google Cloud Storage backend ? first-class object buckets.
2
3       Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls it;
4       install with the ``storage-gcs`` extra. Credentials come from Application Default Credentials
5       (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload identity) ?
6       never from config. A ``client`` may be injected for tests.
7       """
8
9       from __future__ import annotations
10
11      from datetime import timedelta
```

```
    from typing import Any, Iterator, Optional

    from backend.storage.base import StorageBackend
    from backend.storage.ref import StorageRef, StorageStat


    class GCSStorage(StorageBackend):
        def __init__(
            self,
            *,
            client: Any = None,
            project: Optional[str] = None,
            signed_url_ttl: int = 3600,
            **_ignored,
        ) -> None:
            self._ttl = signed_url_ttl
            if client is not None:
                self._client = client
                return
            from google.cloud import (
                storage,  # type: ignore  # lazy: only when a gcs:// URI is used
            )

            self._client = storage.Client(project=project)

        def _blob(self, ref: StorageRef):
            return self._client.bucket(ref.bucket).blob(ref.key)

        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError("GCSStorage.fetch_to_local requires a dst path")
            import os

            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            self._blob(ref).download_to_filename(dst)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
            return ref

        def exists(self, ref) -> bool:
            return bool(self._blob(ref).exists())

        def stat(self, ref) -> StorageStat:
            b = self._blob(ref)
            b.reload()
            return StorageStat(
                size=int(b.size or 0),
                modified=b.updated,
                etag=b.etag,
                content_type=b.content_type,
            )

        def list(self, prefix) -> Iterator[StorageRef]:
            for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
                yield StorageRef(
                    "gcs", prefix.bucket, blob.name, f"gcs://{prefix.bucket}/{blob.name}"
                )

        def delete(self, ref) -> None:
            self._blob(ref).delete()
```

```
77
78          def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
79              return self._blob(ref).generate_signed_url(
80                  version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method
81              )
82
```

18. user (2026-07-02T02:49:44.733Z)

```
1       """Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant ``user_id``
(M9, D9).
2
3       **DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict
NO-OP (returns
4       ``None`` = the local single-user) ? nothing about the existing local path changes. When
enabled, the
5       ``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256) against the
team JWKS ?
6       signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't
spoof** the
7       identity header (the explicit D9 anti-spoof requirement). Returns the verified
``user_id`` (the
8       token's ``email``, else ``sub``).
9
10      ``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the base runtime works without it
(install the
11      ``auth`` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole
verifier is
12      unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no
Cloudflare.
13      """
14
15      from __future__ import annotations
16
17      import json
18      import time
19      import urllib.request
20      from typing import Any, Callable, Mapping, Optional
21
22      # Cloudflare Access sends the signed assertion in this header (it terminates at the CF
edge).
23      CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
24      _JWKS_PATH = "/cdn-cgi/access/certs"
25      _JWKS_TTL_SEC = (
26          600.0  # re-fetch the team JWKS at most every 10 min (keys rotate slowly)
27      )
28
29      JwksFetcher = Callable[[str], dict]
30
31
32      class EdgeAuthError(Exception):
33          """A required Cf-Access token was missing or failed verification ? the caller
returns 401/403."""
34
35
36      # module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only;
tests inject.
37      _jwks_cache: dict[str, tuple[float, dict]] = {}
38
39
40      def _issuer(team_domain: str) -> str:
41          return team_domain.rstrip("/")
42
43
44      def _default_jwks_fetcher(team_domain: str) -> dict:
```

```
45          """Fetch the team's public JWKS (``<team_domain>/cdn-cgi/access/certs``). Network ?
only the
46          production path reaches here; tests pass an injected fetcher."""
47          url = _issuer(team_domain) + _JWKS_PATH
48          with urllib.request.urlopen(url, timeout=5) as resp:  # noqa: S310 (https team
domain)
49              data: Any = json.load(resp)
50          if not isinstance(data, dict) or "keys" not in data:
51              raise EdgeAuthError(f"unexpected JWKS shape from {url}")
52          return data
53
54
55      def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
56          """Return the JWKS for ``team_domain``, cached with a TTL. An injected ``fetcher``
(tests)
57          bypasses the cache so each test is deterministic."""
58          if fetcher is not None:
59              return fetcher(team_domain)
60          now = time.time()
61          cached = _jwks_cache.get(team_domain)
62          if cached and (now - cached[0]) < _JWKS_TTL_SEC:
63              return cached[1]
64          jwks = _default_jwks_fetcher(team_domain)
65          _jwks_cache[team_domain] = (now, jwks)
66          return jwks
67
68
69      def verify_cf_access_token(
70          token: str, *, team_domain: str, aud: str, jwks: dict
71      ) -> str:
72          """Verify a Cf-Access JWT against ``jwks`` (RS256; checks sig + exp + issuer +
audience) and
73          return the user_id (``email`` else ``sub``). Raises ``EdgeAuthError`` on ANY
failure."""
74          try:
75              import jwt
76              from jwt import PyJWKSet
77          except (
78              Exception
79          ) as exc:  # pragma: no cover - only when the auth extra isn't installed
80              raise EdgeAuthError(
81                  "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
82                  "(pip install -e .[auth]) on a host with edge_auth_enabled."
83              ) from exc
84
85          try:
86              kid = jwt.get_unverified_header(token).get("kid")
87              if not kid:
88                  raise EdgeAuthError("token header has no kid")
89              signing_key = PyJWKSet.from_dict(jwks)[kid]
90              claims = jwt.decode(
91                  token,
92                  signing_key.key,
93                  algorithms=["RS256"],
94                  audience=aud,
95                  issuer=_issuer(team_domain),
96                  options={"require": ["exp", "iss", "aud"]},
97              )
98          except EdgeAuthError:
99              raise
100         except KeyError as exc:  # kid not in the JWKS
101             raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
102         except (
103             Exception
104         ) as exc:  # jwt.* errors: bad sig / expired / wrong aud|iss / malformed
```

```
105             raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc
106 
107         user = claims.get("email") or claims.get("sub")
108         if not user:
109             raise EdgeAuthError("verified token carries no email/sub claim")
110         return str(user)
111 
112 
113     def resolve_tenant(
114         headers: Mapping[str, str],
115         config: Any,
116         *,
117         jwks_fetcher: Optional[JwksFetcher] = None,
118     ) -> Optional[str]:
119         """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
120 
121         DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-
user; unchanged).
122         Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403)
on any miss.
123         ``jwks_fetcher`` is injectable for offline tests.
124         """
125         sec = getattr(config, "security", None)
126         if not sec or not getattr(sec, "edge_auth_enabled", False):
127             return None  # DEFAULT-OFF ? no tenant identity, byte-identical to today
128 
129         # Strip so a whitespace-only value counts as unset ? matching the boot-time exposure
check in
130         # config/startup.py (EXPOSED_EDGE_AUTH_INCOMPLETE); one semantics for "configured",
both places.
131         team_domain = (getattr(sec, "cf_team_domain", "") or "").strip()
132         aud = (getattr(sec, "cf_access_aud", "") or "").strip()
133         if not team_domain or not aud:
134             raise EdgeAuthError(
135                 "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
configured"
136             )
137 
138         # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
support a plain
139         # dict too (tests) by trying both casings.
140         token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
141         if not token:
142             raise EdgeAuthError(
143                 f"missing {CF_ACCESS_HEADER} header (request did not pass Cloudflare
Access)"
144             )
145 
146         jwks = _get_jwks(team_domain, jwks_fetcher)
147         return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
148 
```

19. user (2026-07-02T02:49:50.088Z)

```
  227 backend/storage/gdrive_api.py
  148 backend/storage/gdrive.py
   72 backend/storage/local.py
   96 backend/storage/s3.py
  106 backend/storage/ref.py
  291 backend/config/loader.py
  324 backend/config/startup.py
  132 backend/config/presets.py
  496 backend/config/models.py
   68 backend/utils/paths.py
   83 backend/utils/freespace.py
```

```
  748 backend/worker/__main__.py
  261 backend/providers/gemini.py
 3052 total
```

20. user (2026-07-02T02:49:54.883Z)

```
1       """Google Drive **API** backend (OAuth / service-account) ? ``gdrive-api://`` (M12, D14
Northstar).
2
3       The cloud-native Drive round-trip: read a source from + write the stitched reel **back
to** a user's
4       Google Drive **via the Drive API** ? no local mount required (unlike the ``gdrive://``
backend, which
5       is plain local-FS over a *mounted* Drive for Desktop and stays untouched). This is the
headless path a
6       Cloud Run / GPU box uses.
7
8       ``gdrive-api://<path>`` names a path **relative to a root folder** (default the Drive
``root``, or
9       ``storage.gdrive-api.root_folder_id``); the backend resolves each path component to a
Drive file id via
10      ``files().list``. The Google libraries (``google-api-python-client`` + ``google-auth``)
are imported
11      **lazily** ? ``import backend.storage`` never pulls them ? and a ``service`` may be
**injected** for
12      fully-offline tests (mirroring ``GCSStorage(client=...)``).
13
14      ? **Credentials are never in config.** The real service is built from Application
Default Credentials
15      (a service account / workload identity) over the ``drive`` scope. **Per-user interactive
OAuth (the
16      consumer "connect your Drive" flow: client id/secret + refresh tokens) is OWNER-gated**
? this backend
17      ships the service-account/ADC build + the injected fake; the live per-user OAuth app is
a follow-up.
18
19      **Scope caveats (M12):** handles **binary** files (videos / reels / JSON) ? Google-
native Docs
20      (Sheets/Slides) have no direct download and would need ``files().export_media`` (out of
scope; export
21      them first). ``put`` is idempotent (deletes an existing file at the same path before
creating). Path
22      resolution keys on ``(name, parent)``; if Drive already holds duplicate-named files
under one folder
23      (a rare pre-existing state this backend never creates), resolution returns the first
match.
24      """
25
26      from __future__ import annotations
27
28      import io
29      import os
30      from datetime import datetime
31      from typing import Any, Iterator, Optional
32
33      from backend.storage.base import StorageBackend
34      from backend.storage.ref import StorageRef, StorageStat
35
36      _FOLDER_MIME = "application/vnd.google-apps.folder"
37      _DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"
38
39
40      def _q_escape(name: str) -> str:
41          """Escape a value for a Drive ``q`` string literal (single-quoted): ``\\`` then
``'``."""
```

```
        return name.replace("\\", "\\\\").replace("'", "\\'")


    class GDriveApiStorage(StorageBackend):
        """Drive-API-backed storage. Construct with an injected ``service`` (tests) or let
it build one
        from ADC (production). ``root_folder_id`` anchors relative ``gdrive-api://``
paths."""

        def __init__(
            self, *, service: Any = None, root_folder_id: str = "root", **_ignored
        ) -> None:
            self._root_id = root_folder_id or "root"
            if service is not None:
                self._service = service
                return
            self._service = self._build_service()

        # -- construction (real path; lazy Google imports)
---------------------------------------
        def _build_service(self) -> Any:
            try:
                import google.auth  # type: ignore
                from googleapiclient.discovery import build  # type: ignore
            except (
                ImportError
            ) as e:  # pragma: no cover - exercised only without the extra installed
                raise RuntimeError(
                    "the gdrive-api backend needs the 'storage-gdrive-api' extra "
                    "(google-api-python-client + google-auth). Install it on a host that
uses "
                    "gdrive-api:// URIs; credentials come from ADC (a service account /
workload "
                    "identity over the Drive scope) ? never from config."
                ) from e
            creds, _ = google.auth.default(scopes=[_DRIVE_SCOPE])
            return build("drive", "v3", credentials=creds, cache_discovery=False)

        def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:
            """The upload media object for ``files().create`` ? lazy ``MediaFileUpload``
(real path).
            A test seam: monkeypatch this to avoid the ``googleapiclient`` dependency."""
            from googleapiclient.http import (
                MediaFileUpload,  # type: ignore  # pragma: no cover
            )

            return MediaFileUpload(
                local_path, mimetype=content_type, resumable=True
            )  # pragma: no cover

        # -- path -> id resolution
-------------------------------------------------------------
        def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:
            """Walk ``key``'s path components from the root folder, returning the leaf's
file id (or
            ``None`` if a component is missing). With ``create_dirs`` EVERY missing
component is created as
            a folder ? ``put`` uses this on the parent-directory path (where all components
are folders),
            so a write into a fresh subfolder works. The read paths leave it ``False`` (a
missing component
            ? ``None``), never creating anything."""
            parent = self._root_id
            parts = [p for p in key.split("/") if p]
            for part in parts:
```

```
                q = (
                    f"name = '{_q_escape(part)}' and '{_q_escape(parent)}' in parents "
                    f"and trashed = false"
                )
                found = (
                    self._service.files()
                    .list(q=q, spaces="drive", fields="files(id, mimeType)")
                    .execute()
                    .get("files", [])
                )
                if found:
                    parent = found[0]["id"]
                elif create_dirs:
                    parent = (
                        self._service.files()
                        .create(
                            body={
                                "name": part,
                                "mimeType": _FOLDER_MIME,
                                "parents": [parent],
                            },
                            fields="id",
                        )
                        .execute()["id"]
                    )
                else:
                    return None
            return parent

        def _split(self, key: str) -> tuple[str, str]:
            """Split a ``gdrive-api://`` key into (parent dir path, leaf name)."""
            clean = key.strip("/")
            head, _, name = clean.rpartition("/")
            return head, name

        # -- StorageBackend interface
------------------------------------------------------------
        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError(
                    "GDriveApiStorage.fetch_to_local requires a dst path (no local mount)"
                )
            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            file_id = self._resolve_id(ref.key)
            if not file_id:
                raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
            data = (
                self._service.files().get_media(fileId=file_id).execute()
            )  # alt=media ? raw bytes
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            with io.open(dst, "wb") as fh:
                fh.write(data)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            parent_path, name = self._split(ref.key)
            parent_id = (
                self._resolve_id(parent_path, create_dirs=True)
                if parent_path
                else self._root_id
            )
            if parent_id is None:
                raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
```

```
            # Idempotent: Drive allows duplicate (name, parent), so a plain create() on a
re-put would
            # ORPHAN the prior file (and reads would keep returning the stale first match).
Delete an
            # existing file at this exact path first, so put() overwrites rather than
accumulates.
            existing = self._resolve_id(ref.key)
            if existing:
                self._service.files().delete(fileId=existing).execute()
            self._service.files().create(
                body={"name": name, "parents": [parent_id]},
                media_body=self._media_body(str(local_path), content_type),
                fields="id",
            ).execute()
            return ref

        def exists(self, ref) -> bool:
            return self._resolve_id(ref.key) is not None

        def stat(self, ref) -> StorageStat:
            file_id = self._resolve_id(ref.key)
            if not file_id:
                raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
            meta = (
                self._service.files()
                .get(fileId=file_id, fields="size, modifiedTime, md5Checksum, mimeType")
                .execute()
            )
            modified = None
            if meta.get("modifiedTime"):
                # Drive returns RFC-3339 (e.g. "2026-06-22T10:00:00.000Z"); normalize the
trailing Z.
                modified = datetime.fromisoformat(
                    meta["modifiedTime"].replace("Z", "+00:00")
                )
            return StorageStat(
                size=int(meta.get("size") or 0),
                modified=modified,
                etag=meta.get("md5Checksum"),
                content_type=meta.get("mimeType"),
            )

        def list(self, prefix) -> Iterator[StorageRef]:
            folder_id = (
                self._resolve_id(prefix.key) if prefix.key.strip("/") else self._root_id
            )
            if not folder_id:
                return
            base = prefix.key.strip("/")
            page_token: Optional[str] = None
            while True:
                resp = (
                    self._service.files()
                    .list(
                        q=f"'{_q_escape(folder_id)}' in parents and trashed = false",
                        spaces="drive",
                        fields="nextPageToken, files(id, name)",
                        pageToken=page_token,
                    )
                    .execute()
                )
                for f in resp.get("files", []):
                    key = f"{base}/{f['name']}" if base else f["name"]
                    yield StorageRef("gdrive-api", "", key, f"gdrive-api://{key}")
                page_token = resp.get("nextPageToken")
```

```
                if not page_token:
                    break

        def delete(self, ref) -> None:
            file_id = self._resolve_id(ref.key)
            if file_id:
                self._service.files().delete(fileId=file_id).execute()

```

21. user (2026-07-02T02:49:55.629Z)

```
      """Path-safety helpers for the API boundary (untrusted request fields)."""

      import os
      from typing import Any, Optional


      def output_base(config: Any) -> str:
          """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
          (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
          chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
          a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.

          This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
          ``gs://`` URI you can't ``makedirs``). Accepts the typed AppConfig or the legacy
dict-shim
          (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``"output"``."""
          try:
              out = (
                  config.output.output_dir
                  if not isinstance(config, dict)
                  else config.get("output", {}).get("output_dir")
              )
          except Exception:
              out = None
          return out or "output"


      def safe_output_filename(name: str) -> str:
          """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).

          `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
          and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
          primitive. Returns the name unchanged when safe; raises ValueError otherwise.
          """
          if not name or not name.strip():
              raise ValueError("output_filename must not be empty")
          if "\x00" in name:
              raise ValueError("output_filename must not contain a null byte")
          if "/" in name or "\\" in name:
              raise ValueError(f"output_filename must not contain a path separator: {name!r}")
          if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
              raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
          return name


      def resolve_under_root(path: str, root: str) -> str:
```

```
46          """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
47          rp = os.path.realpath(path)
48          rr = os.path.realpath(root)
49          try:
50              contained = os.path.commonpath([rp, rr]) == rr
51          except ValueError:  # different drives on Windows
52              contained = False
53          if not contained:
54              raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
55          return rp
56
57
58      def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
59          """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
60
61          With no allowed_root (default, single-user local mode) only the null-byte check
applies,
62          so the local 'point it at any of my videos' workflow is preserved. Setting an
63          allowed_root (a future hosted/tenant deployment) enforces containment.
64          """
65          if "\x00" in path:
66              raise ValueError("video_path must not contain a null byte")
67          if allowed_root:
68              resolve_under_root(path, allowed_root)
69
```

22. user (2026-07-02T02:50:06.135Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      import os
14      from pathlib import Path
15      from typing import Any
16
17      from pydantic import BaseModel
18
19      from backend.utils.app_home import default_config_path
20
21      from .models import AppConfig
22      from .presets import (
23          COMPUTE_TARGET_FOR_PROFILE,
24          PROFILE_PRESETS,
25          is_known_profile,
26          preset_for,
27          suggest_profile,
28      )
29
30      logger = logging.getLogger(__name__)
31
32      # Config path resolution now lives in ``backend.utils.app_home.default_config_path()`` ?
it honours
33      # ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")`` (CWD-
relative) when
34      # the env is unset, exactly the pre-P0a value. (The old module-level
```

```
``DEFAULT_CONFIG_PATH`` constant
    # was dropped: it had no external consumers and a frozen-at-import snapshot would
silently diverge
    # from a later RALLY_APP_HOME.)


    def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
        """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
        replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
        overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
        path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
        out = dict(base)
        for key, val in override.items():
            if isinstance(val, dict) and isinstance(out.get(key), dict):
                out[key] = _deep_merge(out[key], val)
            else:
                out[key] = val
        return out


    def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
        """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
        is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
        environment never silently selects a profile (and never silently spends on cloud).

        An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
        profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
        asserting a preset that was never applied. A bad value in config.json is left to
surface as
        a hard, typed-Literal error downstream (config-file correctness)."""
        env_profile = os.environ.get("DEPLOYMENT_PROFILE")
        if env_profile and env_profile.strip():
            ep = env_profile.strip()
            if is_known_profile(ep):
                return ep
            logger.warning(
                "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
                "and falling back to config.json deployment.profile.",
                ep,
                ", ".join(PROFILE_PRESETS),
            )
            # fall through to the file's profile (env typo must not suppress a valid file
choice)
        dep = file_data.get("deployment")
        if isinstance(dep, dict):
            prof = dep.get("profile")
            if isinstance(prof, str) and prof.strip():
                return prof.strip()
        return ""


    def _unknown_key_paths(
        data: dict[str, Any], model_cls: type[BaseModel], prefix: str = ""
    ) -> list[str]:
        """Dotted paths in `data` that the typed config will not honor.

        Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
        kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
```

```
        typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
        user's intent is lost ? collect every such key so load_config can warn.
        """
        unknown: list[str] = []
        fields = model_cls.model_fields
        for key, val in data.items():
            if key not in fields:
                unknown.append(prefix + key)
                continue
            ann = fields[key].annotation
            if (
                isinstance(val, dict)
                and isinstance(ann, type)
                and issubclass(ann, BaseModel)
            ):
                unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
        return unknown


    # Cached model defaults ? Pydantic field defaults are class-level constants
    # that never change at runtime. Initialised lazily on first call to
    # _get_builtin_defaults() so importing this module is still cheap.
    _BUILTIN_DEFAULTS_CACHE: dict[str, Any] | None = None


    def _get_builtin_defaults() -> dict[str, Any]:
        """Return a plain dict of the current built-in defaults.

        The AppConfig model is the single source of truth for defaults.
        Result is cached ? Pydantic field defaults never change at runtime.
        (The live config returned by load_config() is NOT cached ? it is
        a fresh AppConfig instance every call, reflecting config.json/env.)
        """
        global _BUILTIN_DEFAULTS_CACHE
        if _BUILTIN_DEFAULTS_CACHE is None:
            _BUILTIN_DEFAULTS_CACHE = AppConfig().model_dump(mode="json")
        return _BUILTIN_DEFAULTS_CACHE


    def load_config(path: str | Path | None = None) -> AppConfig:
        """Load configuration with the following precedence:

        1. Built-in defaults defined in the Pydantic models.
        2. Values from the JSON file (if present and readable).
        3. (Future) environment / CLI overrides.

        Missing file or unreadable file ? returns a fully defaulted AppConfig.
        Unknown keys in the file are tolerated (extra="allow" on the model).
        """
        cfg_path = Path(path) if path is not None else default_config_path()

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                logger.warning(
                    f"Failed to read {cfg_path}: {exc}. Using defaults + partial data."
                )

        # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`,
`null`) would
```

```
        # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
        # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
        # loader's "bad input is non-fatal" contract.
        if not isinstance(file_data, dict):
            logger.warning(
                "%s is not a JSON object (got %s); ignoring it and using defaults.",
                cfg_path,
                type(file_data).__name__,
            )
            file_data = {}

        if file_data:
            unknown = _unknown_key_paths(file_data, AppConfig)
            if unknown:
                logger.warning(
                    "[CONFIG WARNING] %s contains keys the typed config does not "
                    "recognize (typo'd or removed knobs ? top-level extras are kept "
                    "but unread; unknown section keys are silently dropped): %s",
                    cfg_path,
                    ", ".join(sorted(unknown)),
                )

        # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
        # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
        # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
        # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
        defaults = _get_builtin_defaults()
        profile = _resolve_explicit_profile(file_data)

        if profile and not is_known_profile(profile):
            # Only an unrecognised value from config.json reaches here (env typos already
fell back
            # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
            # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
            logger.warning(
                "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
                profile,
                ", ".join(PROFILE_PRESETS),
            )
            profile = ""

        if profile:
            merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
            # Record the profile actually applied ? the env may have chosen it over
config.json,
            # so re-stamp it onto the merged deployment section to keep the record honest.
            merged.setdefault("deployment", {})
            if isinstance(merged["deployment"], dict):
                merged["deployment"]["profile"] = profile
                ct = merged["deployment"].get("compute_target")
                canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
                if not ct and canonical:
                    # An active profile with no explicit compute_target (e.g. config.json
blanked it
                    # to "") gets its canonical target back ? the record never silently lies
about an
```

```
                    # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
                    merged["deployment"]["compute_target"] = canonical
                elif ct and canonical and ct != canonical:
                    logger.warning(
                        "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
                        "config.json overrides it to %r ? honoring the explicit override.",
                        profile,
                        canonical,
                        ct,
                    )
                logger.info(
                    "[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
                    profile,
                    merged["deployment"].get("compute_target"),
                )
        else:
            # No profile ? same deep merge so a config.json sub-key overrides its
            # default while NEW defaults added in the model since the file was written
            # survive ? no silent loss of new knobs on upgrade (D4).
            merged = _deep_merge(defaults, file_data)
            suggestion = (
                suggest_profile()
            )  # cheap, env-only (no torch); SUGGESTS only (D15)
            if suggestion:
                logger.debug(
                    "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
                    "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
                    "auto-applies ? D15).",
                    suggestion,
                )

        return AppConfig.model_validate(merged)


    def save_default_config(path: str | Path | None = None) -> Path:
        """Write a fresh default config.json.

        Used by scripts/doctor.py diagnostics and the --setup CLI path.
        Overwrites if the file exists.
        """
        cfg_path = Path(path) if path is not None else default_config_path()
        cfg = AppConfig()
        cfg_path.parent.mkdir(parents=True, exist_ok=True)
        cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
        return cfg_path


    # The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate O8).
    # Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies
skip_ai_handoff,
    # local storage and the compute_target, so this stays the handful of knobs a fresh, non-
technical
    # user gets ? a torch-free, offline, zero-setup experience:
    #   * default_segmenter "motion"  ? pure-OpenCV, no torch / GPU / weights / API key
(LIGHT tier);
    #   * profile "truly-local"        ? no Gemini handoff, local storage ? never surprise-
bills on
    #                                     Gemini nor returns 0 rallies on WASB-without-a-key
(O8 intent);
    #   * use_gpu false                ? honest for a LIGHT (no-GPU) box (motion ignores it
either way).
```

```
260     # The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
261     LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
262         "sport": "badminton",
263         "deployment": {"profile": "truly-local"},
264         "indexing": {"default_segmenter": "motion", "use_gpu": False},
265     }
266
267
268     def write_light_default_config(
269         path: str | Path | None = None, *, overwrite: bool = False
270     ) -> tuple[Path, bool]:
271         """Seed the batteries-included LIGHT default config (``LIGHT_DEFAULT_CONFIG``).
272
273         Returns ``(path, written)``. Unlike :func:`save_default_config` (which dumps a FULL
AppConfig
274         and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by
default ? **never
275         overwrites an existing file**, so a user's edits or the first-run wizard's choices
are preserved
276         (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent
dirs.
277         """
278         cfg_path = Path(path) if path is not None else default_config_path()
279         if cfg_path.exists() and not overwrite:
280             return cfg_path, False
281         cfg_path.parent.mkdir(parents=True, exist_ok=True)
282         cfg_path.write_text(
283             json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n", encoding="utf-8"
284         )
285         return cfg_path, True
286
287
288     # Backwards-compatible alias used by some older call sites during transition.
289     def get_default_config() -> dict[str, Any]:
290         """Return defaults as a plain dict (for transitional use only)."""
291         return _get_builtin_defaults()
292
```

23. user (2026-07-02T02:50:06.587Z)

```
1       """Free-space guards for the local box (STUDIO_MEDIA_LIFECYCLE.md P2, D11).
2
3       The single-box appliance fills its disk in three peaks: a chunked **upload** (raw file
into
4       ``upload_dir``), a **fetch-to-scratch** of a bucket/Drive source (a full local copy
under the temp
5       dir), and ? the real balloon ? **WSL frame extraction**, which can materialise ~15× the
source in
6       PNGs. D11: BLOCK on a local/WSL shortfall (buckets are warn-only elsewhere ? they aren't
7       capacity-bounded from here).
8
9       ``require_free_bytes`` is the HTTP-boundary guard: it raises **HTTP 507 (Insufficient
Storage)**
10      when the filesystem holding ``path`` can't hold ``need`` bytes, so the client fails fast
instead of
11      crashing a job mid-flight. It is best-effort about *measuring* (an unreadable
``disk_usage`` never
12      blocks ? inability to measure must not deny a legitimate request), but strict once a
real shortfall
13      is seen. The WSL frame-stage guard lives on ``WslCommandMixin`` (a different filesystem,
measured via
14      ``df`` inside WSL, not ``shutil.disk_usage``) ? see ``pipeline/detectors/base.py``.
15      """
16
17      from __future__ import annotations
```

```
18
19      import logging
20      import shutil
21      from typing import Optional
22
23      from fastapi import HTTPException
24
25      logger = logging.getLogger(__name__)
26
27
28      def free_bytes(path: str) -> Optional[int]:
29          """Best-effort free bytes on the filesystem holding ``path`` (``None`` if
undeterminable).
30
31          Never raises. ``shutil.disk_usage`` wants an existing path; we walk up to the
nearest existing
32          ancestor (an ``upload_dir``/scratch target may not exist yet at guard time)."""
33          import os.path
34
35          probe = path
36          for _ in range(64):  # bounded ancestor walk (defensive against a pathological path)
37              if not probe:
38                  return None
39              try:
40                  return int(shutil.disk_usage(probe).free)
41              except (OSError, ValueError):
42                  parent = os.path.dirname(probe.rstrip("/\\"))
43                  if parent == probe:  # reached the root and still can't stat ? give up
44                      return None
45                  probe = parent
46          return None
47
48
49      def require_free_bytes(
50          path: str, need: Optional[int], *, stage: str = "write"
51      ) -> None:
52          """Raise HTTP 507 if ``path``'s filesystem can't hold ``need`` bytes; otherwise
return.
53
54          ``need`` of ``None`` or ``<= 0`` is a no-op (nothing to reserve). Measuring is best-
effort: an
55          undeterminable free-space read logs a warning and proceeds (we never deny on
inability to
56          measure). A confirmed shortfall raises so the caller fails fast with an actionable
507."""
57          if not need or need <= 0:
58              return
59          free = free_bytes(path)
60          if free is None:
61              logger.warning(
62                  "free-space guard (%s): could not read free space at %s ? proceeding.",
63                  stage,
64                  path,
65              )
66              return
67          if free < need:
68              need_mb = need / (1024 * 1024)
69              free_mb = free / (1024 * 1024)
70              logger.warning(
71                  "free-space guard (%s): %s needs %.0f MB but only %.0f MB free ? refusing
(507).",
72                  stage,
73                  path,
74                  need_mb,
75                  free_mb,
```

```
76              )
77              raise HTTPException(
78                  status_code=507,
79                  detail=(
80                      f"Not enough free disk space for {stage}: need ~{need_mb:.0f} MB, "
81                      f"only ~{free_mb:.0f} MB free. Free some space and retry."
82                  ),
83              )
84
```

24. user (2026-07-02T02:51:06.791Z)

```
backend\worker\__main__.py:209:    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-
worker-")
backend\worker\__main__.py:473:    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-
train-")
backend\providers\gemini.py:28:# are ever deleted by cleanup_orphaned_files().
backend\providers\gemini.py:53:        """Best-effort cleanup of an uploaded File (never
raises)."""
backend\providers\gemini.py:59:    def cleanup_orphaned_files(self, grace_sec: int = 3600) ->
int:
backend\providers\gemini.py:62:        leaves the uploaded clip behind forever ? the per-call
cleanup only covers paths
backend\storage\__init__.py:89:    scratch = tempfile.mkdtemp(prefix="rally-input-",
dir=scratch_parent)
backend\utils\annotation_proxy.py:47:def _cleanup(path: str) -> None:
backend\utils\annotation_proxy.py:84:        _cleanup(tmp)
backend\utils\annotation_proxy.py:90:        _cleanup(tmp)
backend\utils\annotation_proxy.py:96:        _cleanup(tmp)
backend\compute\cloud_run.py:220:                tempfile.mkdtemp(prefix="cr-read-"), ref.name
or "obj.json"
backend\eval\golden_real_fixtures.py:355:        tempfile.mkdtemp(prefix=f".{out_slug}.tmp.",
dir=str(fixtures_dir))
backend\main.py:892:    with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
backend\main.py:1483:# sequential-part logic, and abort cleanup ? is unchanged.
backend\main.py:1823:        deleted = provider.cleanup_orphaned_files()
backend\pipeline\stitcher\compiler.py:419:        with
tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
backend\pipeline\detectors\native_wasb_runner.py:175:        """Best-effort recursive delete of
a native path (post-success cleanup; never raises)."""
backend\pipeline\segmenters\gemini.py:129:        Returns (video_to_upload, cleanup_temp_files)
where cleanup_temp_files removes any
backend\pipeline\segmenters\gemini.py:183:        def cleanup_temp_files():
backend\pipeline\segmenters\gemini.py:190:        return video_to_upload, cleanup_temp_files
backend\pipeline\segmenters\gemini.py:203:        cleanup_temp_files: Callable[[], None],
backend\pipeline\segmenters\gemini.py:375:        cleanup_temp_files()
backend\pipeline\segmenters\gemini.py:394:        cleanup_temp_files: Callable[[], None],
backend\pipeline\segmenters\gemini.py:428:        cleanup_temp_files()
backend\pipeline\segmenters\gemini.py:630:        # Optionally trim the video first (debug
feature); cleanup removes any temp file.
backend\pipeline\segmenters\gemini.py:631:        video_to_upload, cleanup_temp_files =
self._maybe_trim_video(
backend\pipeline\segmenters\gemini.py:655:        # video keeps its own dir, so cleanup here
never removes another run's chunks.
backend\pipeline\segmenters\gemini.py:669:                cleanup_temp_files,
backend\pipeline\segmenters\gemini.py:678:                cleanup_temp_files,
backend\pipeline\detectors\base.py:162:            f"Free space (e.g. python -m
tools.cleanup_caches) and retry."
backend\pipeline\detectors\base.py:166:        """Best-effort recursive delete of WSL paths
(post-success cleanup; never raises)."""
backend\pipeline\detectors\base.py:174:            logger.warning("WSL cache cleanup failed
(non-fatal): %s", e)
```

25. user (2026-07-02T02:51:07.442Z)

```
"""Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.

    python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
        [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
    python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)

The URI scheme selects the storage backend via ``backend.storage`` (a bare path / ``local://``
is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing local runs
are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).

This is a thin orchestrator: it composes the SAME building blocks the ``backend.main`` CLI uses
(validator registry ? segmenter ? report), never a forked pipeline.
"""

from __future__ import annotations

import argparse
import csv
import json
import logging
import os
import re
import sys
import tempfile
from typing import Any, List

from backend import storage
from backend.compute import ComputeJobSpec, get_compute_backend
from backend.config import (
    StartupCheckError,
    enforce_startup_checks,
    load_config,
    probe_cuda_available,
)
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.validators.base import registry as validator_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure
from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
from backend.utils.hashing import compute_video_id

logger = logging.getLogger("backend.worker")


def _coerce(v: str) -> Any:
    """Coerce a ``--set`` string value to bool/int/float where it round-trips, else str."""
    low = v.lower()
    if low in ("true", "false"):
        return low == "true"
    for cast in (int, float):
        try:
            return cast(v)
        except ValueError:
            pass
    return v


```

```
    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow(
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
                )


    def _write_report_json(
        video_id: str, segments: List[dict], status: str, path: str
    ) -> None:
        with open(path, "w") as f:
            json.dump(
                {
                    "video_id": video_id,
                    "status": status,
                    "segment_count": len(segments),
                    "segments": [
                        {
                            "start": float(s.get("start_time", 0.0)),
                            "end": float(s.get("end_time", 0.0)),
                        }
                        for s in segments
                    ],
                },
                f,
                indent=2,
            )


    def _run_startup_checks(config: Any) -> bool:
        """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
        best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
        cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
        (returns True, ZERO work) when no deployment.profile is selected.

        The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
```

```
        torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
        WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
        no-op of work, not just of output."""
        profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
        cuda = (
            probe_cuda_available() if profile else None
        )  # torch-free on the default-OFF path
        try:
            enforce_startup_checks(config, cuda_available=cuda, logger=logger)
            return True
        except StartupCheckError:
            return False  # already logged at ERROR by enforce_startup_checks


    def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
        """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
        outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
        container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.

        A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
        poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
        timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
        from backend.infrastructure.execution_policy import PolicyTimeout

        spec = ComputeJobSpec(
            video_uri=args.video_uri,
            out_uri=args.out_uri,
            segmenter=args.segmenter,
            config_overrides=tuple(args.set or ()),
        )
        try:
            result = compute.run_infer(spec)
        except PolicyTimeout as e:
            logger.warning(
                "[worker] remote compute did not complete (no marker within budget): %s", e
            )
            return 4
        logger.info(
            "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
            result.video_id,
            result.returncode,
            result.outputs_uri,
        )
        return result.returncode


    def _write_done_marker(
        done_uri: str,
        video_id: str,
        returncode: int,
        segment_count: int,
        status: str,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
```

```
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success return."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline (the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download to scratch.
        local_video = storage.fetch(
            args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
        )
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI).
        val_results, val_context = validator_registry.run_all_with_context(
            local_video, config
        )
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(
            video_id,
            local_video,
            "failed" if failures else "processed",
            [r.model_dump() for r in val_results],
        )
```

```
234
235         def _fail(rc: int) -> int:
236             # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
237             # dispatcher's bucket-poll terminates FAST + accurately instead of hanging until
its poll
238             # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
239             if getattr(args, "done_uri", None):
240                 _write_done_marker(
241                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
242                 )
243             return rc
244
245         segments: List[dict] = []
246         segmenter_actual = None
247         segmentation_ran = False
248         status = "failed"
249         try:
250             if failures:
251                 print(
252                     "[worker] validation FAILED: "
253                     + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
254                 )
255                 return _fail(2)
256             seg_name = args.segmenter or config.indexing.default_segmenter
257             segmenter_cls = segmenter_registry.get(seg_name)
258             if not segmenter_cls:
259                 print(
260                     f"[worker] unknown segmenter: {seg_name!r} "
261                     f"(have {list(segmenter_registry.list_available())})"
262                 )
263                 return _fail(2)
264             segmenter_actual = seg_name
265             result = segmenter_cls(db, config).process_video(
266                 video_id, local_video, max_frames=args.max_frames
267             )
268             segmentation_ran = True
269             if isinstance(result, Failure):
270                 print(f"[worker] segmenter FAILED: {result.message}")
271                 return _fail(3)
272             segments = result.value if hasattr(result, "value") else result
273             status = "success"
274             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
275             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
276             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
277             if not segments:
278                 detector_impl = (
279                     os.environ.get("RALLY_DETECTOR_IMPL")
280                     or getattr(config.indexing, "detector_impl", None)
281                     or "auto"
282                 )
283                 logger.warning(
284                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
285                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
286                     "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
287                     "the WASB env produced no detections, or the input resolution differs
from the tuned "
288                     "proxy resolution. Re-run with -v for the native command + frame
```

```
counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(
            video_id, segments, status, os.path.join(out_local, "report.json")
        )

        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)

        print(
            f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
        )
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
            )
        return 0


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

    #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
```

```
DD>_<name>.rallies.csv``,
    #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
    #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
    _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")


    def _label_clip_name(fname: str) -> str:
        """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.

        Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
        canonical dated label and a legacy bare label resolve to the SAME bare clip stem:

        * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
        * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
        * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
        * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)

        Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
        video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
        """
        name = fname.replace(".rallies.csv", "").replace(".csv", "")
        return _LABEL_DATE_PREFIX.sub("", name)


    def _probe_video_fps_width(path: str):
        """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.

        The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
        fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
        cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
        inference report (it flags the fallback) but corrupting for REAL training data. So
here we
        probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
        rather than guesses.
        """
        import json
        import subprocess

        try:
            out = subprocess.check_output(
                [
                    ffprobe_bin(),
                    "-v",
                    "error",
                    "-select_streams",
                    "v:0",
                    "-show_entries",
                    "stream=r_frame_rate,width",
                    "-of",
                    "json",
                    path,
                ],
```

```
                stderr=subprocess.DEVNULL,
            ).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
        except (
            subprocess.SubprocessError,
            ValueError,
            KeyError,
            OSError,
            json.JSONDecodeError,
        ):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo) has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(
                f"[worker] unknown segmenter: {args.segmenter!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
```

```
shell out
456         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
457
458         Two trajectory sources (the train pipeline + harness are identical either way):
459           * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
460             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
461           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
462             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
463             is the M3.5 "first real training" path; only the trajectory source flips.
464         """
465         import subprocess
466
467         config = load_config(args.config) if args.config else load_config()
468         _apply_overrides(config, args.set)
469         if not _run_startup_checks(config):
470             return 2
471         storage_cfg = config.storage.model_dump()
472
473         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
474         labels_dir = os.path.join(scratch, "labels")
475         traj_dir = os.path.join(scratch, "trajectories")
476         videos_dir = os.path.join(scratch, "videos")
477         os.makedirs(labels_dir, exist_ok=True)
478         os.makedirs(traj_dir, exist_ok=True)
479
480         corpus = args.corpus_uri.rstrip("/")
481         label_uris = sorted(
482             u
483             for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
484             if u.lower().endswith(".csv")
485         )
486         if len(label_uris) < 2:
487             print(
488                 f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
489                 f"under {corpus}/labels/"
490             )
491             return 2
492
493         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
494         runner = None
495         video_by_stem: dict = {}
496         if args.trajectory_source == "detector":
497             os.makedirs(videos_dir, exist_ok=True)
498             runner = _resolve_train_detector(args, config)
499             if runner is None:
500                 return 2
501             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
502                 vname = storage.parse_uri(vuri).name
503                 if not vname.lower().endswith(_VIDEO_EXTS):
504                     continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
505                 video_by_stem[os.path.splitext(vname)[0]] = vuri
506
507         print(f"[worker] trajectory source: {args.trajectory_source}")
508         specs: List[dict] = []
509         for uri in label_uris:
510             fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
511             name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
512             local_label = storage.fetch(
```

```
                uri, os.path.join(labels_dir, fname), config=storage_cfg
            )
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(
                        f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
                    )
                    continue
                local_video = storage.fetch(
                    vuri,
                    os.path.join(videos_dir, storage.parse_uri(vuri).name),
                    config=storage_cfg,
                )
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(
                        f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
                    )
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(
                        f"[worker] detector produced no trajectory for {name!r} ? skipping."
                    )
                    continue
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": fps,
                        "frame_width": frame_width,
                    }
                )
            else:
                from backend.pipeline.detectors.stub_runner import (
                    synth_trajectory_from_labels,
                )

                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(
                    local_label, traj, fps=args.fps, frame_width=args.frame_width
                )
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": args.fps,
                        "frame_width": args.frame_width,
                    }
                )

        if len(specs) < 2:
            print(
                f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
```

```
            )
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = (
            "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
        )
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [
            sys.executable,
            "-m",
            "training.gen0.harness",
            "--manifest",
            manifest_path,
            "--out",
            out_dir,
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            prog="python -m backend.worker",
            description="Storage-aware worker: pull ? run pipeline ? push.",
        )
        p.add_argument(
            "-v",
            "--verbose",
            action="store_true",
            help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
        )
        sub = p.add_subparsers(dest="cmd", required=True)

```

```
        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument(
            "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
        )
        pi.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (outputs/<video_id>/? is appended)",
        )
        pi.add_argument(
            "--segmenter", default=None, help="default: indexing.default_segmenter"
        )
        pi.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pi.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument(
            "--done-uri",
            default=None,
            help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
            "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
            "Default-OFF: omit it for the normal local run.",
        )
        pi.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set indexing.skip_ai_handoff=true",
        )
        pi.set_defaults(func=cmd_infer)

        pt = sub.add_parser(
            "train",
            help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
        )
        pt.add_argument(
            "--corpus-uri",
            required=True,
            help="prefix containing labels/ (+ videos/ for "
            "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
        )
        pt.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (weights/<version>/ is appended)",
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
            "--segmenter",
            default="wasb_hybrid",
            help="trajectory hybrid whose detector is used for --trajectory-source
detector",
        )
```

```
702         pt.add_argument(
703             "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
704         )
705         pt.add_argument("--config", default=None)
706         pt.add_argument("--scratch", default=None)
707         pt.add_argument(
708             "--fps",
709             type=float,
710             default=30.0,
711             help="synth trajectory fps (detector probes the video)",
712         )
713         pt.add_argument(
714             "--frame-width",
715             type=float,
716             default=1280.0,
717             help="synth trajectory frame width (detector probes the video)",
718         )
719         pt.add_argument("--set", action="append", default=[], metavar="k=v")
720         pt.set_defaults(func=cmd_train)
721         return p
722
723
724     def _setup_logging(verbose: bool) -> None:
725         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
726         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
727         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
728         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
729         entry), not at import, so importing the module never reconfigures a caller's
logging."""
730         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
731         # configured logging at import), which would leave the level at WARNING and re-
suppress the
732         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
733         logging.basicConfig(
734             level=logging.DEBUG if verbose else logging.INFO,
735             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
736             datefmt="%H:%M:%S",
737             force=True,
738         )
739
740
741     def main(argv=None) -> int:
742         args = build_parser().parse_args(argv)
743         _setup_logging(getattr(args, "verbose", False))
744         return args.func(args)
745
746
747     if __name__ == "__main__":
748         sys.exit(main())
749
```

26. user (2026-07-02T02:51:18.322Z)

```
1       import json
2       import logging
3       import os
4       import time
5       from datetime import datetime, timedelta, timezone
6       from typing import Tuple
```

```

    from google import genai
    from google.genai import types

    from backend.infrastructure.execution_policy import (
        ExecutionPolicy,
        PolicyTimeout,
        poll_until,
        run_bounded,
    )
    from backend.providers.base import AIVideoProvider

    logger = logging.getLogger(__name__)

    # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
    # each uploaded clip is below this. Oversize uploads are refused (not silently
attempted).
    MAX_UPLOAD_MB = 500

    # O1/P11: every upload is tagged with this display_name prefix so the startup orphan
    # sweep can distinguish OUR files from other apps sharing the same Gemini API key
    # (the File-API namespace is account-global, per-key). Only files matching this prefix
    # are ever deleted by cleanup_orphaned_files().
    UPLOAD_TAG = "rally-indexer"

    # Per-request HTTP timeout (ms) for the Gemini client ? bounds list/upload/delete so a
    # hung TCP connection can never stall startup (the sweep) or block a worker
indefinitely.
    # analyze_video's generate/poll already run under ExecutionPolicy; this covers the rest.
    _HTTP_TIMEOUT_MS = 30_000


    class GeminiProvider(AIVideoProvider):
        def __init__(self, api_key: str, model_name: str):
            super().__init__(api_key, model_name)
            # Initialize client lazily to avoid connection/auth issues during class
instantiation
            self._client = None

        @property
        def client(self):
            if self._client is None:
                self._client = genai.Client(
                    api_key=self.api_key,
                    http_options=types.HttpOptions(timeout=_HTTP_TIMEOUT_MS),
                )
            return self._client

        def _safe_delete(self, video_file) -> None:
            """Best-effort cleanup of an uploaded File (never raises)."""
            try:
                self.client.files.delete(name=video_file.name)
            except Exception:
                pass

        def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
            """Sweep and delete video files orphaned on the Gemini File API by a previous
            crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
            leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
            that reach a ``finally``/except).

            Scoped to OUR uploads: only files whose ``display_name`` matches ``UPLOAD_TAG``
are
            considered, so the sweep never touches another app's files under a shared API
key
```

```
67              (the File-API namespace is account-global, per-key).
68
69              Best-effort: lists files, deletes any of ours older than ``grace_sec`` (so we
never
70              race a concurrent worker mid-upload). Bounded by the client HTTP timeout. Never
raises;
71              returns the count deleted. Intended to run once at startup.
72
73              Args:
74                  grace_sec: only delete files older than this (default 1h). Protects a file a
75                      live worker may still be processing.
76              """
77              cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
78              deleted = 0
79              try:
80                  files = list(self.client.files.list())
81              except Exception as e:
82                  logger.warning(
83                      "[Gemini GC] failed to list remote files (skipping sweep): %s", e
84                  )
85                  return 0
86              for f in files:
87                  try:
88                      # Scope to OUR uploads only (display_name tag) ? never another app's
files.
89                      if getattr(f, "display_name", None) != UPLOAD_TAG:
90                          continue
91                      ct = getattr(f, "create_time", None)
92                      if ct is None:
93                          continue  # No timestamp ? can't prove it's stale; leave it (safe
default).
94                      # The SDK returns a tz-aware datetime; fall back to parsing a string
form.
95                      if isinstance(ct, datetime):
96                          created = ct if ct.tzinfo else ct.replace(tzinfo=timezone.utc)
97                      else:
98                          created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
99                      if created > cutoff:
100                         continue  # too recent ? a live worker may own it
101                     self.client.files.delete(name=f.name)
102                     deleted += 1
103                 except Exception as e:
104                     logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
105             if deleted:
106                 logger.info(
107                     "[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
108                     deleted,
109                     grace_sec,
110                 )
111             return deleted
112
113         def analyze_video(
114             self, video_path: str, prompt: str, chunk_duration: float = 0.0, label: str = ""
115         ) -> Tuple[list, dict]:
116             log_prefix = f"Gemini {label}" if label else "Gemini"
117
118             metrics: dict = {
119                 "upload_time": 0.0,
120                 "process_time": 0.0,
121                 "gen_time": 0.0,
122             }
123
124             # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
125             try:
```

```
126                 size_mb = os.path.getsize(video_path) / (1024 * 1024)
127             except OSError:
128                 size_mb = 0.0
129             if size_mb > MAX_UPLOAD_MB:
130                 logger.error(
131                     f"[{log_prefix}] {video_path} is {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap
? "
132                     f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
133                     f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or chunk
smaller."
134                 )
135                 metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
136                 metrics["oversize_skipped"] = True
137                 return [], metrics
138 
139             # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
140             logger.info(f"[{log_prefix}] Uploading {video_path}...")
141             t_upload_start = time.time()
142             video_file = None
143             for attempt in range(3):
144                 try:
145                     video_file = self.client.files.upload(
146                         file=video_path,
147                         config=types.UploadFileConfig(display_name=UPLOAD_TAG),
148                     )
149                     metrics["upload_time"] = time.time() - t_upload_start
150                     logger.info(
151                         f"[{log_prefix}] Upload complete in {metrics['upload_time']:.1f}s.
URI: {video_file.uri}"
152                     )
153                     break
154                 except Exception as e:
155                     logger.warning(
156                         f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed: {e}"
157                     )
158                     if attempt < 2:
159                         time.sleep(2 * (attempt + 1))
160             if video_file is None:
161                 logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
162                 metrics["error"] = "upload failed after 3 attempts"
163                 return [], metrics
164 
165             # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
166             # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
167             t_process_start = time.time()
168             process_policy = ExecutionPolicy(
169                 poll_every_n_sec=10.0, max_wait_sec=1200.0
170             )  # 20 min cap
171 
172             def _processed():
173                 nonlocal video_file
174                 video_file = self.client.files.get(name=video_file.name)
175                 return None if video_file.state.name == "PROCESSING" else video_file
176 
177             try:
178                 video_file = poll_until(
179                     _processed,
180                     process_policy,
181                     label=f"{log_prefix} processing",
182                     logger=logger,
183                 )
184             except PolicyTimeout:
```

```
185                 logger.error(
186                     f"[{log_prefix}] Processing exceeded {process_policy.max_wait_sec:.0f}s
? giving up."
187                 )
188                 self._safe_delete(video_file)
189                 metrics["error"] = (
190                     f"processing timed out after {process_policy.max_wait_sec:.0f}s"
191                 )
192                 return [], metrics
193             except Exception as e:
194                 logger.error(f"[{log_prefix}] Status check failed: {e}")
195                 self._safe_delete(video_file)
196                 metrics["error"] = f"status check failed: {e}"
197                 return [], metrics
198             if video_file.state.name == "FAILED":
199                 logger.error(
200                     f"[{log_prefix}] Video processing failed: {video_file.error.message}"
201                 )
202                 self._safe_delete(video_file)
203                 metrics["error"] = f"video processing failed: {video_file.error.message}"
204                 return [], metrics
205
206             metrics["process_time"] = time.time() - t_process_start
207             logger.info(
208                 f"[{log_prefix}] Processing finished in {metrics['process_time']:.1f}s."
209             )
210
211             # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
killer that was
212             # missing: a generate() that never returns previously blocked forever) + retry
the 503/network
213             # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).
214             logger.info(
215                 f"[{log_prefix}] Submitting multimodal request to {self.model_name}..."
216             )
217             t_gen_start = time.time()
218             gen_policy = ExecutionPolicy(
219                 op_timeout_sec=180.0, max_retries=5, backoff_base_sec=3.0
220             )
221             raw_text = None
222
223             def _generate():
224                 return self.client.models.generate_content(
225                     model=self.model_name,
226                     contents=[video_file, prompt],
227                     config=types.GenerateContentConfig(
228                         response_mime_type="application/json", temperature=0.0
229                     ),
230                 )
231
232             try:
233                 response = run_bounded(
234                     _generate, gen_policy, label=f"{log_prefix} generate", logger=logger
235                 )
236                 raw_text = response.text
237                 metrics["gen_time"] = time.time() - t_gen_start
238                 logger.info(
239                     f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}"
240                 )
241             except Exception as e:  # PolicyTimeout (hang) or the last 503 ? both end here
242                 logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
243                 metrics["error"] = f"generate failed after retries: {e}"
244                 metrics["generate_failed"] = True
```

```
245             self._safe_delete(video_file)
246
247             if not raw_text:
248                 return [], metrics
249
250             # 4. Parse JSON
251             try:
252                 segments = json.loads(raw_text)
253                 if not isinstance(segments, list):
254                     logger.error(f"[{log_prefix}] Response is not a JSON array.")
255                     metrics["error"] = "response is not a JSON array"
256                     return [], metrics
257                 return segments, metrics
258             except Exception as e:
259                 logger.error(f"[{log_prefix}] JSON parse failed: {e}")
260                 metrics["error"] = f"JSON parse failed: {e}"
261                 return [], metrics
262
```

27. user (2026-07-02T02:52:21.953Z)

```
272-    default_highlights_name: str = "highlights.mp4"
273-    db_name: str = "sports_indexer.db"
274-    # Directory where the DB, highlight reels, and processed clips are written.
275-    # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
276:    output_dir: str = "output"
277-
278-
279-class WatermarkConfig(BaseModel):
280-    """Branding overlay burned into each highlight clip during the per-clip re-encode
--
331-
332-    # --- Chunked upload (B2, PR-2) ---
333-    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
334-    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
335:    # contain upload_dir or /api/process will reject the uploaded paths.
336:    upload_dir: str = "output/uploads"
337-    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
338-    # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
339-    max_upload_mb: int = 4096
340-    max_part_mb: int = 95
--
365-    **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
366-    (boto3/google auth chains supply credentials). See ``backend/storage/``."""
367-
368-    backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
369:    # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
370-    output_root: str = ""
371-    # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
372-    # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is read
IN PLACE
373-    # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
--
383-    # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the root
384-    # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5). When
OFF
385-    # the legacy flat layout is used unchanged.
386-    single_folder_per_video: bool = False
387:    # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
388-    workspace_root: str = ""
389-    # Cloud namespace prefix under the root so per-video folders sit tidily beside
```

```
390-    # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
391-    workspace_prefix: str = "workspaces"
```

28. user (2026-07-02T02:52:22.226Z)

```
1       """Resolved application-data HOME for a packaged, batteries-included install.
2
3       PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4       the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5       ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
from
6       the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
silently
7       "loses" their index. This module introduces ONE resolver every state path roots on.
8
9       Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10        * ``RALLY_APP_HOME`` env set  ? that directory is the app home (the packaged launcher
sets it).
11        * ``RALLY_APP_HOME`` unset    ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.
12
13      The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
``~/.local/share``)
14      is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15      calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16      this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17      real user profile during a test run).
18
19      stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20      the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21      """
22
23      from __future__ import annotations
24
25      import os
26      import platform
27      from pathlib import Path
28
29      #: Environment variable the packaged launcher sets to pin the app-data home.
30      APP_HOME_ENV = "RALLY_APP_HOME"
31
32      #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33      APP_DIR_NAME = "Khelsutra"
34
35
36      def app_home() -> Path:
37          """The resolved application-data home.
38
39          ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40          to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41          """
42          env = os.environ.get(APP_HOME_ENV)
43          if env and env.strip():
44              return Path(env.strip()).expanduser()
45          return Path(".")
46
47
48      def app_path(*parts: str | os.PathLike[str]) -> Path:
```

```
        """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
        return app_home().joinpath(*[str(p) for p in parts])


    def default_config_path() -> Path:
        """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.

        With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
        """
        return app_home() / "config.json"


    def default_env_path() -> Path:
        """``<app-home>/.env`` ? the dotenv file the server prefers (e.g.
``GEMINI_API_KEY``)."""
        return app_home() / ".env"


    def env_file_to_load() -> Path | None:
        """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
DEFAULT
        (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
behaviour.

        Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
        env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
        prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
        """
        home = app_home()
        if (
            str(home) == "."
        ):  # env unset (or pinned to CWD) ? historical load_dotenv() search
            return None
        candidate = home / ".env"
        return candidate if candidate.exists() else None


    def resolve_output_dir(output_dir: str) -> str:
        """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.

        With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
``"output"`` when
        empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
pathlib
        normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir
like
        ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
        app's state; an absolute path ? or a Windows drive-anchored one (``C:\\x`` /
``C:x``) ? passes
        through untouched.
        """
        home = app_home()
        if str(home) == ".":  # env unset (or pinned to CWD) ? verbatim, today's behaviour
            return output_dir or "output"
        if not output_dir:
            return str(home / "output")
        if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
            return output_dir
```

```
100         return str(home / output_dir)
101
102
103     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
104         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
105         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
106
107         * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
108         * macOS   ? ``~/Library/Application Support/<app_name>``
109         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)
110         """
111         system = platform.system()
112         if system == "Windows":
113             base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
114         elif system == "Darwin":
115             base = str(Path.home() / "Library" / "Application Support")
116         else:
117             base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
118         return Path(base) / app_name
119
```

29. user (2026-07-02T02:52:47.232Z)

```
docs\ANALYZERS_AND_RECONCILERS.md:618:  surfaced); claim cautiously, validate from scratch.
docs\ANALYZERS_AND_RECONCILERS.md:711:  the high from-scratch-validation risk) and the measured-
before-apply reconciler bundle (also absence-based).
docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md:95:> polls. See
`scratch/PR1_HANDOFF.md` for the review bundle.
docs\CODE_MAP.md:34:| A throwaway repro / debug script | `scratch/` | **Gitignored** + excluded
from ruff/pytest. Never imported by app code. |
docs\CODE_MAP.md:116:[Omitted long matching line]
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\lovo_resweep.py:9:Reproduce: add
any new clip via scratch/ingest_golden.py, then run this. The sys.path insert
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\README.md:59:  via
`scratch/ingest_golden.py` first). It self-reports the cap sweep + LOVO + R4 + significance.
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md:28:- IoU 0.5. Repro:
`scratch/ab_n6_lovo.py` (machine-side; reads the gitignored manifest + the external
docs\archives\decisions\DECISIONS.md:680:- **`deploy/digitalocean/` scripts are KEPT** ?
`bootstrap.sh` / `mount_scratch.sh` / `gpu_setup.sh` are **provider-agnostic** Linux-GPU setup
the GCP runbook reuses; only the DO-specific *provisioning* (doctl droplet create, Spaces) is
dropped. (Optional later cleanup: move them to `deploy/common/`.)
docs\COMPUTE_DECOUPLED_SERVING\README.md:118:| M3 | Configurable output/scratch base ? kill
hardcoded `output/` (`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays
`output/`**; golden + stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? |
[#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |
docs\COMPUTE_DECOUPLED_SERVING\README.md:161:- 2026-06-21 ? **M2 landed**
([#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280)):
`input_backend`/`input_root` + URI-aware main API/CLI input via
`resolve_input_uri`/`acquire_local_input` (local = identity passthrough, byte-for-byte
unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted
on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report
provenance. Suite 1105 green, cov 84.84%.
docs\COMPUTE_DECOUPLED_SERVING\README.md:162:- 2026-06-21 ? **M3 landed**
([#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282)): configurable
output/scratch base via `backend/utils/paths.output_base(config)` ? the 3 segmenters'
chunk/handoff/temp dirs root under `output.output_dir` (default `"output"`, byte-identical;
default-OFF ? no Launch Report). Fixes the latent bug where scratch ignored `--output-dir`.
Suite 1114 green, cov ~84.9%.
docs\archives\past_projects\DECOUPLED_COMPUTE\02-COMPUTE-MAP-AND-CONTRACT.md:23:  image, a TF
image) sharing a `/scratch` volume. So the container topology itself is non-trivial.
docs\archives\past_projects\DECOUPLED_COMPUTE\03-PLATFORM-OPTIONS.md:33:`/scratch` and decode
*there*; do **not** FUSE-mount and random-seek over HTTP. Outputs can stream back via
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md:20:   - sync-first:
rclone cp bucket ? /scratch (local NVMe); decode/detect on GPU
```

docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md:21:   - write artifacts
? /scratch ? upload to bucket; checkpoint train ?30 min
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md:38:2. `rclone copy` the
corpus to `/scratch` (sync-first).
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md:48:1. `rclone copyto
$VIDEO_URI /scratch/in.mp4` ? **sync-first, not a FUSE random-access decode.**
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md:120:WSL box. (WASB and
TrackNetV4 are torch vs TF ? keep them as separate runners/images sharing `/scratch`;
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md:158: WSL into an in-
container module reading frames from `/scratch`; **vendor** the research repos + weights;
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md:161: Torch and TF talk
over a shared `/scratch` volume ? **never co-install TF + torch in one image** (segfault).
docs\DATA_IN_GCS.md:148:`trajectories/` + `labels/`). Sources (resolved 2026-06-22 ? see
`scratch/copy_videos_to_gcs.ps1`):
docs\DATA_IN_GCS.md:160:- **Interim (today, owner box ? streams Drive/F: ? local ? GCS):** `pwsh
scratch/copy_videos_to_gcs.ps1
docs\DATA_IN_GCS.md:198:> `scratch/`; promote it to `scripts/` if corpus-sync recurs).
docs\DATA_IN_GCS.md:205:> script (`scratch/copy_videos_to_gcs.ps1`) stages such paths through a
bracket-free temp file; the
docs\DATA_IN_GCS.md:230:5. **Reproducibility:** the migration + copy helpers live in gitignored
`scratch/`
docs\archives\past_projects\DECOUPLED_COMPUTE\06-RISKS-AND-OPEN-QUESTIONS.md:24:2. **TF + torch
in one image segfaults.** *Mitigation:* two single-framework images sharing `/scratch`.
docs\archives\past_projects\DECOUPLED_COMPUTE\HANDOFF.md:55:| #253 | GPU provisioning ? NVMe
scratch + **native WASB setup** + port runbook |
`deploy/digitalocean/{bootstrap,mount_scratch,gpu_setup}.sh` |
docs\archives\past_projects\DECOUPLED_COMPUTE\HANDOFF.md:87:2. **Port** (`SETUP_NEW_MACHINE.md`
§9): git-archive ? scp ? `mount_scratch.sh` ? `bootstrap.sh --gpu` ? `gpu_setup.sh`
docs\data_pipeline\GOLDEN_VIDEOS.md:23: (`gts` + candidates), append a row to
`output/human_lovo_manifest.json`. (`scratch/at_par.py` scores a
docs\EXECUTION_POLICY.md:117:(`scratch/run_gx010125_full.py`, where a 7-min cap had to be hand-
rolled).
docs\PACKAGING_PLAN.md:154:| **CUJ-2** In-UI `gs://` bucket ingest | DONE |
`acquire_local_input` + gcs backend already accept `gs://`; preserved when the user opts into a
cloud storage backend (+ ADC). |
docs\PERSONALIZATION_PLAN.md:5:live in the working artifacts
`scratch/PERSONALIZATION_PIVOT_ANALYSIS.md` (the decision-grade
docs\PERSONALIZATION_PLAN.md:6:counter-case) and `scratch/PERSONALIZATION_FEATURE_DESIGN.md`
(the code-grounded design); this doc is
docs\PERSONALIZATION_PLAN.md:151:- `scratch/PERSONALIZATION_PIVOT_ANALYSIS.md` ? the decision-
grade strategy analysis (pros/cons, harness-fit, counter-case, recommendation).
docs\PERSONALIZATION_PLAN.md:152:- `scratch/PERSONALIZATION_FEATURE_DESIGN.md` ? the code-
grounded design (schema, serving bridge, APIs, critical files).
docs\PER_VIDEO_WORKSPACE.md:12:[Omitted long matching line]
docs\PER_VIDEO_WORKSPACE.md:26:| D3 | **Two roots:** a *durable* workspace root (may be `gs://`)
for artifacts to keep; a *fast local compute-scratch* mirror (always local) for frame
caches/chunks. Compute writes locally, then **pushes durable artifacts** to the workspace. |
design + back-compat | WSL/native frame extraction never writes to a cloud root (perf +
correctness). |
docs\PER_VIDEO_WORKSPACE.md:36:| Working video copy (cloud fetch) / source | `scratch/in.mp4`,
`~/clips/<key>.<ext>` | video_id | `source/` (copy) or `source/source.uri` (local pointer) |
docs\PER_VIDEO_WORKSPACE.md:39:| Frame PNG cache | `~/clips/<key>_frames/`,
`<scratch>/<key>_frames/` | video_id | `tmp/` (**local-only**) |

[Showing results with pagination = limit: 40]

30. user (2026-07-02T02:52:48.693Z)

388   refactor: migrate async-job plumbing (_get_executor/_drain_due_jobs/_arm_wake_timer) out
of main.py
384   ci: single-source the shared offline guard lanes + coverage floor via a workflow_call
reusable
349   WASB native: decode is CPU-bound & single-threaded in streaming mode ? L4 GPU ~80% idle
(no NVDEC/parallel decode)
340   [cost] /api/process re-runs the paid segmenter for an already-processed video ?

refresh/"Try again" re-bills Gemini
332   [safety] Content-safety pre-flight litmus: flag non-sports / NSFW video BEFORE sending
it to Gemini
331   [perf] Hosted processing is CPU-bound (ffmpeg/cv2) ? measured per-phase breakdown +
levers (GPU/NVENC, parallel extraction, async compile)
330   [optimization] MD5 segmentation reuse ? skip Gemini re-segmentation when the same video
(video_id) is re-uploaded
328   Surface the per-video runlog / execution trace to the user (endpoint + UI), not just
server logs
327   Allow gs:// / s3:// / gdrive-api:// ingest on an exposed (jailed) box ? scheme-exempt
recognized storage backends from the path-jail
326   Hosted-app observability: request tracing + replay for debugging production CUJ failures
301   T11 follow-up: run the upload retention sweep as an automated scheduled job (hosted
alpha)
175   Shared cloud container (OneDrive) for GPU-extractor golden data ? sync GPU box ? dev
sessions
172   Calibrate the ship-gate F1@tIoU target (currently provisional ? NOT owner-ratified)

31. user (2026-07-02T02:53:52.519Z)

1   """Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).
2
3   A deployment profile (``truly-local`` / ``cloud-serving`` / ``cpu-mock``) implies a
coherent
4   runtime. These checks **fail/warn fast at process startup** ? a loud signal *before* any
5   (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7   * **cloud-serving with a non-cloud storage backend** ? a hard **error** (the one
config bug we
8   can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9   bucket can never serve). This is the README §4.3 "loud error before any job".
10   * a **GPU mismatch** (e.g. ``device='cuda'`` on a box where CUDA isn't visible, or
``device='auto'``
11   falling back to CPU) ? a **warning**, never an error: the probe is best-effort
(torch may live
12   only in the detector subprocess env ? see ``presets.probe_cuda_available``), and the
detector's
13   own healthcheck is the real authority. A warning surfaces it early without false-
failing.
14   * **creds that can't be confirmed** (no ``GOOGLE_APPLICATION_CREDENTIALS`` / not on
Cloud Run) ?
15   a **warning**: Application Default Credentials can come from the metadata server /
gcloud, which
16   we can't probe offline, so the storage backend's own init is the authority; we just
flag it.
17
18   **Exposure-safety posture (server only, ``include_exposure=True``).** When the engine is
exposed
19   behind an edge gate (the owner flips ``security.edge_auth_enabled`` on for the
Cloudflare-Access
20   boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the
safety story.
21   A half-configured exposure starts *looking* healthy then leaks / 500s per request, so we
verify the
22   rest of the posture at **boot**: the ``allowed_video_root`` path-jail is set (else an
authenticated
23   tenant could read an arbitrary local file ? **error**), the CF team-domain + AUD are
present (else
24   no token can verify ? **error**, lifting the current first-request failure to boot), and
the
25   ``auth`` extra is importable (else every request 500s ? **warning**). Only the HTTP
server passes
26   ``include_exposure=True``; the worker reads ``--video-uri`` (not untrusted tenant
paths), so it is

```
27      never "exposed" and stays unaffected.
28
29      **DEFAULT-OFF (both dimensions):** with no profile selected (``profile == ""``) the
profile checks are
30      a strict **no-op** ? exactly the pre-M5 behaviour; with ``edge_auth_enabled=False``
(today's default)
31      the exposure checks are a strict **no-op** too. So nothing changes for today's manual-
knob deployments.
32      """
33
34      from __future__ import annotations
35
36      import logging
37      import os
38      from dataclasses import dataclass
39      from typing import Any, Mapping, Optional
40
41      LEVEL_ERROR = "error"
42      LEVEL_WARN = "warn"
43
44      # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
45      _CLOUD_STORAGE = ("gcs", "s3")
46      # STRONG env signals that Google ADC *might* resolve. Deliberately NOT
GOOGLE_CLOUD_PROJECT /
47      # CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a
config dir
48      # with no active login), so they would suppress the heads-up falsely. (s3 creds are
intentionally
49      # left to boto3's default chain ? no parallel heads-up, by design.)
50      _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")
51
52
53      @dataclass(frozen=True)
54      class StartupCheck:
55          """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
up)."""
56
57          level: str
58          code: str  # stable, greppable code (e.g. "STORAGE_INCOHERENT")
59          message: str
60
61
62      class StartupCheckError(RuntimeError):
63          """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL
check."""
64
65          def __init__(self, failures: list[StartupCheck]):
66              self.failures = failures
67              super().__init__(
68                  "deployment profile startup check failed: "
69                  + "; ".join(f"[{c.code}] {c.message}" for c in failures)
70              )
71
72
73      def _creds_discoverable(env: Mapping[str, str]) -> bool:
74          """True if *some* Google ADC source is hinted by the environment. Best-effort ? a
False here
75          only downgrades to a WARNING, never a hard failure (the storage init is the
authority)."""
76          return any(env.get(k) for k in _GCP_ENV_HINTS)
77
78
79      def _auth_extra_available() -> bool:
80          """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it
(find_spec),
```

```
81          so this stays cheap + side-effect-free. Module-level so tests can monkeypatch it
deterministically
82          (whether PyJWT is installed in the test env must not change the WARN's presence)."""
83          try:
84              import importlib.util
85
86              return importlib.util.find_spec("jwt") is not None
87          except (
88              Exception
89          ):  # pragma: no cover - find_spec only raises on a broken parent package
90              return False
91
92
93      def _exposure_checks(config: Any, env: Mapping[str, str]) -> list[StartupCheck]:
94          """Exposure-safety posture for an *exposed* HTTP server. Gated on
``security.edge_auth_enabled``
95          (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ?
the "I am
96          exposing this engine behind the CF-Access gate" signal ? the gate alone isn't
enough: verify the
97          rest of the posture at boot so a half-configured exposure fails fast instead of
leaking / 500-ing
98          per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe); ``env``
is the
99          caller's environment view (so the host-allowlist check is testable)."""
100         sec = getattr(config, "security", None)
101         if not sec or not getattr(sec, "edge_auth_enabled", False):
102             return []  # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical
to today).
103
104         out: list[StartupCheck] = []
105
106         # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process:
WITHOUT it,
107         # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset
= "any local
108         # file", the single-user default). Fatal for an exposed engine.
109         root = getattr(sec, "allowed_video_root", None)
110         if not root or not str(root).strip():
111             out.append(
112                 StartupCheck(
113                     LEVEL_ERROR,
114                     "EXPOSED_NO_VIDEO_ROOT",
115                     "security.edge_auth_enabled is ON (engine exposed) but
security.allowed_video_root is "
116                     "unset ? an authenticated tenant could make /api/process read an
arbitrary local file. "
117                     "Set allowed_video_root to a jail directory before exposing the
engine.",
118                 )
119             )
120
```

32. user (2026-07-02T02:53:52.876Z)

```
1       """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3       A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4       config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5       knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7       Precedence (lowest ? highest), realised by the loader::
8
9           built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set
```

```
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55              "storage": {"backend": "gcs"},
56          },
57          "cpu-mock": {
58              "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59              # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61              "storage": {"backend": "local"},
62          },
63      }
```

```

    def is_known_profile(profile: str) -> bool:
        """True for a non-empty, recognised profile name."""
        return bool(profile) and profile in PROFILE_PRESETS


    def preset_for(profile: str) -> dict[str, Any]:
        """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
        module-level table), or ``{}`` for ``""`` / an unknown profile."""
        return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))


    def _truthy(val: Optional[str]) -> bool:
        """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
        if val is None:
            return False
        return val.strip().lower() in {"1", "true", "yes", "on"}


    def _cuda_available() -> bool:
        """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
        ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
        try:
            # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
            # and optional). Only reached when a caller opts into probe_cuda.
            import torch

            return bool(torch.cuda.is_available())
        except Exception:
            return False


    def probe_cuda_available() -> bool:
        """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).

        Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
        that *wants* the GPU heads-up can pass the result in without the startup module
importing
        torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
        (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
        treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
        error that could false-fail."""
        return _cuda_available()


    def suggest_profile(
        env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
    ) -> str:
        """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.

        Signals, in priority order:

          - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
          - ``CI`` truthy                        ? ``"cpu-mock"``
          - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
          - otherwise                            ? ``""`` (no strong suggestion)
```

```
120
121         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
122         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
123         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
124         """
125         e = os.environ if env is None else env
126         if e.get("K_SERVICE"):
127             return "cloud-serving"
128         if _truthy(e.get("CI")):
129             return "cpu-mock"
130         if probe_cuda and _cuda_available():
131             return "truly-local"
132         return ""
133
```

33. user (2026-07-02T02:54:01.028Z)

```
backend\config\models.py:335:    # contain upload_dir or /api/process will reject the uploaded
paths.
backend\config\models.py:336:    upload_dir: str = "output/uploads"
backend\utils\freespace.py:4:``upload_dir``), a **fetch-to-scratch** of a bucket/Drive source (a
full local copy under the temp
backend\utils\freespace.py:32:    ancestor (an ``upload_dir``/scratch target may not exist yet
at guard time)."""
backend\api\uploads.py:5:under security.upload_dir, then submitted to /api/process by the
returned path.
backend\api\uploads.py:7:aborts partial uploads and the client re-inits. Per-user partitioning
of upload_dir
backend\api\uploads.py:50:    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
backend\api\uploads.py:57:    return upload_dir, max_bytes, part_bytes, exts
backend\api\uploads.py:73:    upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
backend\api\uploads.py:89:    os.makedirs(upload_dir, exist_ok=True)
backend\api\uploads.py:93:    require_free_bytes(upload_dir, req.total_size_bytes,
stage="upload")
backend\api\uploads.py:95:    tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
backend\api\uploads.py:160:    upload_dir, _, _, _ = _upload_cfg()
backend\api\uploads.py:170:    final_path = os.path.join(upload_dir, st["filename"])
backend\api\uploads.py:175:        final_path = os.path.join(upload_dir,
f"{stem}-{upload_id[:8]}{ext}")
```

34. user (2026-07-02T02:54:01.823Z)

```
28:    "fastapi>=0.111.0",
52:    "httpx",        # fastapi TestClient transport
```

35. assistant (2026-07-02T02:54:26.986Z)

```
All hot-path files reviewed. Compiling verified findings.
```

36. user (2026-07-02T02:55:42.054Z)

```
Structured output provided successfully
```